



FRONT-END AVANÇADO

JavaScript na Prática

Construindo uma Single Page Application do zero,
sem frameworks — do primeiro `console.log` ao
consumo de APIs reais.

Apostila do curso · Projeto **App Livros**

Sumário

1. Capítulo 0 — Boas-vindas
2. Capítulo 1 — Variáveis, tipos e decisões
3. Capítulo 2 — Arrays e Objetos
4. Capítulo 3 — Funções
5. Capítulo 4 — Loops: repetição com controle
6. Capítulo 5 — Manipulando o DOM e reagindo a eventos
7. Capítulo 6 — O que é uma SPA, URL, URI e o `hash`
8. Capítulo 7 — Renderização dinâmica com `hash` (o primeiro roteador)
9. Capítulo 8 — Componentização com ES Modules
10. Capítulo 9 — Páginas como objetos e o menu dinâmico
11. Capítulo 10 — Centralizando o roteamento
12. Capítulo 11 — Estilo com um microframework CSS e temas
13. Capítulo 12 — Programação assíncrona: callbacks e o "callback hell"
14. Capítulo 13 — Promises e o `fetch`
15. Capítulo 14 — `async`/`await` e a camada de serviços
16. Capítulo 15 — Formulário de cadastro com a API ViaCEP
17. Capítulo 16 — Consumindo a API de Livros (lista, paginação e modal)
18. Capítulo 17 — Reaproveitando o padrão: a API do Rick and Morty
19. Capítulo 18 — Boas práticas, organização e próximos passos
20. Capítulo 19 — Guardando dados no navegador: armazenamentos, cache e estratégias
21. Capítulo 20 — Service Workers e PWA: o app que funciona offline
22. Apêndice A — Referência rápida
23. Apêndice B — Gabarito comentado dos exercícios

Capítulo 0 — Boas-vindas

O que vamos construir juntos

Se você está começando agora no mundo do desenvolvimento web, provavelmente já ouviu falar de nomes como React, Vue ou Angular. São ferramentas poderosas — mas elas escondem, atrás de muita “mágica”, conceitos que **todo bom desenvolvedor precisa dominar**. A proposta deste curso é justamente o contrário: vamos abrir o capô do carro e entender o motor.

Ao longo desta apostila vamos construir, do zero e **sem nenhum framework**, uma aplicação chamada **App Livros**. Ela é uma *SPA* (Single Page Application, ou “aplicação de página única”). Na prática, isso significa uma aplicação que carrega **uma única página HTML** e, a partir daí, troca o conteúdo na tela dinamicamente conforme o usuário navega — sem nunca recarregar a página inteira.

O App Livros terá:

- Um **menu de navegação** gerado automaticamente por JavaScript;
- Várias **páginas** (Home, Sobre, Serviços, Contato, Cadastro, Livros, Rick and Morty), cada uma sendo um pequeno módulo independente;
- Um **roteador** próprio, que decide qual página mostrar com base na URL;
- **Formulários** que reagem ao usuário — inclusive um que busca um endereço a partir do CEP consultando uma API pública;
- **Telas alimentadas por APIs reais**, com listagem, paginação e uma janela de detalhes (modal);
- Um **microframework de CSS** próprio, com suporte a temas (dia, tarde, noite).

Tudo isso usando apenas **HTML, CSS e JavaScript puro** (também chamado de “vanilla JavaScript”). Quando você terminar, entenderá o que um framework faz por você — e será capaz de aprender qualquer framework muito mais rápido, porque os fundamentos estarão sólidos.

A filosofia do curso: montar um quebra-cabeça

Ninguém aprende a construir uma casa começando pelo telhado. Por isso, este livro é **incremental**. Cada capítulo entrega uma peça, e essa peça se encaixa nas anteriores. Veja o mapa da jornada:

Fundamentos da linguagem

variáveis → condicionais → arrays/objetos → funções → loops



Interagindo com a página

DOM → eventos → formulários



Estruturando a aplicação

SPA → hash → roteador → módulos → componentes



Trazendo dados do mundo real

assíncrono → callbacks → promises → fetch → async/await → APIs

Repare que **só chegamos às APIs no final**. Isso é proposital: consumir uma API exige entender funções, objetos, loops, o DOM e o assincronismo. Se tentássemos começar pelo “legal” (dados vindos da internet), você ficaria perdido. Vamos construir a fundação primeiro.



Nos bastidores O projeto real foi desenvolvido *aula a aula, aos sábados*. O histórico de commits do repositório conta essa evolução: primeiro vieram exercícios de fundamentos, depois as funções que geram páginas, depois o menu dinâmico, a centralização do roteamento e, por fim, o consumo de APIs. Esta apostila segue exatamente essa mesma ordem.

Preparando o ambiente

Você precisa de pouca coisa para começar. Nada de instalar frameworks ou gerenciadores de pacote complicados.

1. Um editor de código

Recomendamos o **Visual Studio Code** (gratuito). Baixe em <https://code.visualstudio.com>. Instale também a extensão **Live Server** — ela sobe um servidor local com um clique, e vamos precisar disso.

2. Um navegador com boas ferramentas

Use **Google Chrome**, **Microsoft Edge** ou **Firefox**. Vamos usar bastante o **Console do navegador** (as *DevTools*). Você abre pressionando **F12** ou clicando com o botão direito na página → *Inspecionar* → aba *Console*.

O Console é o seu laboratório. É onde você vê mensagens de `console.log()`, testa trechos de código e descobre erros. Abra-o agora e digite:

```
console.log("Olá, mundo!");  
2 + 2;
```

Pressione `Enter` depois de cada linha. Viu a resposta aparecer? Esse será o seu companheiro durante todo o curso.

3. Por que preciso de um servidor local?

Mais adiante, o projeto usará **ES Modules** (arquivos JavaScript que importam uns aos outros com `import / export`). Por segurança, os navegadores **não permitem** que módulos sejam carregados quando você abre o arquivo direto pelo sistema (endereços que começam com `file://`). É preciso servir os arquivos por HTTP.

Há duas formas simples:

Opção A — Live Server (a mais fácil): No VS Code, clique com o botão direito no arquivo `index.html` e escolha *Open with Live Server*. O navegador abre sozinho, geralmente em `http://127.0.0.1:5500`.

Opção B — Terminal:

```
# Se você tem Python instalado:  
python -m http.server 8000  
  
# Ou, se tem Node.js:  
npx serve
```

Depois é só acessar `http://localhost:8000` (ou a porta indicada).

 **Cuidado** Se em algum momento os `import` pararem de funcionar com um erro parecido com “CORS policy” ou “Cross origin requests are only supported for HTTP”, é quase certo que você abriu o arquivo com `file://` em vez de usar um servidor. Volte para o Live Server.

A estrutura final do projeto

Só para você ter uma visão do destino (não se assuste, vamos criar cada arquivo com calma), este é o formato que o projeto terá ao final:

```
app_livros/
├── index.html           # a única página HTML da SPA
├── src/
│   ├── css/
│   │   └── microframework.css # nosso "mini Bootstrap" com temas
│   ├── img/             # imagens usadas nas páginas
│   └── js/
│       ├── main.js      # ponto de entrada: inicia tudo
│       ├── components/
│       │   ├── navbar/navbar.js # o menu de navegação
│       │   ├── footer/footer.js
│       │   ├── rotas/rotas.js   # a lista de rotas da aplicação
│       │   └── paginas/         # cada página é um arquivo
│       │       ├── home.js
│       │       ├── sobre.js
│       │       ├── servicos.js
│       │       ├── contato.js
│       │       ├── formCad.js   # cadastro com busca de CEP
│       │       ├── livros.js    # tela alimentada por API
│       │       └── rickandmorty.js # outra tela alimentada por API
│       ├── services/
│       │   ├── api.js          # a camada que conversa com as APIs
│       │   ├── livros.js       # função genérica de requisição
│       │   ├── rickandmorty.js
│       │   └── loading.js      # indicador de "carregando..."
└── readme.md
```

Não decore isso. A cada capítulo você vai entender por que cada pasta existe. Por enquanto, guarde só uma ideia: **cada arquivo tem uma responsabilidade única**. É esse princípio que mantém projetos grandes sob controle.

O que você precisa saber antes de começar

Este curso é de **Front-End Avançado**, mas “avançado” aqui significa que vamos aprofundar JavaScript — não que você precisa ser um expert. O ideal é que você já tenha tido contato com:

- **HTML básico:** tags como `<div>`, `<h1>`, `<p>`, `<form>`, `<input>`;
- **CSS básico:** seletores, cores, um pouco de layout;

- Nenhuma experiência prévia com JavaScript é obrigatória — começamos do começo no Capítulo 1.

Se você travar em algum ponto de HTML ou CSS, a [documentação da MDN](#) é a melhor referência gratuita em português.

Recapitulando

- Vamos construir uma **SPA** completa, do zero, sem frameworks.
- O aprendizado é **incremental**: cada capítulo encaixa uma peça nova.
- Você precisa de um **editor** (VS Code), um **navegador** e um **servidor local** (Live Server resolve).
- O **Console do navegador** é o seu laboratório — mantenha-o aberto.

No próximo capítulo, começamos pelos tijolos mais básicos da linguagem: variáveis, tipos de dados e as decisões (condicionais). Vamos lá! 🚀

Exercícios do Capítulo 0

1. Instale o VS Code e a extensão Live Server.
2. Crie uma pasta `app_livros`, coloque dentro um arquivo `index.html` com o conteúdo mínimo abaixo e abra-o com o Live Server. Confirme que aparece o texto na tela.

```
<!DOCTYPE html>
<html lang="pt-BR">
<head><meta charset="UTF-8"><title>App Livros</title></head>
<body>
  <h1>Meu primeiro passo!</h1>
  <script>console.log("O JavaScript está rodando.");</script>
</body>
</html>
```

3. Abra o Console (`F12`) e confirme que a mensagem *"O JavaScript está rodando."* apareceu.
4. Ainda no Console, calcule quantos segundos há em um dia digitando `24 * 60 * 60`.

Capítulo 1 — Variáveis, tipos e decisões

Todo programa faz basicamente três coisas: **guarda informações**, **toma decisões** com base nelas e **repete tarefas**. Neste capítulo cobrimos as duas primeiras. A repetição fica para o Capítulo 4.

Abra o Console do navegador (**F12** → aba *Console*). Vamos experimentar tudo ao vivo.

1.1 Variáveis: caixas com etiqueta

Uma **variável** é um espaço na memória onde guardamos um valor e ao qual damos um nome. Pense numa caixa com uma etiqueta: a etiqueta é o nome, o conteúdo é o valor.

Em JavaScript moderno declaramos variáveis com **let** e **const** :

```
let idade = 18;
const nome = "Maria";
```

- **let** cria uma variável cujo valor **pode mudar** depois.
- **const** cria uma **constante**: depois de definido, o valor **não pode ser reatribuído**.

```
let pontos = 10;
pontos = 20;           // ✔ permitido, pontos agora é 20

const pi = 3.14;
pi = 3;                // ✖ erro: Assignment to constant variable
```

 **Nos bastidores** Existe também a palavra **var** , mais antiga. Ela ainda funciona, mas tem comportamentos confusos de escopo (veremos “hoisting” no Capítulo 3). A regra prática moderna é: **use **const** por padrão** e troque para **let** apenas quando precisar reatribuir o valor. Evite **var** .

Regras e boas práticas para nomes

- Não pode começar com número: **1nome** é inválido, **nome1** é válido.
 - Não use espaços nem acentos. Use **camelCase**: **nomeDoUsuario** , **totalDePaginas** .
 - Escolha nomes que **descrevem** o conteúdo. **x** não diz nada; **precoTotal** diz tudo.
-

1.2 Tipos de dados

O valor guardado numa variável tem um **tipo**. Os que mais usaremos:

Tipo	O que representa	Exemplo
<code>string</code>	texto	<code>"Maria"</code> , <code>'Olá'</code> , <code>`oi`</code>
<code>number</code>	números (inteiros ou não)	<code>18</code> , <code>3.14</code> , <code>-7</code>
<code>boolean</code>	verdadeiro ou falso	<code>true</code> , <code>false</code>
<code>undefined</code>	“sem valor definido”	<code>let x;</code>
<code>null</code>	“vazio intencional”	<code>let y = null;</code>
<code>object</code>	coleção estruturada	<code>{ nome: "Ana" }</code> (Cap. 2)
<code>array</code>	lista ordenada	<code>[1, 2, 3]</code> (Cap. 2)

Você pode descobrir o tipo de qualquer valor com o operador `typeof`:

```
typeof "Maria";    // "string"
typeof 18;         // "number"
typeof true;       // "boolean"
typeof undefined; // "undefined"
```

Strings e template strings

Textos podem ser escritos com aspas simples (`'...'`) ou duplas (`"..."`). Mas há uma terceira forma **muito importante** para nós: as *template strings*, com crase (```). Elas permitem **interpol**ar valores dentro do texto usando `${...}`:

```
const nome = "Maria";
const idade = 28;

// jeito antigo, com concatenação (+):
console.log("Olá, " + nome + "! Você tem " + idade + " anos.");

// jeito moderno, com template string:
console.log(`Olá, ${nome}! Você tem ${idade} anos.`);
```

As duas linhas produzem o mesmo resultado, mas a segunda é bem mais legível. E tem outra vantagem: a template string pode ocupar **várias linhas**.

🧩 **Montando o quebra-cabeça** Guarde bem as template strings. Mais para a frente, **toda a interface do App Livros** será construída com elas. Cada página vai retornar um texto gigante com HTML dentro, misturando marcação e valores dinâmicos com `${...}`. É o alicerce de tudo o que faremos no DOM.

1.3 Operadores de comparação

Para tomar decisões, precisamos **comparar** valores. As comparações resultam sempre em um booleano (`true` ou `false`):

Operador	Significado	Exemplo (<code>let a=5</code>)	Resultado
<code>==</code>	igual a (sem checar o tipo)	<code>a == "5"</code>	<code>true</code>
<code>===</code>	igual a e do mesmo tipo	<code>a === "5"</code>	<code>false</code>
<code>!=</code>	diferente de	<code>a != 3</code>	<code>true</code>
<code>!==</code>	diferente estrito (valor ou tipo)	<code>a !== "5"</code>	<code>true</code>
<code><</code>	menor que	<code>a < 10</code>	<code>true</code>
<code>></code>	maior que	<code>a > 10</code>	<code>false</code>
<code><=</code>	menor ou igual	<code>a <= 5</code>	<code>true</code>
<code>>=</code>	maior ou igual	<code>a >= 6</code>	<code>false</code>

⚠ **Cuidado — a diferença entre `==` e `===`** O `==` faz “conversão de tipo” antes de comparar, o que gera surpresas: `0 == ""` é `true`, `1 == true` é `true`. Isso confunde e esconde bugs. **Use sempre `===` e `!==`**. No projeto inteiro só usamos comparação estrita. Grave essa regra.

1.4 Condicionais: `if` / `else`

Condicionais são estruturas que testam se uma expressão é verdadeira e, com base nisso, decidem qual bloco de código executar.

```
let idade = 18;

if (idade >= 18) {
  console.log("Maior de idade.");
} else {
  console.log("Menor de idade.");
}
```

Lê-se: “SE a idade for maior ou igual a 18, mostre ‘Maior de idade’; SENÃO, mostre ‘Menor de idade’.”

Quando há mais de duas possibilidades, encadeamos com `else if`:

```
let nota = 7;

if (nota >= 9) {
  console.log("Excelente");
} else if (nota >= 7) {
  console.log("Aprovado");
} else if (nota >= 5) {
  console.log("Recuperação");
} else {
  console.log("Reprovado");
}
```

O JavaScript testa as condições **de cima para baixo** e para na primeira que for verdadeira.

1.5 Operadores lógicos e a tabela verdade

Muitas vezes uma decisão depende de **mais de uma condição** ao mesmo tempo. Para combiná-las usamos os operadores lógicos:

- `&&` (**E / AND**): verdadeiro apenas se **ambos** os lados forem verdadeiros.
- `||` (**OU / OR**): verdadeiro se **pelo menos um** lado for verdadeiro.
- `!` (**NÃO / NOT**): inverte o valor.

A **tabela verdade** resume todas as combinações possíveis:

A	B	A && B	A B	!A
true	true	true	true	false
true	false	false	true	false
false	true	false	true	true
false	false	false	false	true

Um exemplo clássico: uma pessoa só pode entrar no bar se tiver 18 anos e possuir documento.

```
let idade = 20;
let possuiDocumento = true;

if (idade >= 18 && possuiDocumento) {
  console.log("Pode entrar no bar.");
} else {
  console.log("Não pode entrar no bar.");
}
```

A tabela verdade nos ajuda a **prever** o comportamento antes mesmo de rodar o código. Se a idade fosse 20 (`true`) mas `possuiDocumento` fosse `false`, a linha do `&&` teria `true && false`, que é `false` — logo, não entra.

1.6 O operador ternário

Quando a decisão é simples (“se sim, um valor; se não, outro valor”), existe um atalho: o **operador ternário**. Sua forma é:

```
condição ? valorSeVerdadeiro : valorSeFalso
```

```
let idade = 20;
const podeEntrar = idade >= 18 ? "Pode entrar" : "Não pode entrar";
console.log(podeEntrar); // "Pode entrar"
```

Lê-se: “idade é maior ou igual a 18? Se sim, ‘Pode entrar’; senão, ‘Não pode entrar’.” O `?` é a pergunta, o `:` separa as duas respostas.

🧩 **Montando o quebra-cabeça** O ternário vai aparecer bastante no projeto para pequenas escolhas dentro do HTML. Por exemplo, na tela de livros decidimos se um botão de paginação fica desabilitado assim: `botao.disabled = paginaAtual <= 1;`. É a mesma ideia de decidir um valor a partir de uma condição.

1.7 `switch` / `case`

Quando comparamos **uma mesma variável** contra vários valores possíveis, uma cadeia de `else if` fica cansativa. O `switch` organiza isso:

```
let diaDaSemana = 3;
let mensagem;

switch (diaDaSemana) {
  case 1:
    mensagem = "Segunda-feira";
    break;
  case 2:
    mensagem = "Terça-feira";
    break;
  case 3:
    mensagem = "Quarta-feira";
    break;
  default:
    mensagem = "Outro dia";
}

console.log(mensagem); // "Quarta-feira"
```

Pontos de atenção:

- Cada `case` compara a expressão com um valor (usando comparação estrita).
- O `break` é essencial: sem ele, a execução “vaza” para o próximo caso.
- O `default` é opcional e roda quando nenhum caso corresponde.

💡 **Nos bastidores** — `switch(true)` Existe um truque elegante: usar `switch (true)` e colocar expressões booleanas completas em cada `case`. Isso permite condições combinadas, quase como um `if/else if`, mas mais organizado:

```
switch (true) {
  case temperatura === "quente" && sol:
    mensagem = "Dia de praia!";
    break;
  case temperatura === "frio":
    mensagem = "Leve um casaco.";
    break;
}
```

1.8 Ligando os pontos com o projeto

Você deve estar se perguntando: “o que decisões têm a ver com trocar de página numa SPA?” Tudo. Adiante, o **primeiro roteador** que vamos escrever será literalmente uma cadeia de `if/else` decidindo qual conteúdo mostrar com base na URL:

```
const hash = window.location.hash;

if (hash === "#home") {
  app.innerHTML = home;
} else if (hash === "#sobre") {
  app.innerHTML = sobre;
} else if (hash === "#contato") {
  app.innerHTML = contato;
}
```

Não entenda tudo agora — repare apenas que é **exatamente** o `if/else` deste capítulo, usando comparação estrita (`===`). Quando chegarmos lá, esse código vai parecer familiar. Depois, no Capítulo 10, vamos aposentar esses `if` em favor de uma solução mais elegante. Mas o começo é aqui.

Recapitulando

- Declare variáveis com `const` (padrão) e `let` (quando mudar).
- Os tipos essenciais são `string`, `number`, `boolean`, além de `null` e `undefined`.

- **Template strings** (``...${valor}...``) serão a base de toda a nossa interface.
- Compare sempre com `===` e `!==` .
- Combine condições com `&&` , `||` e `!` , guiando-se pela **tabela verdade**.
- Decida com `if/else` , o **ternário** e o `switch` .

Exercícios do Capítulo 1

1. Declare `const nome` com o seu nome e `let idade` com sua idade. Use uma template string para imprimir: *“Olá, sou <nome> e tenho <idade> anos.”*
2. Escreva um `if/else if/else` que classifica uma nota (0 a 10) em “Aprovado” (≥ 7), “Recuperação” (≥ 5) ou “Reprovado”.
3. **Aluno aprovado (baseado na tabela verdade):** um aluno é aprovado se tem nota mínima 7.0 **E** no máximo 20 faltas. Escreva o `if` e teste as quatro combinações possíveis de (nota alta/baixa) $\times$ (faltas ok/altas).
4. **Segurança do banco:** o acesso só é liberado se o nome for válido **E** a senha estiver correta. Simule com duas variáveis booleanas e imprima “Acesso permitido” ou “Acesso negado”.
5. Reescreva o exercício 2 usando `switch(true)` .
6. Transforme a decisão do exercício 4 em uma única linha com o operador ternário.

Capítulo 2 — Arrays e Objetos

Uma variável guarda **um** valor. Mas o mundo real é feito de **coleções**: uma lista de compras, os alunos de uma turma, os livros de uma estante. Para representar isso, o JavaScript oferece duas estruturas fundamentais: os **arrays** e os **objetos**. Domine essas duas e você terá a chave para praticamente tudo o que faremos com dados no App Livros.

2.1 Arrays: coleções ordenadas

Um **array** é uma coleção **ordenada** de elementos. Imagine uma fila de caixas, onde cada caixa guarda um item — que pode ser um número, um texto, outro array ou até um objeto.

```
const frutas = ["maçã", "pêra", "morango"];
```

Cada item ocupa uma **posição** (o “índice”), e a contagem **começa em zero**:

```
índice:      0        1        2
            ["maçã",  "pêra",  "morango"]
```

Para acessar um item, usamos colchetes com o índice:

```
frutas[0]; // "maçã"
frutas[2]; // "morango"
frutas[3]; // undefined (essa posição não existe)
```

Duas propriedades úteis desde já:

```
frutas.length; // 3 → quantos elementos há
frutas[frutas.length - 1]; // "morango" → o último elemento
```

 **Cuidado** O índice do **último** elemento é sempre **length - 1**, nunca **length**. Como a contagem começa em 0, um array de 3 itens tem índices 0, 1 e 2. Esse é um dos erros mais comuns de quem começa.

2.2 Adicionando e removendo elementos

Arrays são dinâmicos: crescem e encolhem. Pense numa **fila de pessoas** — dá para adicionar ou remover gente do início ou do fim.

Adicionar

```
const numeros = [1, 2, 3];

numeros.push(4);    // adiciona no FIM → [1, 2, 3, 4]
numeros.unshift(0); // adiciona no INÍCIO → [0, 1, 2, 3, 4]
```

Remover

```
const numeros = [0, 1, 2, 3, 4];

numeros.pop();    // remove do FIM e retorna 4 → [0, 1, 2, 3]
numeros.shift();  // remove do INÍCIO e retorna 0 → [1, 2, 3]
```

Um jeito de memorizar:

Método	Ponta que mexe	Efeito
<code>push()</code>	fim	adiciona
<code>pop()</code>	fim	remove
<code>unshift()</code>	início	adiciona
<code>shift()</code>	início	remove

`pop()` e `shift()` **retornam** o elemento removido, então você pode guardá-lo:

```
const atendido = fila.shift(); // remove o primeiro e guarda quem foi atendido
```

2.3 Copiando um pedaço: `slice()`

O método `slice(início, fim)` retorna uma **cópia** de um pedaço do array, do índice `início` até **antes** do índice `fim` (o fim não é incluído). O array original **não é alterado**.

```
let numeros = [1, 2, 3, 4, 5];
let parte = numeros.slice(1, 3); // pega índices 1 e 2 → [2, 3]

console.log(parte); // [2, 3]
console.log(numeros); // [1, 2, 3, 4, 5] (intacto)
```

🧩 **Montando o quebra-cabeça** O `slice` aparece de verdade no App Livros. No serviço de livros, a API devolve uma longa lista de assuntos de cada obra, e pegamos só os **cinco primeiros** para não poluir a tela:

```
dados.subjects.slice(0, 5).join(', ');
```

Ou seja: “copie do índice 0 até antes do 5” e depois junte tudo num texto com vírgulas. Guarde essa linha — vamos reencontrá-la no Capítulo 16.

2.4 Objetos: coleções com etiquetas

Se o array é bom para uma **lista** de itens semelhantes, o **objeto** é ideal para descrever **uma única coisa** com várias características.

Enquanto no array acessamos por posição numérica, no objeto acessamos por **nome** (a “chave”). Objetos usam **chaves** `{ }`:

```
const pessoa = {
  nome: "Maria",
  idade: 28,
  falar: function () {
    console.log("Olá, sou " + this.nome);
  }
};
```

Um objeto agrupa duas coisas:

- **Propriedades** (os dados): `nome` e `idade` — “*coisas que a pessoa tem*”.
- **Métodos** (comportamentos): `falar` — “*coisas que a pessoa faz*”. Um método nada mais é do que uma propriedade cujo valor é uma função.

💡 **Nos bastidores — o `this`** Dentro de `falar`, escrevemos `this.nome`. A palavra `this` se refere ao **próprio objeto** em que o método está. Então `this.nome` é o `"Maria"` do objeto atual. Por enquanto, basta saber que `this` é “eu mesmo, o objeto”.

Acessando propriedades

Há duas notações:

```
 pessoa.nome;           // notação de PONTO   → "Maria"
 pessoa["idade"];       // notação de COLCHETE → 28
```

A notação de ponto é a mais comum e legível. A de colchetes é útil quando o nome da propriedade está guardado numa variável (veremos isso com `for...in` no Capítulo 4).

Adicionando e modificando propriedades

Objetos também são dinâmicos:

```
const pessoa = { nome: "Maria", idade: 28 };

pessoa.profissao = "Desenvolvedora"; // adiciona uma nova propriedade
pessoa.idade = 30;                   // modifica uma existente

console.log(pessoa);
// { nome: "Maria", idade: 30, profissao: "Desenvolvedora" }
```

2.5 O casamento perfeito: arrays de objetos

Aqui está a estrutura mais importante de todo o desenvolvimento web. Na prática, quase sempre lidamos com **listas de coisas**, e cada “coisa” tem vários atributos. A solução é um **array de objetos**:

```
const livros = [
  { titulo: "PHP",      autor: "ABCD", ano: 1995 },
  { titulo: "JavaScript", autor: "A1B2", ano: 1995 },
  { titulo: "Python",   autor: "XYZ",  ano: 1991 }
];
```

Para acessar um dado específico, combinamos as duas notações: primeiro o índice do array, depois a propriedade do objeto.

```
livros[1].titulo; // "JavaScript"
livros[0].ano;    // 1995
livros[2].autor;  // "XYZ"
```

Leia com calma: `livros[1]` seleciona o **segundo objeto** da lista; o `.titulo` pega a propriedade `titulo` **daquele** objeto.

🧩 **Montando o quebra-cabeça** Esta é, literalmente, a forma de todos os dados do App Livros. Alguns exemplos reais que vamos construir:

A lista de rotas (o mapa de navegação da SPA) é um array de objetos:

```
const roteador = [
  { url: "#home", label: "Home", pagina: home },
  { url: "#sobre", label: "Sobre", pagina: sobre },
  // ...
];
```

Os cards de serviços também:

```
const detalhes = [
  { titulo: "Jogo das quartas", descricao: "...", imagem: "src/img/2002_1.webp" },
  { titulo: "Jogo especial",    descricao: "...", imagem: "src/img/2002_2.jpg" }
];
```

E os dados que vêm da API: quando pedirmos a lista de livros à internet, vamos transformá-la exatamente neste formato:

```
livros.push({
  id: item.id,
  titulo: item.title,
  autores: extrairAutores(item.authors),
  imagem: extrairImagem(item.formats)
});
```

Ou seja: **tudo o que você vê na tela do App Livros nasce de um array de objetos.** Dominar esta seção é dominar metade do curso.

2.6 Objetos aninhados

Uma propriedade de um objeto pode ser... outro objeto. Isso forma estruturas “aninhadas”, muito comuns em dados vindos de APIs:

```
const personagem = {
  nome: "Rick",
  origem: { name: "Earth (C-137)" },
  localizacao: { name: "Citadel of Ricks" }
};

personagem.origem.name; // "Earth (C-137)"
personagem.localizacao.name; // "Citadel of Ricks"
```

🧩 **Montando o quebra-cabeça** Isso não é invenção: é exatamente o formato que a API do Rick and Morty devolve. No Capítulo 17 vamos escrever `dados.origin.name` e `dados.location.name` para “cavar” dentro desses objetos aninhados. Se você entendeu o exemplo acima, já entendeu aquele código.

2.7 Uma prévia dos métodos que percorrem coleções

Ainda não vamos nos aprofundar (isso é assunto dos capítulos de loops e funções), mas vale plantar a semente. Arrays têm métodos poderosos que **percorrem** todos os elementos para você. O mais emblemático é o `.map()`, que **transforma** cada item e devolve um novo array:

```
const numeros = [1, 2, 3];
const dobrados = numeros.map(n => n * 2); // [2, 4, 6]
```

E o `.join()`, que transforma um array num texto:

```
["Ana", "Bruno", "Carla"].join(", "); // "Ana, Bruno, Carla"
```

🧩 **Montando o quebra-cabeça** O `.map()` é o motor do nosso **menu dinâmico** (Capítulo 9): pegamos o array de rotas e transformamos cada rota num pedacinho de HTML `<li>`. O `.join("")` depois cola tudo num só texto. Sem esses dois métodos, teríamos que escrever o menu à mão, item por item. Com eles, o menu se atualiza sozinho quando adicionamos uma rota nova.

Recapitulando

- **Arrays** (`[]`) são listas ordenadas, acessadas por **índice** (começando em 0).
 - Manipule-os com `push` / `pop` (fim) e `unshift` / `shift` (início); copie pedaços com `slice`.
 - **Objetos** (`{ }`) agrupam **propriedades** (dados) e **métodos** (comportamentos), acessados por **nome**.
 - A estrutura rainha do front-end é o **array de objetos** — a forma de praticamente todos os dados do nosso projeto.
 - Objetos podem ser **aninhados**, como nos dados que virão das APIs.
-

Exercícios do Capítulo 2

1. **Painel de fila (prioridade):** crie um array `fila`. Escreva um código que recebe um número (a idade de uma pessoa): se for **maior que 65**, a pessoa vai para o **início** da fila (`unshift`); senão, vai para o **fim** (`push`). Crie também um “atender”, que remove a pessoa do início (`shift`).
2. **Cadastro de alunos:** monte um array de objetos, cada aluno com `nome`, `curso` e `ano`. Depois: (a) adicione um novo aluno; (b) imprima o nome do segundo aluno; © mostre o `curso` do último aluno da lista.
3. **Lista de tarefas:** crie um array de objetos onde cada tarefa tem `descricao` e `concluida` (booleano). Marque a primeira tarefa como concluída alterando sua propriedade.
4. Dado `const livros = [{titulo:"PHP", ano:1995}, {titulo:"Python", ano:1991}]`, escreva a expressão que acessa o `ano` do livro “Python”.
5. Crie um objeto `endereco` aninhado com uma propriedade `cidade` que é outro objeto contendo `nome` e `estado`. Acesse `endereco.cidade.estado`.

Capítulo 3 — Funções

Se as variáveis são caixas e os arrays/objetos são coleções, as **funções** são as **máquinas** do seu programa. Uma função é um bloco de código reutilizável que executa uma tarefa específica. Em vez de repetir o mesmo trecho dez vezes, você o escreve uma vez dentro de uma função e a “chama” quando precisar.

Funções são o coração da **modularização** — a arte de quebrar um problema grande em pedaços pequenos e organizados. E é justamente isso que faz uma SPA funcionar: **cada página do App Livros será uma função**.

3.1 Anatomia de uma função

Declaramos uma função com a palavra-chave `function`, damos um nome a ela e colocamos o código entre chaves:

```
function saudar() {  
  console.log("Olá! Bem-vindo ao curso.");  
}
```

Isso apenas **define** a função — nada acontece ainda. Para executá-la, temos de **chamá-la** (ou “invocá-la”) pelo nome seguido de parênteses:

```
saudar(); // agora sim: imprime "Olá! Bem-vindo ao curso."
```

Pense na função como uma receita de bolo: escrever a receita não faz o bolo; você precisa **executá-la**.

3.2 Parâmetros e retorno

Funções ficam realmente úteis quando recebem **entradas** (parâmetros) e devolvem **saídas** (retorno). Vamos ver os quatro cenários, do mais simples ao mais completo.

Sem parâmetros e sem retorno

```
function somar() {  
  console.log(5 + 3); // sempre soma os mesmos números  
}  
somar(); // 8
```

Limitada: só sabe somar 5 e 3.

Com parâmetros, sem retorno

```
function somar(num1, num2) {  
  console.log(num1 + num2);  
}  
somar(10, 7); // 17
```

Agora é flexível — recebe os números de fora. `num1` e `num2` são os **parâmetros** (os nomes na definição); `10` e `7` são os **argumentos** (os valores reais na chamada).

Com parâmetros e com retorno

O `console.log` só **mostra** um valor. Na maioria das vezes, queremos que a função **devolva** um resultado para continuarmos usando. Para isso existe o `return`:

```
function somar(num1, num2) {  
  return num1 + num2;  
}  
  
const resultado = somar(5, 4);  
console.log(resultado); // 9
```

A diferença é enorme: com `return`, o valor “sai” da função e pode ser guardado numa variável, usado em outra conta, passado adiante...

 **Cuidado** Assim que o JavaScript encontra um `return`, ele **encerra** a função na hora. Qualquer código depois do `return` (na mesma linha de execução) é ignorado. Se uma função não tem `return`, ela devolve `undefined`.

3.3 Funções de expressão (guardadas em variáveis)

Em JavaScript, funções são “valores de primeira classe”: podem ser guardadas em variáveis, passadas como argumento e retornadas por outras funções. Quando atribuímos uma função a uma variável, temos uma **função de expressão**:

```
const soma = function (num1, num2) {  
  return num1 + num2;  
};  
  
soma(6, 4); // 10
```

Repare que a função, aqui, **não tem nome próprio** — chamamos isso de **função anônima**. Quem tem nome é a variável `soma`.

3.4 Funções anônimas e callbacks

Uma **função anônima** é uma função sem nome, usada “na hora”. O caso mais comum é passá-la como **argumento** de outra função. Uma função que é passada como argumento para ser executada depois chama-se **callback** (“função de retorno de chamada”).

```
function primeira(callback) {  
  const nome = "Gabriel";  
  callback(nome); // executa a função que recebeu  
}  
  
primeira(function (nome) {  
  console.log("Olá, " + nome);  
});  
// imprime: Olá, Gabriel
```

O conceito parece abstrato agora, mas é **absolutamente central**. Veja um exemplo que você vai usar o tempo todo:

```
botao.addEventListener("click", function () {  
  console.log("Você clicou!");  
});
```

Aqui, `addEventListener` é uma função que recebe **dois argumentos**: o nome do evento (`"click"`) e um **callback** — a função anônima que deve rodar *quando* o clique acontecer. Você não chama essa função; você a **entrega** ao navegador, que a chamará no momento certo.

🧩 **Montando o quebra-cabeça** Callbacks estão em todo o App Livros. Cada botão, cada formulário, cada card clicável usa `addEventListener` com um callback. Exemplo real da tela de livros:

```
document.getElementById("proximaPagina")
    .addEventListener("click", () => mudarPagina(1));
```

E, no Capítulo 12, veremos que os callbacks também são a **base histórica do assincronismo** — a forma como o JavaScript lidava com tarefas demoradas antes das Promises.

3.5 Arrow functions (funções de seta)

O JavaScript moderno (a partir do ES6, 2015) trouxe uma sintaxe mais curta para funções: as **arrow functions**, batizadas assim pela seta `=>`.

Compare:

```
// função de expressão tradicional
const somar = function (a, b) {
  return a + b;
};

// a mesma coisa, como arrow function
const somar = (a, b) => {
  return a + b;
};
```

E há atalhos poderosos quando a função só tem uma expressão:

- Se o corpo é um **único** `return`, podemos remover as chaves e a palavra `return` (o retorno passa a ser implícito):

```
const somar = (a, b) => a + b;
const ehPar = (x) => x % 2 === 0;
```

- Se há **um único parâmetro**, os parênteses são opcionais:

```
const dobro = x => x * 2;
```

Resumindo as regras da arrow function:

- Não precisa da palavra `function`.
- Com corpo de uma linha, não precisa de `{ }` nem de `return`.
- Com um só parâmetro, não precisa de `( )`.

⚠ **Cuidado** Se você abrir as chaves `{ }`, o `return` volta a ser **obrigatório**:

```
const somar = (a, b) => { a + b; }; // ❌ retorna undefined!  
const somar = (a, b) => { return a + b; }; // ✅  
const somar = (a, b) => a + b; // ✅ (sem chaves)
```

🧩 **Montando o quebra-cabeça** Arrow functions curtas aparecem por todo o projeto, principalmente como callbacks enxutos:

```
botao.addEventListener("click", () => mudarPagina(-1));  
const links = itens.map(item => `<li>...${item.label}...</li>`);
```

Aquele `() =>` significa “uma função sem parâmetros que faz o seguinte”. É a forma mais concisa de escrever um callback.

3.6 Escopo e hoisting

O **escopo** é a região do código onde uma variável ou função “existe” e pode ser usada. Variáveis declaradas dentro de uma função só valem dentro dela — isso evita que os nomes de uma parte do programa colidam com os de outra.

```
function calcular() {  
  const interno = 42; // só existe aqui dentro  
  console.log(interno);  
}  
calcular();  
console.log(interno); // ❌ erro: interno is not defined
```

O **hoisting** (“içamento”) é um comportamento do JavaScript que “eleva” as declarações de funções para o topo do escopo antes de rodar o código. Por isso é possível **chamar uma função declarada com `function` antes** de sua definição:

```
soma(6, 4); // funciona! → 10
```

```
function soma(a, b) {  
  return a + b;  
}
```

⚠ **Cuidado** O hoisting **completo** só vale para funções declaradas com `function`. Com funções de expressão (`const soma = function...` ou arrow), a variável ainda não recebeu a função no momento da chamada antecipada, e você recebe um erro. Regra prática: **declare antes de usar** e você nunca terá surpresas.

3.7 Funções recursivas

Uma função que **chama a si mesma** é chamada de recursiva. A recursão é uma alternativa aos loops: ambas repetem código, e ambas precisam de uma **condição de parada** para não rodar para sempre.

```
function contar(x) {  
  console.log(x);  
  if (x >= 10) {  
    return; // condição de parada  
  }  
  contar(x + 1); // chama a si mesma com o próximo valor  
}  
contar(0); // imprime 0, 1, 2, ..., 10
```

Sem o `if (x >= 10) return`, teríamos uma **recursão infinita** e o programa travaria. Guarde a ideia: *toda recursão precisa de uma saída*.

3.8 Closures

Este é um conceito mais avançado, mas vale conhecer. O JavaScript permite **aninhar** funções (uma dentro da outra), e a função interna tem acesso às variáveis da função externa — **mesmo depois** de a externa já ter terminado. Esse “acesso preservado” chama-se **closure** (fechamento).

```
const pessoa = function (nome) {  
  const getNome = function () {  
    return nome; // a interna "lembra" do nome da externa  
  };  
  return getNome; // devolve a função interna  
};  
  
const aluno = pessoa("Gabriel");  
console.log(aluno()); // "Gabriel"
```

Mesmo depois de `pessoa("Gabriel")` terminar, a função `getNome` continua “lembrando” que `nome` era `"Gabriel"`. Isso é uma closure.

💡 **Nos bastidores** Closures são a base de muitos padrões avançados (fábricas de funções, dados “privados”, memoização). Você não precisa dominá-las agora, mas vai reencontrar o conceito ao longo da sua carreira. Por ora, basta entender: *funções internas enxergam as variáveis das externas*.

3.9 Por que isso tudo importa para a SPA?

Tudo o que vimos aqui converge para uma ideia simples e poderosa que sustentará o projeto inteiro:

Cada página do App Livros será uma função que devolve (ou monta) o HTML daquela página.

Veja uma prévia — não precisa entender cada detalhe, apenas reconhecer os conceitos deste capítulo:

```
function home(app) {  
  const paginaInicial = `

# 

  app.innerHTML = paginaInicial;  
}  
  
export default {  
  url: "#home",  
  label: "Home",  
  pagina: home // a função guardada como valor de uma propriedade!  
}
```

Repare: `home` é uma função com um parâmetro (`app`); ela monta um HTML com template string; e é **guardada como valor** dentro de um objeto (`pagina: home`). Cada conceito deste capítulo — parâmetros, funções como valores, template strings — está ali. É por isso que estudamos funções antes de tocar na interface.

Recapitulando

- Uma **função** empacota um trecho de código reutilizável; você a **define** e depois a **chama** com `()`.
 - **Parâmetros** são entradas; **return** é a saída.
 - Funções são **valores**: podem ser guardadas em variáveis (função de expressão) e passadas como argumento (**callback**).
 - **Arrow functions** (`=>`) são a forma curta e moderna, ótimas para callbacks.
 - **Escopo** limita onde os nomes existem; **hoisting** eleva declarações de `function`.
 - **Recursão** é uma função que se chama (com condição de parada); **closures** preservam o acesso às variáveis externas.
-

Exercícios do Capítulo 3

1. **A calculadora**: crie quatro funções — `somar`, `subtrair`, `multiplicar` e `dividir` — cada uma recebendo dois números e **retornando** o resultado. Teste todas.
2. Reescreva as quatro funções do exercício 1 como **arrow functions** de uma linha só.
3. Crie uma função `saudacao(nome)` que **retorna** (não imprime) a string ``Olá, ${nome}!``. Guarde o retorno numa variável e só então imprima.
4. **Callback na prática**: crie uma função `processar(valor, callback)` que executa `callback(valor)`. Chame-a passando uma função anônima que imprime o valor em maiúsculas (dica: `texto.toUpperCase()`).
5. **Recursão**: escreva uma função recursiva `contagemRegressiva(n)` que imprime de `n` até 0.
6. Explique, com suas palavras, a diferença entre **parâmetro** e **argumento**.

Capítulo 4 — Loops: repetição com controle

Computadores são incansáveis: eles fazem a mesma coisa milhões de vezes sem reclamar. Os **loops** (laços de repetição) são a estrutura que nos permite aproveitar isso — executar um bloco de código repetidas vezes sem escrevê-lo repetidas vezes.

No App Livros, os loops são o que transforma **dados em tela**: percorremos um array de livros e, para cada livro, montamos um card. Percorremos a lista de rotas e, para cada rota, montamos um item de menu. Sem loops, teríamos que escrever cada card na mão. Vamos dominá-los.

4.1 O loop **for**

O **for** é o loop mais conhecido. Ele é ideal quando você **sabe quantas vezes** quer repetir. Sua estrutura tem três partes, separadas por ponto e vírgula:

```
for (inicialização; condição; incremento) {  
  // código executado a cada repetição  
}
```

- **Inicialização**: roda uma vez, no começo. Normalmente cria um contador.
- **Condição**: testada **antes** de cada volta. Enquanto for **true**, o loop continua.
- **Incremento**: roda ao **fim** de cada volta. Normalmente aumenta o contador.

```
for (let i = 1; i <= 10; i++) {  
  console.log(i); // imprime 1, 2, 3, ..., 10  
}
```

Leia assim: “comece com **i** valendo 1; enquanto **i** for menor ou igual a 10, execute o bloco; ao final de cada volta, some 1 em **i**.”

 **Nos bastidores** — o **i++** **i++** é um atalho para **i = i + 1**. Existe também **i--** (diminui 1), **i += 2** (soma 2), **soma += i** (soma o valor de **i** à variável **soma**). Você verá todos eles.

Acumulando um valor

Um uso clássico é somar tudo:

```
let soma = 0;
for (let i = 1; i <= 10; i++) {
  soma += i; // vai acumulando: 1, depois 3, depois 6...
}
console.log(soma); // 55
```

4.2 Percorrendo um array com **for**

Como o índice de um array vai de `0` até `length - 1`, o **for** casa perfeitamente com arrays:

```
let lista = [1, 2, 3, 4, 5, 6];

for (let i = 0; i < lista.length; i++) {
  console.log(lista[i]);
}
```

Repare em dois detalhes cruciais:

- Começamos em `i = 0` (o primeiro índice).
- A condição é `i < lista.length` (com `<`, **não** `<=`), porque o último índice válido é `length - 1`.

 **Cuidado** Se você usar `i <= lista.length`, na última volta `lista[i]` será `undefined`, porque essa posição não existe. Esse é o famoso erro “*off by one*” (erro de um a mais). Para percorrer arrays, a fórmula é sempre `i = 0; i < lista.length; i++`.

4.3 Interrompendo e pulando: **break** e **continue**

Às vezes queremos parar o loop no meio ou pular uma volta específica.

break — para o loop imediatamente

```
let lista = [1, 2, 3, 4, 5, 6];

for (let i = 0; i < lista.length; i++) {
  if (lista[i] === 4) {
    break; // encontrou o 4? para tudo.
  }
  console.log(lista[i]); // imprime 1, 2, 3 e para
}
```

continue — pula para a próxima volta

```
let lista = [1, 2, 3, 4, 5, 6];

for (let i = 0; i < lista.length; i++) {
  if (lista[i] % 2 === 0) {
    continue; // é par? pula esta volta.
  }
  console.log(lista[i]); // imprime só os ímpares: 1, 3, 5
}
```

A diferença: **break** encerra o loop; **continue** apenas **abandona a volta atual** e segue para a próxima.

4.4 O **for...of** — percorrendo valores (ES6)

O **for** clássico exige controlar o índice manualmente. Na maioria das vezes, só queremos os **valores** do array, não os índices. Para isso existe o **for...of**, introduzido no ES6:

```
let frutas = ["maçã", "pêra", "morango"];

for (const fruta of frutas) {
  console.log(fruta); // maçã, pêra, morango
}
```

Muito mais limpo! A cada volta, **fruta** já é o **próprio valor** — sem colchetes, sem índice, sem **.length**. O **for...of** funciona com qualquer coisa “iterável”: arrays, strings, e outras estruturas.

O **break** e o **continue** funcionam normalmente aqui:

```
for (const fruta of frutas) {
  if (fruta === "pêra") break;
  console.log(fruta); // imprime só "maçã"
}
```

🌱 **Montando o quebra-cabeça** O `for...of` é o loop preferido do App Livros. Quase toda renderização usa este padrão — percorrer os dados e ir montando o HTML numa string:

```
let html = "";
for (const livro of livros) {
  html += `<div class="bem-card">
    <h3>${livro.titulo}</h3>
    <p>${livro.autores}</p>
  </div>`;
}
listaLivros.innerHTML = html;
```

Guarde esse padrão: `let html = ""` → **laço que faz** `html += ...` → **joga no** `innerHTML`. Ele se repete em várias telas do projeto.

4.5 O `for...in` — percorrendo as chaves de um objeto

Enquanto o `for...of` percorre **valores** de arrays, o `for...in` percorre as **chaves** (propriedades) de um objeto:

```
let carro = {
  marca: "Toyota",
  modelo: "Corolla",
  ano: 2021
};

for (const prop in carro) {
  console.log(`${prop}: ${carro[prop]}`);
}
// marca: Toyota
// modelo: Corolla
// ano: 2021
```

Note o uso da **notação de colchetes** aqui: `carro[prop]`. Como `prop` é uma variável que guarda o nome da propriedade (um texto que muda a cada volta), não podemos usar `carro.prop` (isso procuraria uma propriedade literalmente chamada "prop"). Por isso os

colchetes são obrigatórios. Lembra do Capítulo 2? Este é o caso em que a notação de colchetes brilha.

⚠ Cuidado — não confunda!

- `for...of` → **valores** → use com **arrays**.
- `for...in` → **chaves** → use com **objetos**.

Usar `for...in` num array até funciona, mas devolve os índices como texto ("0", "1", "2") e pode trazer surpresas. Para arrays, prefira `for...of` ou o `for` clássico.

4.6 `while` e `do...while`

O `for` é ótimo quando sabemos o número de repetições. Mas e quando **não sabemos**? Aí entram o `while` e o `do...while`.

`while` — repita enquanto a condição for verdadeira

A condição é testada **antes** de cada volta:

```
let contador = 1;
let limite = 10;

while (contador <= limite) {
  console.log(contador);
  contador++; // ESSENCIAL: sem isso, loop infinito!
}
```

⚠ **Cuidado — o loop infinito** Num `while`, é você quem deve garantir que a condição um dia se torne `false`. Se esquecer de atualizar a variável (`contador++`), o loop nunca termina e a aba do navegador congela. Sempre pergunte: “o que faz essa condição virar falsa?”

`do...while` — execute pelo menos uma vez

O `do...while` é irmão do `while`, mas com uma diferença crucial: ele executa o bloco **primeiro** e só então testa a condição. Ou seja, roda **no mínimo uma vez**, mesmo que a condição já comece falsa.

```
let entrada;
do {
  entrada = prompt("Digite 'ok' para continuar:");
} while (entrada !== "ok");
```

Perfeito para situações do tipo “peça algo ao usuário e repita até ele acertar” — você precisa perguntar pelo menos uma vez.

4.7 Escolhendo o loop certo

Situação	Loop recomendado
Sei o número de repetições / preciso do índice	<code>for</code>
Quero os valores de um array	<code>for...of</code>
Quero as chaves de um objeto	<code>for...in</code>
Repito até uma condição, sem saber quantas vezes	<code>while</code>
Preciso executar pelo menos uma vez	<code>do...while</code>

No dia a dia do App Livros, o campeão é o `for...of` para percorrer dados, e o `for` clássico quando precisamos do índice.

Resumindo

- O `for` repete um número conhecido de vezes, com contador.
- Para arrays, use `i = 0; i < lista.length; i++`.
- `break` encerra o loop; `continue` pula uma volta.
- `for...of` percorre **valores** (arrays); `for...in` percorre **chaves** (objetos).
- `while` repete enquanto uma condição vale; `do...while` garante ao menos uma execução.
- O padrão `let html = ""` + laço + `innerHTML` é o motor de renderização do nosso projeto.

Exercícios do Capítulo 4

1. **Inverter um array:** dado `[1, 2, 3, 4, 5]`, use um `for` (começando do fim) para criar um novo array `[5, 4, 3, 2, 1]`.
2. **Maior número:** dado `[5, 3, 9, 1, 6]`, use um `for` para encontrar e imprimir o maior valor.
3. **Contar caracteres:** dada a string `"banana"`, use um `for...of` para contar quantas vezes a letra `'a'` aparece.
4. **Números pares:** dado `N = 5`, use um `for` para gerar um array com os primeiros N números pares: `[2, 4, 6, 8, 10]`.
5. **Percorrendo objeto:** dado um objeto `pessoa` com `nome`, `idade` e `cidade`, use `for...in` para imprimir cada propriedade no formato `"chave: valor"`.
6. **Simulando o projeto:** dado um array de objetos `[{titulo:"A"}, {titulo:"B"}]`, use `for...of` para montar uma string HTML `"<li>A</li><li>B</li>"` e imprima-a.

Capítulo 5 — Manipulando o DOM e reagindo a eventos

Até aqui, todo o nosso código viveu no Console. Chegou a hora de **mexer na página de verdade**. A ponte entre o JavaScript e o HTML se chama **DOM**, e é ela que nos permite criar, alterar e remover elementos na tela — e reagir ao que o usuário faz.

Este é o capítulo em que a “programação” vira “página web”.

5.1 O que é o DOM?

DOM significa *Document Object Model* (Modelo de Objetos do Documento). Quando o navegador carrega uma página HTML, ele constrói na memória uma representação daquele HTML em forma de **árvore de objetos**. Cada tag vira um “nó” (um objeto) que o JavaScript pode acessar e manipular.

Considere este HTML:

```
<!DOCTYPE html>
<html>
  <head><title>Exemplo DOM</title></head>
  <body>
    <h1 id="titulo">Aula DOM!</h1>
    <p class="paragrafo">Este é um parágrafo.</p>
    <button id="botao">Clique aqui!</button>
  </body>
</html>
```

O navegador o transforma em uma árvore mais ou menos assim:

```
document
├── html
│   ├── head
│   │   └── title → "Exemplo DOM"
│   └── body
│       ├── h1#titulo → "Aula DOM!"
│       ├── p.paragrafo → "Este é um parágrafo."
│       └── button#botao → "Clique aqui!"
```

O objeto global **document** é a porta de entrada para essa árvore. A partir dele, alcançamos qualquer elemento.



Nos bastidores “Saber o que tem por baixo dos panos nos dá mais poder.” Frameworks como React mantêm uma cópia virtual dessa árvore para otimizar mudanças. Nós vamos manipular a árvore **real**, diretamente — e é exatamente isso que os frameworks fazem por baixo, só que escondido.

5.2 Selecionando elementos

Para manipular um elemento, primeiro precisamos **encontrá-lo**. Os principais métodos de seleção são:

```
// por ID (o mais específico – retorna um único elemento)
document.getElementById("titulo");

// por seletor CSS (o mais versátil – retorna o primeiro que casar)
document.querySelector("#titulo"); // # = id
document.querySelector(".paragrafo"); // . = classe
document.querySelector("button"); // nome da tag

// todos os que casam com o seletor (retorna uma lista)
document.querySelectorAll(".bem-card");
```

Repare na lógica dos seletores do `querySelector`, idêntica à do CSS:

- `#` seleciona por **id**: `#titulo`
- `.` seleciona por **classe**: `.paragrafo`
- **o nome direto** seleciona pela **tag**: `button`



Nos bastidores — `getElementById` vs `querySelector` `getElementById("titulo")` e `querySelector("#titulo")` fazem quase a mesma coisa. O `getElementById` é ligeiramente mais rápido e recebe **apenas o id** (sem o `#`). O `querySelector` é mais flexível, aceitando qualquer seletor CSS. No App Livros usamos os dois — `getElementById` para pegar elementos por id e `querySelectorAll` para pegar vários cards de uma vez.

5.3 Modificando elementos

Uma vez selecionado, o elemento é um objeto com propriedades que podemos ler e alterar. As três mais usadas:

Conteúdo de texto — `textContent`

```
document.getElementById("titulo").textContent = "DOM Manipulado!";
```

Conteúdo HTML — `innerHTML`

```
document.getElementById("app").innerHTML = "<h1>Olá</h1><p>Bem-vindo!</p>";
```

A diferença é fundamental:

- `textContent` trata tudo como **texto puro**. Se você atribuir `"<b>oi</b>"`, aparecerá literalmente `<b>oi</b>` na tela.
- `innerHTML` interpreta as tags como HTML. `"<b>oi</b>"` vira um **oi** em negrito.

🌱 **Montando o quebra-cabeça** O `innerHTML` é, sem exagero, a técnica central de todo o **App Livros**. Cada página monta um bloco de HTML numa template string e o “injeta” no elemento `<main id="app">`:

```
app.innerHTML = `<h1>Esta é a página Inicial</h1>`;
```

Trocar o `innerHTML` do `#app` é o que faz a SPA “mudar de página” sem recarregar o navegador. Guarde bem isso: **renderizar = montar HTML numa string e jogar no `innerHTML`**.

Estilo — `style`

```
document.querySelector(".paragrafo").style.color = "blue";
```

Cada propriedade CSS vira uma propriedade do objeto `style` (em camelCase: `background-color` → `backgroundColor`). Na prática, para muitas mudanças de estilo, é mais organizado adicionar/remover **classes** (veja a seção 5.7).

5.4 Criando e removendo elementos

Além de trocar o conteúdo de elementos existentes, podemos **criar novos nós** e inseri-los na árvore.


```
// 1. criar o elemento (ainda solto, fora da página)
let novoParagrafo = document.createElement("p");

// 2. configurar o elemento
novoParagrafo.className = "paragrafo";
novoParagrafo.textContent = "Este é um novo parágrafo!";

// 3. anexá-lo como filho de outro elemento
document.body.appendChild(novoParagrafo);
```

Os três verbos essenciais:

- `document.createElement("p")` — cria um novo elemento (uma tag `<p>`).
- `elementoPai.appendChild(filho)` — anexa um elemento como filho de outro.
- `elemento.removeChild(filho)` ou, mais moderno, `elemento.remove()` — remove um elemento da página.

```
// removendo um elemento
let paragrafo = document.querySelector(".paragrafo");
paragrafo.remove(); // some da tela
```



Nos bastidores — dois estilos de construir a tela Existem duas abordagens para montar interface:

1. **innerHTML com template strings** — escrever o HTML como texto. É mais rápido de ler/escrever para blocos grandes. É o estilo principal do App Livros.
2. **createElement + appendChild** — construir nó por nó. É mais verboso, porém mais seguro contra certos problemas e melhor para adicionar itens um a um.

No projeto, você verá os dois. Na página de contato, por exemplo, cada nova mensagem enviada é um `<li>` criado com `createElement` e anexado com `appendChild` à lista. Já as páginas inteiras nascem de `innerHTML`.

5.5 Eventos: reagindo ao usuário

Uma página só é “viva” se responde às ações do usuário: cliques, digitação, envio de formulários. Essas ações são **eventos**, e nós “escutamos” por elas com o método `addEventListener`:

```
elemento.addEventListener(evento, callback);
```

O nome já explica: `add` + `Event` + `Listener` = “adicione um ouvinte de evento”. Você diz **em qual elemento**, **qual evento** e **qual função** rodar quando ele acontecer.

```
document.querySelector("button").addEventListener("click", function () {  
  alert("Botão clicado!");  
});
```

Lembre-se do Capítulo 3: aquela função anônima é um **callback**. Você não a chama; o navegador a chama por você **quando** o clique acontecer.

Alguns tipos de evento comuns:

Evento	Dispara quando...
<code>click</code>	o usuário clica no elemento
<code>submit</code>	um formulário é enviado
<code>input</code>	o valor de um campo muda enquanto digita
<code>blur</code>	um campo perde o foco (o usuário sai dele)
<code>mouseover</code>	o mouse passa por cima do elemento
<code>keyup</code>	uma tecla é solta

🧩 **Montando o quebra-cabeça** O evento `blur` vai protagonizar um momento marcante do curso. Na tela de cadastro, quando o usuário digita o CEP e **sai do campo** (evento `blur`), disparamos uma busca à API dos Correios e preenchemos o endereço automaticamente. É o Capítulo 15 — e é puro `addEventListener`.

5.6 Manipulando formulários

Formulários são a principal forma de o usuário **enviar dados**. Três tarefas aparecem o tempo todo: ler o valor de um campo, reagir ao envio e impedir o recarregamento padrão da página.

```
<form id="formulario">  
  <input type="text" id="nome" placeholder="Digite seu nome">  
  <button type="submit">Enviar</button>  
</form>
```

```
const form = document.getElementById("formulario");

form.addEventListener("submit", function (event) {
  event.preventDefault(); // impede o recarregamento da página
  let nome = document.getElementById("nome").value;
  alert("Nome: " + nome);
});
```

Dois pontos merecem destaque:

- `.value` lê (ou escreve) o conteúdo de um campo de formulário. Guarde: para campos de input, é `.value`; para outros elementos, é `.textContent`.
- `event.preventDefault()` é **obrigatório** aqui. Por padrão, enviar um formulário faz o navegador **recarregar a página** (comportamento antigo, de quando o servidor processava tudo). Numa SPA, isso destruiria nossa aplicação. O `preventDefault()` cancela esse comportamento padrão, deixando o JavaScript no controle.

 **Cuidado** Esquecer o `event.preventDefault()` num formulário de SPA é um erro clássico: você clica em “Enviar”, a página pisca e recarrega, e parece que “nada aconteceu” (na verdade, tudo aconteceu e foi jogado fora). Se um form estiver se comportando estranho, verifique isso primeiro.

 **Montando o quebra-cabeça** A página de contato do App Livros é exatamente este exemplo, ampliado. No `submit`, chamamos `event.preventDefault()`, lemos os valores dos campos, e criamos um `<li>` com a mensagem, anexando-o a uma lista com `appendChild`. Vamos construí-la no Capítulo 8.

5.7 Bônus: alternando classes com `classList`

Em vez de mexer no `style` propriedade por propriedade, o mais organizado é definir estilos em classes CSS e apenas **adicionar ou remover** essas classes via JavaScript. Para isso existe o objeto `classList`:

```
elemento.classList.add("bem-modal--hidden"); // adiciona a classe
elemento.classList.remove("bem-modal--hidden"); // remove a classe
elemento.classList.toggle("ativo"); // adiciona se não tem, remove se te
```

🧩 **Montando o quebra-cabeça** É assim que abrimos e fechamos as **janelas de detalhes (modais)** do App Livros. A janela existe no HTML o tempo todo, mas fica escondida por uma classe `bem-modal--hidden`. Para mostrá-la, removemos a classe; para escondê-la, adicionamos de volta:

```
modal.classList.remove("bem-modal--hidden"); // abre
modal.classList.add("bem-modal--hidden");    // fecha
```

Simples e eficiente — sem recriar nada, só mostrando e ocultando.

5.8 Juntando tudo: renderizando uma lista de dados

Vamos unir os capítulos anteriores (arrays de objetos + loops + template strings) com o DOM, num exemplo que é praticamente o coração do projeto:

```
const produtos = [
  { nome: "Produto 1", preco: "R$ 10,00" },
  { nome: "Produto 2", preco: "R$ 20,00" },
  { nome: "Produto 3", preco: "R$ 30,00" }
];

let html = "";
for (const produto of produtos) {
  html += `<div class="card">${produto.nome} - ${produto.preco}</div>`;
}

document.getElementById("app").innerHTML = html;
```

Leia esse trecho reconhecendo cada peça já estudada:

1. Um **array de objetos** (Cap. 2) com os dados.
2. Um `for...of` (Cap. 4) percorrendo a lista.
3. **Template strings** (Cap. 1) montando o HTML de cada item.
4. `innerHTML` (este capítulo) despejando tudo na página.

Se você entendeu esse bloco, entendeu a mecânica de **toda** tela de listagem do App Livros — livros, personagens, serviços. O que muda de uma para outra são apenas os dados e as classes CSS.

Recapitulando

- O **DOM** é a árvore de objetos que o navegador cria a partir do HTML; `document` é a porta de entrada.
 - Selecione com `getElementById`, `querySelector` e `querySelectorAll`.
 - Modifique com `textContent` (texto), `innerHTML` (HTML) e `style`.
 - Crie e remova nós com `createElement`, `appendChild` e `remove`.
 - Reaja ao usuário com `addEventListener(evento, callback)`.
 - Em formulários, use `.value` para ler campos e `event.preventDefault()` para impedir o recarregamento.
 - Alterne visibilidade e estilos com `classList`.
-

Exercícios do Capítulo 5

1. Crie um HTML com um `<h1 id="titulo">` e um `<button id="btn">`. Via JavaScript, faça o botão, ao ser clicado, mudar o `textContent` do título para "Você clicou!".
2. Crie um `<input id="nome">` e um botão. Ao clicar, leia o `.value` do input e mostre um `alert` com o texto digitado.
3. Crie um formulário com um campo de texto. No evento `submit`, use `event.preventDefault()`, leia o valor e adicione um novo `<li>` (criado com `createElement`) a uma lista `<ul>` na página.
4. Dado o array `produtos` da seção 5.8, renderize cada produto como um card na página usando `for...of` + `innerHTML`.
5. Crie uma `<div>` escondida com uma classe `escondido` (que tem `display: none` no CSS) e um botão que a mostra usando `classList.remove("escondido")`.

Capítulo 6 — O que é uma SPA, URL, URI e o hash

Agora que dominamos a linguagem e sabemos manipular a página, é hora de entender a **arquitetura** que vamos construir. Antes de escrever o roteador, precisamos responder três perguntas: o que é uma SPA? Como uma URL é formada? E o que é esse tal de **hash** que faz tudo funcionar?

Este capítulo é mais conceitual — mas é o mapa mental que dá sentido a todo o resto do curso.

6.1 Três tipos de site

Landing Page

Uma **Landing Page** é uma página única, focada em **converter** o visitante (virar cliente, capturar um e-mail). É simples e direta, geralmente com um formulário e uma chamada para ação. Muito usada em campanhas de marketing.

-  Simples, ótima para SEO (aparecer no Google), rápida.
-  Pouca interatividade, experiência estática.

MPA — Multi Page Application (aplicação multipágina)

O modelo **clássico** da web: cada link leva a um **novo arquivo HTML**, e o navegador **recarrega a página inteira** a cada navegação. Sites feitos em PHP puro, ou o WordPress tradicional, funcionam assim.

-  Simples, bom para SEO.
-  Cada clique recarrega tudo do zero — a tela “pisca”, é mais lento.

SPA — Single Page Application (aplicação de página única)

A **SPA** carrega **uma única página HTML** e, a partir daí, **atualiza o conteúdo dinamicamente** conforme o usuário interage. Ela troca só o miolo da tela, sem nunca recarregar a página inteira. Para o usuário, “parece” que são várias páginas — mas é sempre a mesma, sendo reescrita por JavaScript.

-  Melhor experiência, navegação instantânea, mais interatividade.
-  Maior complexidade de desenvolvimento e desafios com SEO.

Hash (fragmento)

`#secao1` — uma **âncora** que aponta para uma parte **dentro** da própria página. É o herói da nossa história — merece uma seção só sua.

💡 **Nos bastidores — URL x URI** Você verá os dois termos. **URI** (Identificador) é o conceito geral de “identificar um recurso”; **URL** (Localizador) é uma URI que também diz **como chegar** nele (o protocolo). Na prática do dia a dia, tratamos os dois como sinônimos. Não perca o sono com essa distinção.

6.3 A Query String no JavaScript

Podemos ler os parâmetros da query string com o objeto `URLSearchParams`:

```
// URL: https://exemplo.com/busca?termo=javascript&pagina=2
const params = new URLSearchParams(window.location.search);

params.get("termo"); // "javascript"
params.get("pagina"); // "2"
```

🧩 **Montando o quebra-cabeça** A query string é exatamente como conversamos com as APIs. Quando pedimos a lista de livros, montamos uma URL assim:

```
https://gutendex.com/books/?search=fiction&page=2
```

O `?search=fiction&page=2` é uma query string! Estamos dizendo ao servidor: “busque por ‘fiction’, página 2”. Vamos construir URLs desse tipo no Capítulo 16.

6.4 O `hash`: o segredo da SPA

O **hash** é a parte da URL após o `#`. Ele tem uma característica **mágica** para o nosso propósito:

Mudar o hash **não recarrega a página** e **não faz nova requisição ao servidor**. O que vem depois do `#` **nunca é enviado ao servidor**.

Originalmente, o hash servia para rolar até uma seção da página (uma âncora). Mas os desenvolvedores perceberam algo genial: como mudar o hash **não recarrega nada**, ele pode ser usado para indicar qual “página” da SPA está ativa.

```
www.exemplo.com#home    → mostra a Home
www.exemplo.com#sobre   → mostra o Sobre
www.exemplo.com#contato → mostra o Contato
```

O usuário clica num link `<a href="#sobre">`, a URL muda para `...#sobre`, mas **a página não recarrega**. Nós, no JavaScript, apenas detectamos essa mudança e trocamos o conteúdo da tela. Para o usuário, é como navegar entre páginas — mas é tudo a mesma página.

6.5 As três palavras mágicas: `window.location.hash`

Como o JavaScript descobre qual é o hash atual? Através de um caminho de objetos do navegador:

```
window.location.hash;
```

Desmembrando:

- `window` é o objeto global que representa a janela/aba do navegador. Tudo que é global vive dentro dele.
- `window.location` é um objeto com informações sobre a URL atual (o endereço, o protocolo, o host...).
- `window.location.hash` é, especificamente, a parte após o `#` — por exemplo, `"#sobre"`.

Experimente agora mesmo. Abra qualquer site, vá ao Console e digite:

```
window.location.hash;           // provavelmente "" (vazio)
window.location.hash = "#teste"; // mude o hash pela programação
window.location.hash;           // agora "#teste"
```

Repare que ao atribuir um valor ao `hash`, a URL na barra muda — **sem recarregar**. Esse é o mecanismo inteiro.



Nos bastidores “Saber o que tem por baixo dos panos nos dá mais poder.” O `window.location` tem vários outros campos úteis: `window.location.search` (a query string), `window.location.pathname` (o caminho), `window.location.href` (a URL completa). Vale explorá-los no Console.

6.6 O evento que fecha o ciclo: `hashchange`

Falta uma peça: como saber, **no momento exato** em que o hash muda, para então trocar a tela? O navegador nos avisa disparando um evento chamado `hashchange`. Nós o escutamos como qualquer outro evento (Capítulo 5):

```
window.addEventListener("hashchange", function () {
  const hash = window.location.hash;
  console.log("O usuário navegou para:", hash);
  // aqui vamos trocar o conteúdo da tela!
});
```

Junte as peças e você já enxerga a SPA inteira:

1. Os links do menu apontam para `#home`, `#sobre`, etc.
2. Clicar num link muda o `window.location.hash` (sem recarregar).
3. A mudança dispara o evento `hashchange`.
4. Nosso “ouvinte” lê o novo hash e troca o `innerHTML` do `#app`.

Isso é, em essência, o roteamento no lado do cliente (*client-side routing*). É o que faremos no próximo capítulo, na prática.

6.7 O plano de construção da SPA

Reunindo tudo, o roteiro para montar nossa SPA (que os capítulos seguintes vão executar passo a passo) é:

1. Ter um único `index.html` com um ponto de montagem: `<main id="app">`.
2. Definir o **conteúdo de cada “página”** (a princípio como strings, depois como funções em módulos separados).
3. Criar as **funções de manipulação** que colocam cada conteúdo na tela.
4. **Escutar o `hashchange`** para trocar de página conforme o hash.
5. Gerar o **menu de navegação** dinamicamente.

6. Centralizar tudo num roteador organizado.

✖ **Montando o quebra-cabeça** Este é literalmente o índice da Parte IV da apostila. Cada item acima é um capítulo. No fim, teremos exatamente a arquitetura do `main.js` do projeto: um mapa de rotas, um `addEventListener("hashchange", ...)` e uma função que renderiza a rota atual. Agora que você entende o *porquê*, vamos ao *como*.

Recapitulando

- Uma **SPA** carrega uma página só e troca o conteúdo dinamicamente, sem recarregar — diferente da **MPA** (recarrega tudo) e da **Landing Page** (página estática de conversão).
- Uma **URL** tem esquema, autoridade (domínio + porta), caminho, **query string** (`?chave=valor`) e **hash** (`#fragmento`).
- A **query string** é como conversamos com APIs; leia-a com `URLSearchParams` .
- O **hash** muda a URL **sem recarregar** a página — a base da SPA.
- Lemos o hash atual com `window.location.hash` e detectamos mudanças com o evento `hashchange` .

Exercícios do Capítulo 6

1. Explique, com suas palavras, a diferença entre uma **SPA** e uma **MPA**.
2. Dada a URL `https://loja.com/produtos?categoria=livros&pag=3#topo` , identifique: o esquema, o domínio, o caminho, a query string e o hash.
3. No Console de qualquer site, mude o hash para `#capitulo6` via JavaScript e confirme que a URL mudou sem recarregar.
4. Escreva um `addEventListener("hashchange", ...)` que imprime no console o novo valor de `window.location.hash` sempre que ele mudar. Teste digitando diferentes hashes na barra de endereço.
5. Usando `URLSearchParams` , extraia o valor de `pag` da URL do exercício 2.

Capítulo 7 — Renderização dinâmica com **hash** (o primeiro roteador)

Chega de teoria. Neste capítulo vamos escrever, do zero, a **primeira versão funcional da nossa SPA**. Ela será simples — cheia de defeitos que corrigiremos depois — mas fará a mágica acontecer: trocar de “página” sem recarregar o navegador. É o esqueleto sobre o qual todo o App Livros será construído.

Vamos deliberadamente começar “errado” (ou melhor, ingênuo) para que você **sinta na pele** os problemas e entenda por que as soluções elegantes dos próximos capítulos existem.

7.1 O ponto de partida: o **index.html**

Toda SPA começa com um HTML mínimo, contendo apenas os **pontos de montagem** — os lugares onde o JavaScript vai injetar conteúdo. Crie o arquivo **index.html**:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>App Livros</title>
</head>
<body>
  <main id="app"></main>
  <script src="app.js"></script>
</body>
</html>
```

Note como o **<body>** está quase vazio: só um **<main id="app">** (nosso ponto de montagem) e a inclusão do arquivo **app.js**. **Todo o resto será desenhado por JavaScript**. Esse vazio é a assinatura de uma SPA.



Nos bastidores Por que **<main id="app">**? O **id="app"** é o “gancho” que o JavaScript usa para encontrar o elemento (**getElementById("app")**). A tag **<main>** é semântica — indica que ali está o conteúdo principal da página. Poderia ser uma **<div>**, mas **<main>** comunica melhor a intenção.

7.2 A primeira renderização

Crie o arquivo `app.js` ao lado do `index.html`. Vamos guardar o conteúdo de cada página em uma constante (uma template string com HTML) e mostrar uma delas:

```
const app = document.getElementById("app");

const home = `<h1>Home</h1>
               <p>Bem-vindo à nossa página inicial!</p>`;

const sobre = `<h1>Sobre</h1>
               <p>Saiba mais sobre nós nesta página.</p>`;

const contato = `<h1>Contato</h1>
                 <p>Fale conosco por esta página.</p>`;

app.innerHTML = home;
```

Abra o `index.html` com o Live Server. Você deve ver o título “Home” na tela. **Parabéns — você renderizou conteúdo dinamicamente!** O HTML que aparece não está no arquivo `.html`; foi injetado pelo JavaScript via `innerHTML`.

Reconheça as peças: uma **variável** que aponta para o elemento do DOM (Cap. 5), **template strings** com o HTML (Cap. 1) e o `innerHTML` fazendo a injeção (Cap. 5). Tudo o que estudamos, junto.

7.3 Decidindo a página pelo `hash`

Mostrar sempre a Home é pouco. Queremos mostrar a página **de acordo com o hash** da URL. Lembra do Capítulo 6? Vamos ler `window.location.hash` e decidir com um `if/else` (Cap. 1):

```
const app = document.getElementById("app");

const home = `

# Home</h1><p>Bem-vindo!</p>`; const sobre = `Sobre</h1><p>Sobre nós.</p>`; const contato = `Contato</h1><p>Fale conosco.</p>`; const hash = window.location.hash; if (hash === "#home") { app.innerHTML = home; } else if (hash === "#sobre") { app.innerHTML = sobre; } else if (hash === "#contato") { app.innerHTML = contato; }


```

Agora teste: abra a página e, na barra de endereço, adicione `#sobre` ao final da URL e pressione Enter. Depois `#contato`. O conteúdo muda!

Mas você já deve ter notado **dois problemas**:

1. Ao mudar o hash na barra, **nem sempre** a tela atualiza — às vezes você precisa recarregar. Isso porque o código roda **uma única vez**, quando a página carrega. Ele não fica “escutando” mudanças.
2. Se a URL não tiver hash nenhum (`#`), a tela fica **em branco**, porque nenhum `if` foi satisfeito.

Vamos resolver os dois.

7.4 Reagindo às mudanças com `hashchange`

Para a tela reagir **toda vez** que o hash mudar, precisamos escutar o evento `hashchange` (Cap. 6) e colocar a lógica de decisão dentro de uma **função** (Cap. 3), para poder chamá-la sempre que necessário:

```

const app = document.getElementById("app");

const home = `

# Home</h1><p>Bem-vindo!</p>`; const sobre = `Sobre</h1><p>Sobre nós.</p>`; const contato = `Contato</h1><p>Fale conosco.</p>`; function renderizar() { const hash = window.location.hash || "#home"; // padrão: home if (hash === "#home") { app.innerHTML = home; } else if (hash === "#sobre") { app.innerHTML = sobre; } else if (hash === "#contato") { app.innerHTML = contato; } else { app.innerHTML = "<h1>Página não encontrada (404)</h1>"; } } // renderiza a página inicial ao carregar renderizar(); // e re-renderiza sempre que o hash mudar window.addEventListener("hashchange", renderizar);


```

Analise as melhorias:

- Envolvemos a lógica numa função `renderizar()` — agora podemos chamá-la quantas vezes quisermos.
- Chamamos `renderizar()` **uma vez** ao carregar (para a tela não começar vazia).
- Registramos `renderizar` como **callback** do `hashchange` — agora a tela reage a cada navegação.
- Usamos `window.location.hash || "#home"` para definir uma **rota padrão**: se o hash estiver vazio, assume `#home`. (O `||` retorna o primeiro valor “verdadeiro”; string vazia é “falsa”, então cai no `"#home"`.)
- Adicionamos um `else` final para tratar rotas inexistentes — nossa página **404**.



Cuidado — o **callback** vai sem **parênteses** Escrevemos `addEventListener("hashchange", renderizar)`, e **não** `renderizar()`. Sem os parênteses, entregamos a **função** para o navegador chamar depois. Com os parênteses, chamaríamos a função **agora** e passaríamos o seu retorno (que é `undefined`). Esse detalhe pega muita gente.

7.5 Adicionando a navegação

Falta o menu para o usuário clicar. Como os links de uma SPA apontam para hashes, basta usar âncoras comuns com `href="#..."`. Vamos gerar o menu também por JavaScript e inseri-lo no topo do `<body>`:

```
function carregarMenu() {
  const menu = `
    <nav>
      <ul>
        <li><a href="#home">Home</a></li>
        <li><a href="#sobre">Sobre</a></li>
        <li><a href="#contato">Contato</a></li>
      </ul>
    </nav>`;
  document.body.insertAdjacentHTML("afterbegin", menu);
}

carregarMenu();
renderizar();
window.addEventListener("hashchange", renderizar);
```

O método `insertAdjacentHTML("afterbegin", ...)` insere o HTML logo no início do `<body>`, antes do `<main>`. Assim o menu fica no topo.

Agora clique nos links. A cada clique:

1. O `href="#sobre"` muda o `window.location.hash` para `"#sobre"`.
2. A mudança dispara o evento `hashchange`.
3. O navegador chama `renderizar()`.
4. `renderizar()` lê o novo hash e troca o `innerHTML` do `#app`.

Você tem uma SPA funcionando. Navega entre páginas, sem recarregar, com URL que muda e até botão “voltar” do navegador funcionando. Reserve um momento para apreciar: você construiu do zero o que os frameworks chamam de “roteador”.

7.6 Por que essa versão não escala

Nossa SPA funciona, mas tem defeitos sérios que apareceriam num projeto de verdade:

1. **Tudo num arquivo só.** Conteúdo, lógica de rota e menu misturados no `app.js`. Com 10 páginas, vira um caos de milhares de linhas.

2. **Repetição no `if/else`**. Cada página nova exige mais um `else if`. E o menu precisa ser editado à mão em paralelo. Duas listas para manter sincronizadas — fonte garantida de erros.
3. **Conteúdo como strings soltas**. Uma página real tem lógica própria (reagir a formulários, buscar dados). Strings não comportam isso.

Cada um desses problemas será resolvido nos próximos capítulos:

Problema	Solução	Capítulo
Tudo num arquivo	Módulos (<code>import</code> / <code>export</code>)	8
Página é só string, sem lógica	Página vira função	8
Menu e rotas duplicados	Rotas como array de objetos e menu com <code>.map()</code>	9
<code>if/else</code> que cresce sem parar	Mapa de rotas centralizado	10

🌱 **Montando o quebra-cabeça** Guarde este `app.js` — ele é o **embrião** do `main.js` final do projeto. Compare, desde já, o que temos agora com o destino:

```
// versão final (Cap. 10) – não copie ainda, só observe a semelhança
function iniciar() {
  navbar(roteador);
  renderizarRotaAtual();
  window.addEventListener("hashchange", renderizarRotaAtual);
}
iniciar();
```

A estrutura é a mesma que você acabou de escrever: montar o menu, renderizar a rota atual, escutar o `hashchange`. Vamos apenas torná-la organizada e escalável.

Recapitulando

- Uma SPA começa com um `index.html` quase vazio, só com o ponto de montagem `<main id="app">`.
- Renderizar é **montar HTML numa string e jogar no `innerHTML`**.
- Decidimos a página lendo `window.location.hash` com um `if/else`.
- Envolvemos a lógica numa **função** e a registramos no evento `hashchange` para reagir às navegações.
- Definimos uma **rota padrão** (`|| "#home"`) e uma **404** (o `else` final).

- Esta versão funciona, mas **não escala** — os próximos capítulos a reorganizam com módulos, arrays de objetos e um mapa de rotas.

Exercícios do Capítulo 7

1. Reproduza a SPA deste capítulo do zero: crie `index.html` e `app.js` e confirme que a navegação entre Home, Sobre e Contato funciona sem recarregar.
2. Adicione uma quarta página, “Serviços” (`#servicos`), com um `else if` e um novo item no menu. Sinta o “trabalho dobrado” de editar dois lugares.
3. Faça a rota padrão ser `#home` também quando o usuário digitar um hash inexistente (dica: já está no `else` , teste com `#qualquercoisa`).
4. No callback do `hashchange` , adicione um `console.log` que registra para qual hash o usuário navegou. Observe no Console durante a navegação.
5. **Reflexão:** imagine que o projeto terá 15 páginas. Liste os problemas que o `if/else` gigante causaria e proponha, com suas palavras, como resolvê-los.

Capítulo 8 — Componentização com ES Modules

No capítulo anterior, construímos uma SPA funcional — mas tudo amontoado em um único `app.js`. Agora vamos aprender a técnica que mantém projetos organizados por mais que cresçam: os **módulos**. Vamos quebrar aquele arquivão em vários arquivos pequenos, cada um com uma responsabilidade única. Cada página vira um **componente** independente.

Ao final, teremos a estrutura de pastas real do App Livros começando a tomar forma.

8.1 O que são módulos e por que usá-los

Um **módulo** é simplesmente um arquivo JavaScript que **exporta** funcionalidades (funções, objetos, variáveis) para que **outros arquivos possam importá-las**. Em vez de um arquivo gigante, temos vários pequenos que colaboram.

Por que vale a pena?

1. **Organização** — código relacionado fica junto; fica fácil achar as coisas.
2. **Reutilização** — uma função útil pode ser importada em vários lugares.
3. **Isolamento** — cada módulo tem seu próprio escopo. Variáveis de um arquivo não “vazam” nem colidem com as de outro (adeus, bugs de variável global).
4. **Clareza** — cada arquivo tem um propósito, o que torna o projeto legível.

 **Nos bastidores** Antes dos módulos (padronizados no ES6, 2015), os desenvolvedores despejavam tudo no escopo global, e conflitos de nomes eram um pesadelo. Os ES Modules resolveram isso de vez. Toda ferramenta moderna (React, Vue, Node) é baseada neles. Aprender `import` / `export` aqui é aprender a base de todo o ecossistema.

8.2 Ativando os módulos no HTML

Para usar `import` / `export`, o navegador precisa ser avisado de que o script é um módulo. Isso se faz com o atributo `type="module"` na tag `<script>`:

```
<body>
  <header id="navbar"></header>
  <main id="app"></main>
  <script src="src/js/main.js" type="module"></script>
</body>
```

Duas mudanças em relação ao capítulo anterior:

- Adicionamos `type="module"` ao script.
- Renomeamos o arquivo de entrada para `main.js` e o colocamos em `src/js/` (começando a organização em pastas).
- Adicionamos um `<header id="navbar">` — o ponto de montagem do menu, que construiremos no próximo capítulo.

⚠ Cuidado — precisa de servidor! Como avisamos no Capítulo 0: com `type="module"`, o projeto **exige** um servidor HTTP. Abrir o `index.html` com `file://` faz os `import` falharem com um erro de CORS. Use o **Live Server**. Se algo “parar de funcionar” agora, é quase sempre isso.

8.3 `export` e `import` : as duas formas

Há duas maneiras de exportar de um módulo.

Exportação default (uma por arquivo)

Use quando o arquivo tem **uma coisa principal** a oferecer:

```
// arquivo: saudacao.js
export default function saudacao(nome) {
  return `Olá, ${nome}!`;
}
```

```
// arquivo: main.js
import saudacao from "./saudacao.js";
console.log(saudacao("Maria")); // "Olá, Maria!"
```

Na importação default, você **escolhe o nome** (não precisa ser igual) e **não usa chaves**.

Exportação nomeada (várias por arquivo)

Use quando o arquivo oferece **várias** funções:

```
// arquivo: matematica.js
export function somar(a, b) { return a + b; }
export function subtrair(a, b) { return a - b; }
```

```
// arquivo: main.js
import { somar, subtrair } from "../matematica.js";
console.log(somar(2, 3)); // 5
```

Na importação nomeada, você usa **chaves** `{ }` e os nomes **devem bater** exatamente com os exportados.

Resumo:

Tipo	Exporta	Importa
default	<code>export default funcao</code>	<code>import qualquerNome from "..."</code>
nomeada	<code>export function funcao() {}</code>	<code>import { funcao } from "..."</code>

🧩 **Montando o quebra-cabeça** O App Livros usa as **duas** formas, cada uma no lugar certo:

- **default** para cada **página** (uma página, um arquivo, um export principal): `export default { url, label, pagina } .`
- **nomeada** para os **serviços** que oferecem várias funções: `export { buscarListaLivros, buscarDetalheLivro } .` Ao longo dos próximos capítulos você verá cada padrão nascer naturalmente.

⚠️ **Cuidado — a extensão `.js` é obrigatória** No navegador, os caminhos de import precisam da extensão completa: `import x from "../home.js"` — **não** `../home` . (Em ferramentas como o Node/bundlers isso é opcional, mas no navegador puro, não.) Repare também no `./` no começo: ele significa “na mesma pasta”. Use `../` para “subir uma pasta”.

8.4 Transformando cada página em um módulo

Agora a grande mudança de mentalidade. No Capítulo 7, cada página era uma **string** solta. Vamos promovê-la a um **módulo próprio**, cuja página é uma **função** (Cap. 3) que recebe o elemento `app` e injeta o conteúdo nele.

Crie a pasta `src/js/components/paginas/` e, dentro, o arquivo `home.js`:

```
// src/js/components/paginas/home.js
async function home(app) {
  const paginaInicial = `<h1>Esta é a página Inicial</h1>`;
  app.innerHTML = paginaInicial;
}

export default {
  url: "#home",
  label: "Home",
  pagina: home
};
```

Pare e observe a genialidade dessa estrutura — ela é o **DNA de todo o projeto**. Cada página exporta um **objeto** (Cap. 2) com três informações:

- **url** — o hash que ativa esta página (`"#home"`).
- **label** — o texto que aparecerá no menu (`"Home"`).
- **pagina** — a **função** que sabe desenhar esta página.

Ou seja, cada arquivo carrega tudo o que se precisa saber sobre uma página: seu endereço, seu nome de menu e como desenhá-la. Isso vai nos permitir montar tanto o roteador quanto o menu **automaticamente** a partir dessa lista.

Agora a página `sobre.js`, no mesmo formato:

```
// src/js/components/paginas/sobre.js
function sobre(app) {
  const conteudo = `<h1>Esta é a página Sobre</h1>
    <p>Este site é um exemplo de SPA usando JavaScript puro</p>`;
  app.innerHTML = conteudo;
}

export default {
  url: "#sobre",
  label: "Sobre",
  pagina: sobre
};
```

💡 **Nos bastidores — por que `async` ?** Você reparou que `home` é `async function`, mas `sobre` não. Marcamos as páginas como `async` (assíncronas) por **padronização e antecipação**: algumas páginas (Livros, Rick and Morty) vão precisar **esperar** dados de uma API antes de renderizar, e para isso usarão `await` (Capítulo 14). Deixar todas `async` desde já mantém a interface uniforme — o roteador poderá tratar qualquer página do mesmo jeito. Por ora, o `async` numa página simples não faz diferença nenhuma; é só preparação para o futuro.

8.5 Uma página com lógica: o formulário de contato

Páginas nem sempre são estáticas. Muitas precisam **reagir ao usuário** depois de desenhadas. O módulo de página comporta isso perfeitamente: além de montar o HTML, a função pode registrar eventos (Cap. 5). Vamos construir a página de **contato**, que exibe um formulário e vai listando as mensagens enviadas.

Crie `src/js/components/paginas/contato.js` :

```

// src/js/components/paginas/contato.js
async function contato(app) {
  const paginaDeContato = `
    <h1>Esta é a página Contato</h1>
    <form id="formulario-de-contato">
      <div>
        <label for="assunto">Assunto</label>
        <input type="text" name="assunto" id="assunto">
      </div>
      <div>
        <label for="email">E-mail</label>
        <input type="email" name="email" id="email">
      </div>
      <div>
        <label for="mensagem">Mensagem</label>
        <textarea name="mensagem" id="mensagem" rows="5"></textarea>
      </div>
      <button type="submit">Enviar</button>
    </form>
    <ul id="lista_de_contatos"></ul>
  `;

  app.innerHTML = paginaDeContato;
  await capturarFormulario();
}

async function capturarFormulario() {
  const formulario = document.getElementById("formulario-de-contato");

  formulario.addEventListener("submit", function (event) {
    event.preventDefault(); // impede o recarregamento (Cap. 5)

    const lista = document.getElementById("lista_de_contatos");
    const li = document.createElement("li");

    const assunto = event.target[0].value;
    const email = event.target[1].value;
    const mensagem = event.target[2].value;

    li.textContent = `Assunto: ${assunto} | E-mail: ${email} | Mensagem: ${mensagem}`;
    lista.appendChild(li);

    // limpa os campos após o envio
    event.target[0].value = "";
    event.target[1].value = "";
    event.target[2].value = "";
  });
}

export default {

```



```
url: "#contato",
label: "Contato",
pagina: contato
};
```

Vamos destrinchar as ideias novas aqui, todas revisando o Capítulo 5:

- A função `contato` primeiro **desenha** o HTML (`innerHTML`) e **depois** chama `capturarFormulario()`. A ordem importa: o formulário precisa existir na tela antes de tentarmos “escutar” seus eventos.
- `capturarFormulario` registra um ouvinte de `submit` e chama `event.preventDefault()` — sem isso, o formulário recarregaria a página e quebraria a SPA.
- `event.target` é o próprio formulário que disparou o evento. `event.target[0]`, `[1]`, `[2]` são seus campos, na ordem em que aparecem — e `.value` lê o que foi digitado.
- Cada envio cria um `<li>` novo com `createElement` e o anexa à lista com `appendChild`.
- Por fim, esvaziamos os campos (`... .value = ""`) para o próximo envio.

 **Nos bastidores — duas formas de ler os campos** Usamos `event.target[0].value` (por posição). Há uma alternativa por id, mais explícita e legível: `document.getElementById("assunto").value`. As duas funcionam. A por id é menos frágil (não quebra se você reordenar os campos), mas a por índice é mais concisa. Você encontrará ambas em código real.

 **Montando o quebra-cabeça** Esta página junta praticamente tudo o que vimos até aqui: **módulo** (este capítulo), **função** que desenha e outra que escuta (Cap. 3), **template string** com HTML (Cap. 1), **DOM e eventos** (Cap. 5). Se você entendeu o `contato.js`, você entendeu a essência de um “componente de página”.

8.6 O `main.js` importando os módulos

Com as páginas viradas módulos, o `main.js` fica responsável por **juntá-las**. Uma primeira versão (que vamos aprimorar nos próximos capítulos) seria:

```
// src/js/main.js
import home from "../components/paginas/home.js";
import sobre from "../components/paginas/sobre.js";
import contato from "../components/paginas/contato.js";

const app = document.getElementById("app");

function renderizarRotaAtual() {
  const hash = window.location.hash || "#home";

  if (hash === home.url) {
    home.pagina(app);
  } else if (hash === sobre.url) {
    sobre.pagina(app);
  } else if (hash === contato.url) {
    contato.pagina(app);
  } else {
    app.innerHTML = "<h1>Página não encontrada (404)</h1>";
  }
}

renderizarRotaAtual();
window.addEventListener("hashchange", renderizarRotaAtual);
```

Repare que já é uma evolução do Capítulo 7: em vez de strings soltas, comparamos com `home.url` e chamamos `home.pagina(app)` — usando as propriedades do objeto exportado por cada módulo. O código ficou mais expressivo.

Mas... ainda temos aquele `if/else` que cresce a cada página, e ainda não temos menu. Guarde essa incômoda sensação: ela é o combustível para os próximos dois capítulos, onde eliminaremos a repetição de vez.

Recapitulando

- **Módulos** são arquivos que **exportam** e **importam** funcionalidades, trazendo organização, reutilização, isolamento e clareza.
- Ative-os com `<script type="module">` — e lembre: **exige servidor**.
- **export default** (uma coisa principal) x **export nomeado** (várias coisas); no import, o default vai sem chaves e o nomeado com chaves.
- Cada **página** virou um módulo que exporta um objeto `{ url, label, pagina }`, onde `pagina` é a **função** que a desenha.
- Uma página pode ter **lógica** própria: desenha o HTML e depois registra eventos — como o formulário de contato.

Exercícios do Capítulo 8

1. Converta a página “Serviços” do capítulo anterior num módulo `servicos.js`, exportando `{ url: "#servicos", label: "Serviços", pagina }`.
2. Crie um módulo utilitário `saudacoes.js` com **duas exportações nomeadas**: `bomDia()` e `boaNoite()`. Importe ambas no `main.js` e teste.
3. Reproduza a página de contato e teste: envie três mensagens e confirme que elas aparecem na lista, sem a página recarregar.
4. No `contato.js`, troque a leitura por índice (`event.target[0].value`) pela leitura por id (`document.getElementById("assunto").value`) e confirme que continua funcionando.
5. **Reflexão:** por que a chamada `capturarFormulario()` precisa vir **depois** de `app.innerHTML = ...`? O que aconteceria se invertêssemos a ordem?

Capítulo 9 — Páginas como objetos e o menu dinâmico

No capítulo anterior, cada página passou a exportar um objeto `{ url, label, pagina }`. Foi uma decisão de projeto aparentemente pequena, mas que agora vai render frutos enormes. Neste capítulo, vamos usar essa estrutura para **gerar o menu de navegação automaticamente** — de modo que adicionar uma página nova ao projeto seja trivial.

Este é o momento em que o projeto começa a parecer “mágico”: você adiciona uma rota, e o menu se atualiza sozinho.

9.1 O problema que vamos resolver

Lembra do desconforto do Capítulo 7? Tínhamos **duas listas** para manter sincronizadas manualmente:

1. A cadeia de `if/else` que decidia qual página mostrar.
2. O HTML do menu, com um `<li>` para cada página.

Toda vez que adicionávamos uma página, precisávamos mexer nos **dois** lugares. Esquecer um deles gerava bugs (“a página existe mas não aparece no menu”, ou “o link está no menu mas não faz nada”). Isso viola um princípio importante da programação:

DRY — Don’t Repeat Yourself (“não se repita”). Cada informação deve ter uma **única fonte de verdade**.

Nossa fonte de verdade será **uma lista de rotas**. Dela nascerão tanto a navegação quanto o menu. Uma lista, dois usos, zero duplicação.

9.2 A lista de rotas: `rotas.js`

Vamos criar um módulo que **reúne todas as páginas** numa única lista. Como cada página já exporta um objeto, basta importá-las e colocá-las num array (Cap. 2).

Crie `src/js/components/rotas/rotas.js`:

```
// src/js/components/rotas/rotas.js
import home from "../paginas/home.js";
import sobre from "../paginas/sobre.js";
import servicos from "../paginas/servicos.js";
import contato from "../paginas/contato.js";

const roteador = [
  home,
  sobre,
  servicos,
  contato
];

export default roteador;
```

Simples assim. `roteador` é um **array de objetos** — exatamente a estrutura que estudamos no Capítulo 2. Cada elemento é o objeto `{ url, label, pagina }` de uma página. Se quisermos ver seu conteúdo, seria algo como:

```
[
  { url: "#home",    label: "Home",    pagina: home    },
  { url: "#sobre",   label: "Sobre",   pagina: sobre   },
  { url: "#servicos", label: "Serviços", pagina: servicos },
  { url: "#contato", label: "Contato",  pagina: contato }
]
```

 **Montando o quebra-cabeça** Esta lista é o **coração** do App Livros. Ela é a tal “fonte única de verdade”. Adicionar uma página ao projeto vai se resumir a: (1) criar o módulo da página e (2) importá-lo e incluí-lo neste array. Só isso. O menu e o roteamento se viram sozinhos a partir daqui. Ao final do curso, este array terá sete páginas — e nunca precisaremos tocar no menu ou no roteador para isso.

9.3 Gerando o menu com `.map()`

Agora a estrela do capítulo. Queremos transformar aquele array de rotas em HTML de menu. Para cada rota, queremos um item assim:

```
<li><a href="#home">Home</a></li>
```

Poderíamos usar um `for...of` (Cap. 4) e ir concatenando numa string. Funciona. Mas existe um método de array feito sob medida para “transformar cada item em outra coisa”: o `.map()`.

O `.map()` percorre um array, aplica uma função (Cap. 3) a **cada** elemento, e devolve um **novo array** com os resultados transformados:

```
const numeros = [1, 2, 3];
const dobrados = numeros.map(n => n * 2); // [2, 4, 6]
```

Aplicando às rotas: para cada `item` (uma rota), devolvemos uma string de HTML:

```
const links = roteador.map(item => `
  <li class="bem-navbar__item">
    <a href="${item.url}" class="bem-navbar__link">${item.label}</a>
  </li>`);
```

O resultado é um **array de strings de HTML**, uma para cada rota. Repare como usamos `item.url` e `item.label` — as mesmas propriedades que definimos lá no módulo de cada página. Tudo se conecta.

Juntando com `.join("")`

`.map()` nos dá um **array** de strings, mas o `innerHTML` precisa de **uma string só**. Para colar todos os pedaços, usamos `.join("")` (Cap. 2), que une os elementos do array sem nenhum separador:

```
const links = roteador.map(item => `...${item.label}...`).join("");
```

⚠ Cuidado — não esqueça o `.join("")` Se você jogar o array direto no `innerHTML`, o JavaScript vai juntar os itens com **vírgulas** (comportamento padrão ao converter array em texto), e aparecerão vírgulas estranhas entre os itens do menu. O `.join("")` (com string vazia) elimina qualquer separador. É um erro clássico — fique atento.

9.4 O componente `navbar.js`

Vamos empacotar essa lógica num componente reutilizável. Crie `src/js/components/navbar/navbar.js`:

```
// src/js/components/navbar/navbar.js
function navbar(itens) {
  const nav = document.getElementById("navbar");

  const links = itens.map(item => `
    <li class="bem-navbar__item">
      <a href="${item.url}" class="bem-navbar__link">${item.label}</a>
    </li>`).join("");

  nav.innerHTML = `
    <nav class="bem-navbar">
      <a href="#" class="bem-navbar__brand">Brand</a>
      <ul class="bem-navbar__menu">${links}</ul>
    </nav>`;
}

export default navbar;
```

Analizando:

- `navbar` é uma **função** que recebe `itens` (a lista de rotas) como parâmetro. Ela não sabe *quais* são as rotas — recebe-as de fora. Isso a torna genérica e reutilizável.
- Encontra o ponto de montagem `<header id="navbar">` (definido no `index.html` do capítulo anterior).
- Usa `.map()` + `.join("")` para transformar as rotas em `<li>` s.
- Injeta tudo num `<nav>` com uma marca ("Brand") e o menu.



Nos bastidores — o que é `bem-navbar`? As classes `bem-navbar`, `bem-navbar__item`, `bem-navbar__link` seguem uma convenção de nomes chamada **BEM** (Bloco, Elemento, Modificador), que dá organização ao CSS. O `bem-` é o prefixo do nosso microframework de estilos. Vamos entender tudo isso no Capítulo 11. Por enquanto, apenas mantenha as classes — elas farão o menu ficar bonito quando o CSS entrar.

9.5 Ligando o menu no `main.js`

Agora usamos o componente `navbar` a partir do ponto de entrada. Atualize o `main.js` para importar as rotas e a navbar, e chamar a navbar passando a lista:

```
// src/js/main.js
import navbar from "../components/navbar/navbar.js";
import roteador from "../components/rotas/rotas.js";

const app = document.getElementById("app");

function renderizarRotaAtual() {
  const hash = window.location.hash || "#home";

  // encontra a rota cujo url bate com o hash atual
  for (const rota of roteador) {
    if (rota.url === hash) {
      rota.pagina(app);
      return;
    }
  }
  app.innerHTML = "<h1>Página não encontrada (404)</h1>";
}

function iniciar() {
  navbar(roteador); // desenha o menu a partir das rotas
  renderizarRotaAtual(); // desenha a página inicial
  window.addEventListener("hashchange", renderizarRotaAtual);
}

iniciar();
```

Duas grandes conquistas aqui:

1. **O menu nasce das rotas.** `navbar(roteador)` passa a lista para o componente, que gera os itens automaticamente. Sem mais HTML de menu escrito à mão.
2. **O roteamento também percorre as rotas.** Em vez de um `if/else` para cada página, um único `for...of` (Cap. 4) procura a rota cujo `url` bate com o hash e chama sua função `pagina`. Adicionar uma página não exige mais um novo `else if`.

Agora faça o teste que vale ouro: adicione uma página nova. Crie o módulo, importe-o em `rotas.js`, inclua-o no array `roteador`. **Pronto** — ela aparece no menu e funciona na navegação, sem você tocar em mais nada. Essa é a recompensa da fonte única de verdade.

🧩 **Montando o quebra-cabeça** A função `iniciar()` que acabamos de escrever é **quase idêntica** à versão final do projeto. Compare com o `main.js` real:

```
function iniciar() {  
  navbar(roteador);  
  renderizarRotaAtual();  
  window.addEventListener("hashchange", renderizarRotaAtual);  
}  
iniciar();
```

É exatamente o que temos agora! A única evolução que falta é trocar o `for...of` de busca por uma estrutura ainda mais rápida e elegante — o **mapa de rotas** — que é o assunto do próximo capítulo.

9.6 Um segundo exemplo de `.map()` : a página de serviços

Para fixar o `.map()` (ou seu primo `for`), vale ver outra tela do projeto que transforma dados em HTML: a página de **serviços**, que exibe uma grade de cards a partir de um array de objetos. Crie `src/js/components/paginas/servicos.js`:

```
// src/js/components/paginas/servicos.js
const detalhes = [
  { titulo: "Jogo das quartas de final", descricao: "...", imagem: "src/img/2002_1.w"},
  { titulo: "Jogo especial", descricao: "...", imagem: "src/img/2002_2.j"},
  { titulo: "Camisa azul", descricao: "...", imagem: "src/img/2002_3.j"},
  { titulo: "Ronaldos", descricao: "...", imagem: "src/img/2002_4.w"}
];

function servicos(app) {
  let cardServico = `<div class="bem-grid-auto">`;

  for (let i = 0; i < detalhes.length; i++) {
    cardServico += `
    <div class="bem-card">
      
        <h3 class="bem-card__title">${detalhes[i].titulo}</h3>
        <p>${detalhes[i].descricao}</p>
      </div>
    </div>`;
  }

  cardServico += `</div>`;
  app.innerHTML = cardServico;
}

export default {
  url: "#servicos",
  label: "Serviços",
  pagina: servicos
};
```

Aqui usamos o **for clássico** (Cap. 4) em vez do `.map()`, para você ver que ambos servem — a escolha é de estilo. O padrão é o mesmo do Capítulo 5: `let cardServico = "..."` → laço que faz `cardServico += ...` → `innerHTML`. Percorremos um **array de objetos** e transformamos cada um num **card**.

 **Nos bastidores** — `.map()` ou **for**? O `.map()` brilha quando você quer **transformar** uma lista em outra e já devolvê-la. O `for` / `for...of` é mais flexível quando há lógica extra no meio. No App Livros usamos `.map()` para o menu (transformação limpa) e `for...of` para as listagens de cards. Não existe “certo” e “errado” — existe legível.

Recapitulando

- O princípio **DRY** manda ter uma **fonte única de verdade**; a nossa é o array `roteador` em `rotas.js`.
 - Esse array reúne os objetos `{ url, label, pagina }` de todas as páginas.
 - O `.map()` transforma cada rota num `<li>` de menu; o `.join("")` cola tudo numa string.
 - O componente `navbar(itens)` gera o menu automaticamente a partir das rotas.
 - O `main.js` agora **percorre** as rotas para renderizar e monta o menu com uma linha (`navbar(roteador)`).
 - Adicionar uma página passou a ser trivial: criar o módulo e incluí-lo no array.
-

Exercícios do Capítulo 9

1. Monte o `rotas.js` com as quatro páginas (home, sobre, serviços, contato) e o `navbar.js`. Confirme que o menu aparece com os quatro itens.
2. Crie uma nova página `faq.js` (Perguntas Frequentes) e adicione-a às rotas. Confirme que ela aparece no menu e funciona **sem** você editar o `navbar.js` nem o roteamento.
3. Explique, com suas palavras, por que o `.join("")` é necessário depois do `.map()`. Teste remover o `.join("")` e observe o que acontece no menu.
4. Reescreva a geração de cards da página de serviços usando `.map()` + `.join("")` em vez do `for` clássico.
5. **Reflexão:** o parâmetro `itens` da função `navbar` poderia se chamar `roteador`. Por que dar a ele um nome genérico (`itens`) torna a função mais reutilizável?

Capítulo 10 — Centralizando o roteamento

Nossa SPA já funciona muito bem: o menu se gera sozinho e a navegação percorre a lista de rotas. Mas ainda há uma pequena ineficiência: a cada navegação, percorremos o array inteiro com um `for...of` procurando a rota certa. Com poucas páginas isso é irrelevante, mas é uma ótima oportunidade para aprender uma técnica elegante e chegar à **versão definitiva** do `main.js` — exatamente como ela existe no projeto real.

Neste capítulo transformamos a **lista** de rotas em um **mapa** de rotas.

10.1 Lista x Mapa: o problema da busca

Hoje, para achar a rota de `#contato`, fazemos assim:

```
for (const rota of roteador) {
  if (rota.url === hash) {
    rota.pagina(app);
    return;
  }
}
```

Isso funciona, mas é uma **busca linear**: no pior caso, percorremos todas as rotas até achar (ou não achar) a certa. É como procurar um contato no celular rolando a lista inteira, um por um.

E se, em vez de uma lista, tivéssemos uma estrutura onde pudéssemos ir **direto** à rota pelo seu hash? Como um dicionário: você abre na palavra que quer, sem ler da primeira à última página. É isso que um **mapa de rotas** faz.

10.2 Construindo o mapa de rotas

A ideia: transformar o **array** de rotas em um **objeto** (Cap. 2) onde a **chave** é o `url` da rota e o **valor** é a própria rota. Assim:

```
// de um array...
[
  { url: "#home", label: "Home", pagina: home },
  { url: "#sobre", label: "Sobre", pagina: sobre }
]

// ...para um objeto (mapa):
{
  "#home": { url: "#home", label: "Home", pagina: home },
  "#sobre": { url: "#sobre", label: "Sobre", pagina: sobre }
}
```

Com o mapa, buscar uma rota é instantâneo: `mapa["#sobre"]` já devolve a rota, sem percorrer nada. Acessar uma propriedade de objeto por chave é uma operação direta.

Vamos escrever uma função que faz essa transformação. Ela percorre o array (Cap. 4) e vai preenchendo o objeto:

```
function criarMapaDeRotas(rotas) {
  const mapa = {};
  for (const rota of rotas) {
    mapa[rota.url] = rota; // a chave é o url; o valor é a rota inteira
  }
  return mapa;
}
```

Repare no uso da **notação de colchetes** (`mapa[rota.url]`), do Capítulo 2: como a chave (`rota.url`) está numa variável e muda a cada volta, não podemos usar a notação de ponto. Os colchetes permitem usar o valor da variável como nome da propriedade.



Nos bastidores Fazemos essa conversão **uma única vez**, quando a aplicação carrega. A partir daí, todas as navegações usam o mapa pronto. É o clássico “gaste um pouco na preparação para economizar muito no uso repetido”.

10.3 A página 404 como um objeto

E se o usuário digitar um hash que não existe? No mapa, `mapa["#inexistente"]` devolve `undefined`. Precisamos de um plano B. Vamos criar uma “página” especial de erro, no mesmo formato das outras (uma função que desenha algo em `app`):

```
const paginaNaoEncontrada = {
  pagina: (app) => {
    app.innerHTML = "<div>Página não encontrada 404</div>";
  }
};
```

Note que ela é um objeto com uma propriedade `pagina` que é uma **arrow function** (Cap. 3). Assim ela tem a mesma “cara” das rotas de verdade e pode ser tratada do mesmo jeito. Essa uniformidade vai deixar o código de renderização lindo.

10.4 A função de renderização definitiva

Agora juntamos tudo. A renderização pega o hash, busca a rota no mapa (ou usa a 404) e chama sua função `pagina`:

```
async function renderizarRotaAtual() {
  const hash = window.location.hash || "#home";
  const rota = mapaDeRotas[hash] || paginaNaoEncontrada;
  await rota.pagina(app);
}
```

Três linhas, e cada uma vale ouro:

1. `const hash = window.location.hash || "#home";` — pega o hash atual, ou assume `#home` se estiver vazio (a rota padrão, Cap. 7).
2. `const rota = mapaDeRotas[hash] || paginaNaoEncontrada;` — busca a rota no mapa. Se não existir (`undefined`, que é “falso”), o `||` entrega a `paginaNaoEncontrada`. Aqui está o tratamento de 404, elegante e sem `if`.
3. `await rota.pagina(app);` — chama a função da página, passando o `app`.

 **Nos bastidores — por que `await` aqui?** Lembra que fizemos as páginas `async` no Capítulo 8? Como algumas vão buscar dados de APIs (e isso demora), a função `pagina` pode retornar uma *Promise*. O `await` (Capítulo 14) garante que, se a página for assíncrona, esperamos ela terminar. Para páginas simples, o `await` não atrapalha em nada. Colocá-lo aqui torna o roteador capaz de lidar com **qualquer** página — síncrona ou assíncrona — do mesmo jeito. É a padronização do Capítulo 8 dando lucro.

10.5 O `main.js` completo e final

Reunindo cada peça, este é o `main.js` definitivo do App Livros — o mesmo do projeto real:

```
// src/js/main.js
import navbar from "../components/navbar/navbar.js";
import roteador from "../components/rotas/rotas.js";

const ROTA_PADRAO = "#home";

const app = document.getElementById("app");
const mapaDeRotas = criarMapaDeRotas(roteador);
const paginaNaoEncontrada = {
  pagina: (app) => { app.innerHTML = "<div>Página não encontrada 404</div>"; }
};

function criarMapaDeRotas(rotas) {
  const mapa = {};
  for (const rota of rotas) {
    mapa[rota.url] = rota;
  }
  return mapa;
}

async function renderizarRotaAtual() {
  const hash = window.location.hash || ROTA_PADRAO;
  const rota = mapaDeRotas[hash] || paginaNaoEncontrada;
  await rota.pagina(app);
}

function iniciar() {
  navbar(roteador);
  renderizarRotaAtual();
  window.addEventListener("hashchange", renderizarRotaAtual);
}

iniciar();
```

Vamos ler o arquivo inteiro, de cima a baixo, como um mapa da aplicação:

- **Imports:** trazemos o menu (`navbar`) e a lista de rotas (`roteador`).
- **ROTA_PADRAO** : uma constante para a rota inicial. Usar uma constante nomeada (em vez de espalhar `"#home"` pelo código) é boa prática — se um dia mudar a home, muda num lugar só.
- **app** : o ponto de montagem principal, buscado uma vez.
- **mapaDeRotas** : o dicionário de rotas, criado na hora.
- **paginaNaoEncontrada** : o plano B para hashes inválidos.

- `criarMapaDeRotas` : a função que converte lista em mapa.
- `renderizarRotaAtual` : decide e desenha a página atual.
- `iniciar` : orquestra a inicialização — desenha o menu, renderiza a primeira página e passa a escutar o `hashchange` .
- `iniciar()` : dá a partida em tudo.

⚠ Cuidado — hoisting em ação Repare que usamos `criarMapaDeRotas(roteador)` na linha do `const mapaDeRotas` , **antes** de a função aparecer no arquivo. Isso só funciona porque funções declaradas com `function` sofrem **hoisting** (Cap. 3) — são “içadas” para o topo. Se `criarMapaDeRotas` fosse uma arrow function guardada em `const` , daria erro. É um exemplo real de por que entender hoisting importa.

10.6 O ciclo de vida completo da navegação

Vamos rastrear, passo a passo, o que acontece quando o usuário clica em “Sobre”. Este é o resumo de **tudo** que construímos na Parte IV:

```
1. Usuário clica no link <a href="#sobre"> do menu
   |
2. O navegador muda window.location.hash para "#sobre" (sem recarregar!)
   |
3. A mudança dispara o evento "hashchange"
   |
4. O navegador chama renderizarRotaAtual() (nosso callback)
   |
5. A função lê o hash: "#sobre"
   |
6. Busca no mapa: mapaDeRotas["#sobre"] → a rota do Sobre
   |
7. Chama rota.pagina(app) → a função sobre(app)
   |
8. sobre(app) faz app.innerHTML = "<h1>Sobre</h1>..."
   |
9. A tela mostra a página Sobre. Fim.
```

Cada seta desse fluxo passa por um conceito que estudamos: eventos (Cap. 5), hash (Cap. 6), funções e callbacks (Cap. 3), objetos e mapas (Cap. 2), DOM (Cap. 5). **Você construiu um roteador de SPA do zero, entendendo cada peça.** Isso é exatamente o que bibliotecas como o React Router fazem — só que agora não é mágica para você.

Recapitulando

- Um **mapa de rotas** (objeto cuja chave é o `url`) permite achar uma rota **instantaneamente**, sem percorrer a lista.
 - `criarMapaDeRotas` converte o array em objeto usando a **notação de colchetes**.
 - A **página 404** é modelada como um objeto com `pagina`, igual às demais — uniformidade que simplifica o código.
 - `renderizarRotaAtual` resume tudo em três linhas, usando `||` para rota padrão e para o fallback 404, e `await` para suportar páginas assíncronas.
 - O `main.js` final orquestra menu + render + escuta do `hashchange`.
 - Você domina, agora, o **ciclo de vida completo** de uma navegação em SPA.
-

Exercícios do Capítulo 10

1. Implemente o `main.js` final e confirme que a navegação continua funcionando exatamente como antes — mas agora via mapa de rotas.
2. No Console, depois que a página carregar, digite `mapaDeRotas` para inspecionar o objeto gerado. Confirme que as chaves são os hashes.
3. Digite um hash inexistente na barra (ex.: `#xyz`) e confirme que a página 404 aparece.
4. Explique por que `mapaDeRotas[hash] || paginaNaoEncontrada` trata o caso de rota inexistente sem precisar de um `if`.
5. **Desafio:** o `for...of` do Capítulo 9 e o mapa deste capítulo produzem o mesmo resultado visível. Escreva um parágrafo explicando a vantagem do mapa e em que cenário (muitas rotas, muitas navegações) ela ficaria perceptível.

Capítulo 11 — Estilo com um microframework CSS e temas

Nossa SPA funciona, mas está feia — texto preto num fundo branco. Neste capítulo damos a ela uma cara profissional criando um **microframework CSS** próprio: um pequeno conjunto de classes reutilizáveis, no espírito de um “mini Bootstrap”. De quebra, vamos implementar **temas** (dia, tarde, noite) com uma técnica elegante de CSS moderno.

Este é um respiro no meio da jornada JavaScript — mas um respiro importante, porque as classes que criarmos aqui são as mesmas que já vínhamos usando no HTML das páginas (`bem-card` , `bem-navbar` , etc.).

11.1 Por que um “microframework”?

Você poderia estilizar cada elemento individualmente, mas isso gera repetição e inconsistência. Frameworks como Bootstrap e Tailwind resolvem isso com **classes utilitárias** e **componentes prontos**. Vamos criar uma versão enxuta e caseira para entender o princípio:

- **Componentes:** classes que estilizam blocos inteiros (`bem-card` , `bem-navbar` , `bem-btn` , `bem-modal`).
- **Utilitários:** classes de propósito único (`bem-mt-md` para margem, `bem-flex` para display flex, `bem-text-center` para centralizar).

Todas com o prefixo `bem-` , para não colidirem com nada.

11.2 A metodologia BEM

BEM significa **Bloco, Elemento, Modificador**. É uma convenção de **nomenclatura de classes** que torna o CSS previsível e legível. A estrutura:

```
.bloco           → um componente independente   (.bem-card)
.bloco__elemento → uma parte do bloco             (.bem-card__title)
.bloco--modificador → uma variação do bloco       (.bem-btn--primary)
```

- **Bloco:** a entidade autônoma. Ex.: `bem-card` (um cartão).
- **Elemento** (`__`): uma peça que só faz sentido dentro do bloco. Ex.: `bem-card__title` (o título do cartão), `bem-card__image` (a imagem do cartão).

- **Modificador** (`--`): uma variação de aparência ou estado. Ex.: `bem-btn--primary` (botão na cor primária), `bem-modal--hidden` (modal escondido).

Você já vinha usando esses nomes! Lembre do menu (`bem-navbar__link`) e dos cards de serviço (`bem-card__title`). Agora você entende a lógica por trás deles.



Nos bastidores A grande vantagem do BEM é que, só de ler uma classe, você sabe o que ela é e onde vive. `bem-modal__close` é, sem dúvida, “o botão de fechar do modal”. Isso evita o caos de nomes genéricos como `.titulo` ou `.botao`, que colidem e confundem em projetos grandes.

11.3 Variáveis CSS: a base dos temas

O CSS moderno tem **variáveis** (oficialmente, “custom properties”). Definimos valores num lugar e os reutilizamos em todo o estilo. Elas se declaram dentro de `:root` (a raiz do documento) com o prefixo `--`:

```
:root {
  --bem-primary: #3b82f6;
  --bem-bg: #ffffff;
  --bem-text: #1f2937;
  --bem-spacing-md: 1rem;
  --bem-radius-md: .5rem;
}
```

E as usamos com a função `var()`:

```
body {
  background: var(--bem-bg);
  color: var(--bem-text);
}
.bem-btn--primary {
  background-color: var(--bem-primary);
}
```

A vantagem é enorme: para mudar a cor primária do site inteiro, você altera **uma linha**. E, como veremos já a seguir, isso é o que torna os temas triviais.

11.4 Implementando temas com `data-theme`

A sacada dos temas: se as cores são variáveis, basta **redefinir essas variáveis** para trocar a aparência inteira. Fazemos isso condicionando os valores a um atributo `data-theme` no elemento raiz (`<html>`):

```
/* tema padrão (claro) */
:root {
  --bem-primary: #3b82f6;
  --bem-bg: #ffffff;
  --bem-text: #1f2937;
}

/* tema noite */
[data-theme="noite"] {
  --bem-primary: #2dd4bf;
  --bem-bg: #0f172a;
  --bem-surface: #1e293b;
  --bem-text: #f1f5f9;
  --bem-border: #334155;
}

/* tema tarde */
[data-theme="tarde"] {
  --bem-primary: #f97316;
  --bem-bg: #fdf5ee;
  --bem-text-muted: #78716c;
}
```

Quando o `<html>` tem `data-theme="noite"`, o navegador aplica o novo conjunto de variáveis, e **todo** o site que usa `var(--bem-bg)`, `var(--bem-text)` etc. muda de cor instantaneamente. Nada de reescrever estilos por componente.

Para alternar o tema via JavaScript (aplicando o que aprendemos sobre DOM no Cap. 5), basta escrever no atributo do `<html>`:

```
document.documentElement.setAttribute("data-theme", "noite"); // ativa a noite
document.documentElement.removeAttribute("data-theme"); // volta ao padrão
```

 **Nos bastidores — a transição suave** No `body` colocamos `transition: background .25s, color .25s;`. Isso faz a troca de tema acontecer com uma **animação suave** de um quarto de segundo, em vez de um “pisca” abrupto. Um detalhe pequeno que transmite polimento.

11.5 Anatomia de um componente: o card

Vamos ver como um componente BEM completo é montado. O card, que usamos nas listagens, é assim:

```
.bem-card {
  background-color: var(--bem-surface);
  border-radius: var(--bem-radius-lg);
  box-shadow: var(--bem-shadow-md);
  overflow: hidden;
  transition: all .3s ease;
}
.bem-card:hover {
  box-shadow: var(--bem-shadow-lg);
  transform: translateY(-2px); /* "levanta" um pouco ao passar o mouse */
}
.bem-card__image { width: 100%; height: auto; display: block; }
.bem-card__body { padding: var(--bem-spacing-lg); }
.bem-card__title { font-size: 1.25rem; font-weight: 600; }
.bem-card__subtitle { font-size: .875rem; color: var(--bem-text-muted); }
```

Observe como cada peça (`__image` , `__body` , `__title`) é estilizada separadamente, e como tudo usa **variáveis** (`var(--bem-...)`). Trocar o tema recolora o card sem tocar nessas regras. Esse mesmo padrão se repete para `bem-navbar` , `bem-btn` , `bem-form` , `bem-modal` e `bem-alert` .

11.6 Classes utilitárias

Além dos componentes, o microframework traz dezenas de **utilitários** — classes minúsculas de uma responsabilidade só, para ajustes rápidos direto no HTML:

```
.bem-flex          { display: flex; }
.bem-justify-center { justify-content: center; }
.bem-items-center  { align-items: center; }
.bem-gap-md        { gap: var(--bem-spacing-md); }
.bem-mt-lg         { margin-top: var(--bem-spacing-lg); }
.bem-mb-md         { margin-bottom: var(--bem-spacing-md); }
.bem-text-center   { text-align: center; }
```

E uma classe especialmente útil para as nossas listagens, que cria uma grade responsiva automática:

```
.bem-grid-auto {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: var(--bem-spacing-md);
}
```

O `repeat(auto-fit, minmax(280px, 1fr))` significa: “crie quantas colunas couberem, cada uma com no mínimo 280px”. Numa tela larga, cabem várias colunas; numa estreita (celular), elas se reorganizam em uma só — **sem media query**.

🧩 **Montando o quebra-cabeça** É `bem-grid-auto` que organiza os cards de livros, personagens e serviços numa grade bonita e responsiva. Lembra do `<div id="listaLivros" class="bem-grid-auto">`? Agora você sabe o que aquela classe faz.

11.7 Responsividade e o menu mobile

Por fim, o microframework cuida do celular. Com **media queries**, ajustamos o layout em telas pequenas. O caso mais interessante é o menu, que vira um “menu hambúrguer” (☰):

```
@media (max-width: 768px) {
  .bem-navbar__menu { display: none; } /* esconde o menu por padrão */
  .bem-navbar__toggle { display: block; } /* mostra o botão ☰ */

  /* quando o checkbox invisível está marcado, mostra o menu */
  .bem-navbar__checkbox:checked ~ .bem-navbar__menu {
    display: flex;
    flex-direction: column;
    /* ...posicionamento... */
  }
}
```

💡 **Nos bastidores — o “checkbox hack”** Repare que o menu mobile abre e fecha **sem uma linha de JavaScript**! O truque usa um `<input type="checkbox">` escondido e um `<label>` (o ☰). Clicar no label marca/desmarca o checkbox, e o seletor CSS `:checked ~ .bem-navbar__menu` mostra o menu conforme o estado. É um exemplo lindo de como CSS bem pensado reduz a necessidade de JavaScript.

11.8 Ligando o CSS ao projeto

Para tudo isso valer, basta ligar a folha de estilo no `index.html`, no `<head>`:

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>App Livros</title>
  <link rel="stylesheet" href="src/css/microframework.css">
</head>
```

Salve o `microframework.css` em `src/css/` e recarregue. De repente, todas aquelas classes que vínhamos escrevendo (`bem-card`, `bem-navbar__link`, `bem-btn--primary`) ganham vida. **O projeto fica bonito sem alterarmos uma linha de JavaScript** — prova de que separamos bem estrutura (HTML), comportamento (JS) e apresentação (CSS).

Recapitulando

- Um **microframework CSS** reúne **componentes** (`bem-card`) e **utilitários** (`bem-mt-md`) reutilizáveis, com prefixo `bem-`.
 - **BEM** organiza os nomes: **Bloco** (`bem-card`), **Elemento** (`__title`), **Modificador** (`--primary`).
 - **Variáveis CSS** (`--bem-primary`, usadas com `var()`) centralizam os valores de design.
 - **Temas** trocam a aparência inteira só redefinindo variáveis via `[data-theme="..."]` — e alternamos via JS com `setAttribute`.
 - Utilitários como `bem-grid-auto` dão grades responsivas sem esforço.
 - A **responsividade** e até o menu hambúrguer podem ser resolvidos com CSS puro.
-

Exercícios do Capítulo 11

1. Crie um `microframework.css` com pelo menos as variáveis de `:root`, o componente `bem-card` e os utilitários `bem-flex`, `bem-text-center` e `bem-mt-md`. Ligue-o no `index.html`.
2. Crie um tema `[data-theme="noite"]` redefinindo `--bem-bg` e `--bem-text`. No Console, ative-o com `document.documentElement.setAttribute("data-theme", "noite")`.
3. Crie um botão na página inicial que, ao ser clicado, alterna entre o tema padrão e o "noite" (dica: `classList.toggle` não serve aqui; use uma variável para lembrar o estado e `setAttribute` / `removeAttribute`).
4. Envolve os cards de serviços numa `<div class="bem-grid-auto">` e observe a grade responsiva redimensionando a janela.
5. Explique, com suas palavras, por que usar variáveis CSS torna a implementação de temas tão simples.

Capítulo 12 — Programação assíncrona: callbacks e o “callback hell”

Chegamos à parte mais empolgante do curso: trazer **dados do mundo real** para dentro do App Livros. Antes de pedir informações a um servidor pela internet, precisamos entender um conceito que muda a forma como o JavaScript pensa: o **assincronismo**.

Este capítulo é conceitual, mas fundamental. Sem entender o “por quê” do assincronismo, o `fetch` e o `async/await` dos próximos capítulos parecerão mágica incompreensível.

12.1 Síncrono x assíncrono

Programação síncrona é o modo “natural”: uma tarefa por vez, em ordem. A linha 2 só roda depois que a linha 1 termina completamente. É como uma fila de banco: o próximo só é atendido quando o anterior termina.

```
console.log("Primeiro");
console.log("Segundo");
console.log("Terceiro");
// saída, em ordem: Primeiro, Segundo, Terceiro
```

O problema aparece com tarefas **demoradas**. Imagine pedir dados a um servidor que fica do outro lado do mundo — isso pode levar segundos. Se o JavaScript fosse puramente síncrono, a **página inteira congelaria** durante a espera: nada de clicar, rolar ou digitar. Péssima experiência.

Programação assíncrona resolve isso: operações demoradas são “disparadas” e o programa **continua executando** o resto do código, sem esperar. Quando a operação termina, um aviso nos permite tratar o resultado.

Veja a diferença na prática:

```
console.log("Início");

setTimeout(() => {
  console.log("Pausa... (depois de 2 segundos)");
}, 2000);

console.log("Fim");
```

Qual a saída? Muita gente aposta em “Início, Pausa, Fim”. Mas o correto é:

```
Início
Fim
Pausa... (depois de 2 segundos)
```

O `setTimeout` agenda a função para daqui a 2 segundos e **não bloqueia** o código. Então “Fim” é impresso imediatamente, e só depois — quando os 2 segundos passam — aparece “Pausa”. O programa não ficou parado esperando.

💡 **Nos bastidores** — o `setTimeout` `setTimeout(funcao, milissegundos)` agenda uma função para rodar no futuro. É a forma mais simples de ver o assincronismo em ação. Repare que ele recebe uma **função (callback)** como primeiro argumento — voltamos ao Capítulo 3.

12.2 Por que a web precisa disso

O caso de uso número um do assincronismo é **buscar dados de um servidor**. Quando o App Livros pede a lista de livros a uma API na internet, essa resposta demora — pode ser 100 milissegundos numa boa conexão, ou vários segundos numa ruim. Durante essa espera:

- **Modo síncrono (ruim):** a página trava. O usuário não consegue fazer nada até os dados chegarem.
- **Modo assíncrono (bom):** a página continua responsiva. Podemos até mostrar um “Carregando...” enquanto esperamos, e preencher a tela quando os dados chegam.

Por isso, **toda comunicação com servidores em JavaScript é assíncrona**. Não é opcional — é assim que a linguagem funciona. Dominar isso é dominar o desenvolvimento web moderno.

12.3 Callbacks: a primeira solução

Como o JavaScript nos avisa quando uma tarefa assíncrona termina? Historicamente, a resposta foi: através de um **callback** (Cap. 3) — uma função que passamos para ser executada **quando** a tarefa terminar.

Vejamos um exemplo simples e síncrono, só para firmar o padrão:

```
function somar(a, b, callback) {
  const resultado = a + b;
  callback(resultado); // avisa "terminei, aqui está o resultado"
}

function mostrarResultado(resultado) {
  console.log(`O resultado é: ${resultado}`);
}

somar(1, 2, mostrarResultado); // "O resultado é: 3"
```

A função `somar` não sabe o que fazer com o resultado — ela apenas o entrega ao `callback`. Quem chamou decide o que acontece depois. Esse padrão — “faça isto e, quando terminar, chame aquilo” — é a essência do tratamento assíncrono.

Você já usa callbacks o tempo todo, aliás:

```
botao.addEventListener("click", function () {
  console.log("Cliquei!"); // callback: roda QUANDO o clique acontecer
});
```

O `addEventListener` é assíncrono por natureza: ele não sabe *quando* o clique virá, então você entrega um callback para rodar no momento certo.

12.4 O “callback hell”

Callbacks funcionam, mas têm um problema sério quando as tarefas **dependem umas das outras** em sequência. Imagine: buscar um usuário, depois buscar os pedidos dele, depois os detalhes de cada pedido, depois... Cada passo depende do anterior, e cada um é assíncrono. Com callbacks, isso vira uma pirâmide aninhada:

```
buscarUsuario(id, function (usuario) {
  buscarPedidos(usuario, function (pedidos) {
    buscarDetalhes(pedidos, function (detalhes) {
      calcularTotal(detalhes, function (total) {
        console.log(total);
        // ...e assim vai, cada vez mais fundo →
      });
    });
  });
});
```

Repare no formato de “escada” que desliza para a direita. Esse antipadrão é carinhosamente chamado de “**callback hell**” (o inferno dos callbacks) ou “pirâmide da perdição”. Os problemas:

1. **Difícil de ler** — a lógica se aninha em vez de fluir de cima para baixo.
2. **Difícil de tratar erros** — cada nível precisaria da sua própria checagem de erro, multiplicando a bagunça.
3. **Difícil de manter** — reordenar ou inserir um passo no meio é um pesadelo.

 **Nos bastidores** Os callbacks foram, por muitos anos, a única forma de lidar com a sequência em que as coisas assíncronas aconteciam. Eram uma boa solução para o seu tempo, mas o “callback hell” incomodava tanto que a comunidade criou algo melhor. Essa “coisa melhor” são as **Promises**, o assunto do próximo capítulo — e a razão pela qual você raramente escreverá pirâmides como essa hoje em dia.

12.5 O caminho que vamos percorrer

Para consumir APIs de forma elegante, a linguagem evoluiu em três estágios. Vamos aprender os três, em ordem, porque cada um esclarece o próximo:

Estágio	Ideia	Capítulo
Callbacks	“quando terminar, chame esta função”	12 (aqui)
Promises	um objeto que representa um resultado futuro	13
async/await	escrever código assíncrono com cara de síncrono	14

Todos os três coexistem no JavaScript moderno, e você vai encontrá-los em código alheio. Mas, no App Livros, o nosso destino é o **async/await** — a forma mais limpa e legível. Os callbacks e as Promises são a fundação que torna o async/await compreensível.

 **Montando o quebra-cabeça** No projeto, o assincronismo aparece em quatro momentos: buscar o endereço pelo CEP (Cap. 15), listar livros (Cap. 16), listar personagens do Rick and Morty (Cap. 17) e, em todos eles, mostrar um indicador de “Carregando...”. Cada uma dessas telas espera dados da internet **sem travar** a interface. É o assincronismo trabalhando por nós.

Recapitulando

- **Síncrono** = uma tarefa por vez, em ordem, esperando cada uma terminar.
- **Assíncrono** = tarefas demoradas são disparadas sem bloquear o resto do código.
- Toda comunicação com servidores em JavaScript é **assíncrona**, para não travar a página.
- **Callbacks** são a forma clássica de tratar o “quando terminar”: passamos uma função para ser executada ao fim da tarefa.
- Sequências de callbacks aninhados geram o “**callback hell**” — difícil de ler, tratar erros e manter.
- A evolução callbacks → **Promises** → **async/await** resolve isso; vamos percorrê-la nos próximos capítulos.

Exercícios do Capítulo 12

1. Preveja a saída deste código **antes** de rodar, depois confirme no Console:

```
console.log("A");
setTimeout(() => console.log("B"), 0);
console.log("C");
```

(Sim, mesmo com `0` de tempo o “B” sai por último. Pesquise “event loop” para descobrir por quê.)

2. Escreva uma função `dobrar(numero, callback)` que calcula o dobro e entrega o resultado ao callback. Chame-a passando uma função que imprime o resultado.
3. Explique, com suas palavras, por que uma requisição a um servidor **precisa** ser assíncrona.
4. Descreva um cenário do dia a dia (fora da programação) que seja “assíncrono” — algo que você inicia e continua sua vida sem esperar terminar.
5. **Reflexão:** liste os três problemas do “callback hell” e explique por que cada um dificulta a manutenção do código.

Capítulo 13 — Promises e o `fetch`

No capítulo anterior vimos o problema do “callback hell”. A solução que a linguagem criou é a **Promise** — um objeto que representa um resultado que **ainda vai chegar**. Neste capítulo entendemos as Promises e, com elas, fazemos nossa **primeira requisição de verdade a um servidor** usando o `fetch`.

É aqui que dados da internet começam a entrar na aplicação.

13.1 O que é uma Promise

Uma **Promise** (“promessa”) é um objeto que representa a eventual conclusão (ou falha) de uma operação assíncrona e seu valor resultante.

A analogia perfeita é a de um restaurante: quando você faz o pedido, o garçom te dá uma **comanda**. A comanda **não é a comida** — é uma *promessa* de que a comida vai chegar. Com ela na mão, você pode continuar sua conversa (o programa segue). Quando a comida fica pronta, a promessa é **cumprida**. Se acabou o ingrediente, a promessa é **rejeitada**.

Uma Promise está sempre em um de **três estados**:

- **Pendente** (*pending*): estado inicial; a operação ainda não terminou.
- **Resolvida** (*fulfilled*): a operação foi concluída **com sucesso**, e há um valor.
- **Rejeitada** (*rejected*): a operação **falhou**, e há um motivo (o erro).

Uma Promise começa pendente e, em algum momento, vira resolvida **ou** rejeitada — e aí não muda mais.

13.2 Consumindo uma Promise: `.then()` e `.catch()`

Para reagir ao resultado de uma Promise, usamos dois métodos:

- `.then(callback)` — roda quando a Promise **resolve** (sucesso). Recebe o valor resultante.
- `.catch(callback)` — roda quando a Promise **rejeita** (erro). Recebe o motivo.

```
minhaPromise
  .then((valor) => {
    console.log("Deu certo! Valor:", valor);
  })
  .catch((erro) => {
    console.error("Deu errado:", erro);
  });
```

Repare que os callbacks continuam aqui — mas em vez de aninhados, eles ficam **encadeados** de cima para baixo. Já é mais legível que a pirâmide do callback hell.

13.3 O **fetch** : buscando dados da web

Agora o momento que você esperava. O **fetch** é a função embutida no navegador para fazer requisições HTTP — ou seja, **pedir dados a um servidor**. E ele **retorna uma Promise** (afinal, a resposta demora a chegar).

Vamos fazer uma requisição real a uma API pública brasileira, a **ViaCEP**, que devolve um endereço a partir de um CEP:

```
const fetchPromise = fetch("https://viacep.com.br/ws/01001000/json/");

console.log(fetchPromise); // Promise { <pending> } – ainda pendente!

fetchPromise
  .then((resposta) => {
    if (!resposta.ok) {
      throw new Error(`Erro HTTP: ${resposta.status}`);
    }
    return resposta.json(); // converte a resposta em objeto JS
  })
  .then((dados) => {
    console.log(dados.localidade); // "São Paulo"
  })
  .catch((erro) => {
    console.error(`Não foi possível obter os dados: ${erro}`);
  });

console.log("Requisição iniciada...");
```

Observe a saída no Console:

```
Promise { <pending> }  
Requisição iniciada...  
São Paulo ← chega por último, quando o servidor responde
```

“Requisição iniciada...” aparece **antes** de “São Paulo”, confirmando que o `fetch` não bloqueou o código — puro assincronismo do Capítulo 12.

Por que dois `.then()` ?

Uma sutileza importante do `fetch`:

1. O **primeiro** `.then()` recebe a **resposta** (`resposta`) — os cabeçalhos HTTP, o status, mas **ainda não o conteúdo**. Chamamos `resposta.json()` para extrair e converter o corpo (que também é assíncrono e devolve outra Promise).
2. O **segundo** `.then()` recebe finalmente os **dados** já convertidos em objeto JavaScript, prontos para usar.

 **Cuidado** — `fetch` **não rejeita em erro 404/500** Contraintuitivo: se o servidor responde com erro (404 “não encontrado”, 500 “erro interno”), o `fetch` **não** rejeita a Promise — ele considera que “recebeu uma resposta”, ainda que de erro. Por isso checamos manualmente `if (!resposta.ok)` e lançamos um erro com `throw`. O `.catch()` só pega falhas de **rede** (sem internet, servidor fora do ar) — a menos que você jogue o erro você mesmo, como fizemos. Guarde essa checagem: ela vai reaparecer no nosso serviço de API.

 **Nos bastidores** — **o que é JSON?** **JSON** (JavaScript Object Notation) é o formato de texto universal para troca de dados na web. Ele se parece muito com um objeto JavaScript: `{ "cep": "01001-000", "localidade": "São Paulo" }`. As APIs devolvem texto em JSON, e o `resposta.json()` converte esse texto num objeto JS de verdade, que podemos acessar com `dados.localidade` — exatamente como qualquer objeto do Capítulo 2.

13.4 Criando a própria Promise (para entender por dentro)

Você raramente vai criar Promises do zero — o `fetch` já entrega uma pronta. Mas criar uma manualmente ajuda a entender a mecânica. Uma Promise recebe uma função com dois “gatilhos”: `resolve` (para sucesso) e `reject` (para falha):


```
const promessa = new Promise((resolve, reject) => {
  setTimeout(() => {
    const deuCerto = true;
    if (deuCerto) {
      resolve("Operação concluída com sucesso!"); // → dispara o .then()
    } else {
      reject("Falha na operação."); // → dispara o .catch()
    }
  }, 1000);
});

promessa
  .then((mensagem) => console.log("Sucesso: " + mensagem))
  .catch((erro) => console.error("Erro: " + erro));
```

Aqui simulamos uma operação que demora 1 segundo (o `setTimeout`) e depois “resolve” com sucesso. Trocando `deuCerto` para `false`, você veria o `.catch()` disparar. É exatamente esse mecanismo que o `fetch` usa internamente.

13.5 O encadeamento resolve o callback hell

Lembra da pirâmide do capítulo anterior? Com Promises, aquelas operações dependentes viram uma **corrente vertical**, muito mais legível:

```
// em vez da pirâmide aninhada do callback hell...
buscarUsuario(id)
  .then((usuario) => buscarPedidos(usuario))
  .then((pedidos) => buscarDetalhes(pedidos))
  .then((detalhes) => calcularTotal(detalhes))
  .then((total) => console.log(total))
  .catch((erro) => console.error(erro)); // UM catch trata qualquer falha
```

Duas grandes vitórias sobre os callbacks:

1. A lógica flui **de cima para baixo**, não para a direita.
2. Um único `.catch()` no fim trata erros de **qualquer** etapa da corrente. Chega de tratar erro em cada nível.

Já é ótimo. Mas a linguagem foi além e criou uma forma de escrever isso que parece código síncrono comum, sem nem os `.then()`. É o **async/await**, nosso próximo (e definitivo) passo.

🧩 **Montando o quebra-cabeça** A checagem `if (!resposta.ok) throw new Error(...)` que vimos aqui será o coração da função genérica de requisição do App Livros — o arquivo `services/api.js`. Toda chamada de API do projeto passará por ela. E o `.then()` / `.catch()` deste capítulo será reescrito com `async/await` no próximo. Você está a um capítulo de buscar dados como um profissional.

Recapitulando

- Uma **Promise** representa um resultado futuro; vive em três estados: **pendente**, **resolvida** ou **rejeitada**.
- Consumimos Promises com `.then()` (sucesso) e `.catch()` (erro).
- O `fetch(url)` faz requisições HTTP e **retorna uma Promise**.
- Com `fetch`, o primeiro `.then()` recebe a **resposta** e chama `resposta.json()`; o segundo recebe os **dados**.
- `fetch` **não rejeita** em erros HTTP (404/500) — cheque `resposta.ok` manualmente.
- O encadeamento de `.then()` resolve o callback hell: fluxo vertical e um só `.catch()`.

Exercícios do Capítulo 13

1. No Console, faça um `fetch` para `https://viacep.com.br/ws/01001000/json/`, encadeie os `.then()` e imprima a `localidade` e o `logradouro`.
2. Troque o CEP por um inválido (ex.: `00000000`) e observe o que a API devolve. Depois force um erro de rede (desligue a internet) e veja o `.catch()` disparar.
3. Crie uma Promise manual que resolve com `"Pronto!"` após 2 segundos usando `setTimeout`, e imprima a mensagem no `.then()`.
4. Modifique a Promise do exercício 3 para **rejeitar** metade das vezes (dica: `Math.random() > 0.5`) e trate o erro no `.catch()`.
5. Explique, com suas palavras, por que o `fetch` precisa de **dois** `.then()` para chegar aos dados.

Capítulo 14 — `async` / `await` e a camada de serviços

As Promises com `.then()` já são um enorme avanço sobre os callbacks. Mas o JavaScript moderno oferece uma sintaxe ainda mais limpa: o `async` / `await`. Com ela, código assíncrono passa a **parecer** código síncrono comum — fácil de ler de cima para baixo. Neste capítulo dominamos essa sintaxe e a usamos para construir a **camada de serviços** do App Livros: o arquivo `api.js`, que centraliza toda a comunicação com a internet.

14.1 As palavras-chave `async` e `await`

Duas palavras trabalham juntas:

- `async` vai antes de uma função. Ela marca a função como assíncrona — e uma função `async` **sempre retorna uma Promise** automaticamente.
- `await` vai antes de uma Promise, **dentro** de uma função `async`. Ela **pausa** a função até a Promise resolver, e devolve o valor resolvido. É como dizer “espere aqui até isso ficar pronto, e me dê o resultado”.

 **Cuidado** O `await` só pode ser usado dentro de uma função `async`. Usá-lo fora dá erro de sintaxe. Regra de ouro: viu `await`, a função ao redor precisa ser `async`.

14.2 O mesmo `fetch`, agora legível

Vamos reescrever a requisição do capítulo anterior — primeiro com `.then()`, depois com `async/await` — para você ver a diferença lado a lado.

Com `.then()` (Capítulo 13):

```
function buscarDados() {
  fetch("https://viacep.com.br/ws/01001000/json/")
    .then((resposta) => resposta.json())
    .then((dados) => console.log(dados))
    .catch((erro) => console.error(erro));
}
```

Com `async/await` (agora):

```

async function buscarDados() {
  try {
    const resposta = await fetch("https://viacep.com.br/ws/01001000/json/");
    const dados = await resposta.json();
    console.log(dados);
  } catch (erro) {
    console.error(erro);
  }
}

```

Compare. A versão `async/await`:

- Lê-se de **cima para baixo**, como código comum. `await fetch(...)` “espera” a resposta e a guarda em `resposta`; `await resposta.json()` espera a conversão e guarda em `dados`. Nada de encadear `.then()`.
- Usa `try/catch` para tratar erros. O `try` envolve o código que pode falhar; o `catch` pega qualquer erro que aconteça ali dentro. É o mesmo papel do `.catch()`, mas com uma cara mais familiar e única para todos os `await`.

 **Nos bastidores** Por baixo dos panos, `async/await` é Promises — é apenas uma forma mais agradável de escrevê-las (o que chamamos de “açúcar sintático”). O `await` continua fazendo o trabalho de esperar a Promise resolver; ele só esconde o `.then()` de você. Por isso foi importante entender Promises antes: o `async/await` não é mágica, é conveniência.

14.3 O padrão `try/catch/finally`

O tratamento de erros com `async/await` tem três blocos possíveis:

```

try {
  // código que pode dar errado (as operações com await)
} catch (erro) {
  // roda SE algo no try falhar
} finally {
  // roda SEMPRE, dando certo ou errado
}

```

O `finally` é especialmente útil para “limpeza” — algo que deve acontecer independentemente do resultado. O exemplo perfeito, que usaremos muito: esconder o indicador de “Carregando...”.

```

async function carregarAlgo() {
  mostrarCarregando();          // liga o "Carregando..."
  try {
    const dados = await buscarDados(url);
    // usa os dados...
  } catch (erro) {
    console.error(erro);        // trata a falha
  } finally {
    esconderCarregando();       // desliga o "Carregando...", SEMPRE
  }
}

```

Se colocássemos `esconderCarregando()` só no `try`, ele não rodaria em caso de erro, e o “Carregando...” ficaria eternamente girando na tela. O `finally` garante que ele sempre suma. Esse padrão — **mostrar carregando** → **try** → **catch** → **finally esconde carregando** — se repete em **todas** as telas de API do App Livros. Guarde-o bem.

14.4 Construindo a camada de serviços: `api.js`

Chegou a hora de aplicar tudo isso ao projeto de forma profissional. Em vez de espalhar `fetch` por todas as páginas, vamos **centralizar** a comunicação com a internet numa “camada de serviços”. O primeiro e mais importante arquivo é uma função genérica de requisição.

Crie a pasta `src/js/services/` e, dentro, o arquivo `api.js`:

```

// src/js/services/api.js
async function buscarDados(url) {
  const resposta = await fetch(url);

  if (!resposta.ok) {
    throw new Error(`Erro ${resposta.status} ao chamar ${url}`);
  }

  return resposta.json();
}

export default buscarDados;

```

Pequena, mas poderosa. Vamos entendê-la:

- É uma função `async` que recebe uma `url` qualquer como parâmetro (Cap. 3). Genérica: serve para **qualquer** API.
- `await fetch(url)` dispara a requisição e espera a resposta.

- A checagem `if (!resposta.ok)` (do Capítulo 13!) lança um erro se o servidor respondeu com falha (404, 500...). Assim, quem chamar essa função receberá o erro no seu próprio `catch`.
- `return resposta.json()` devolve os dados já convertidos em objeto. Como a função é `async`, esse retorno vira uma Promise que o chamador vai `await`.

🧩 **Montando o quebra-cabeça** Este arquivinho é a **espinha dorsal** de todo o consumo de dados do App Livros. **Toda** requisição — CEP, livros, personagens — passa por ele. Isso é o princípio DRY (Cap. 9) aplicado ao acesso a dados: a lógica de “fazer a requisição e checar erro” existe em **um único lugar**. Se um dia precisarmos mudar como as requisições funcionam (adicionar um cabeçalho, um log, um token), mudamos aqui e todo o projeto se beneficia.

14.5 O indicador de carregamento: `loading.js`

Já que toda tela de API vai mostrar um “Carregando...”, vale criar um pequeno serviço para isso também. O App Livros usa a biblioteca **SweetAlert2** (um popup bonito). Crie `src/js/services/loading.js`:

```
// src/js/services/loading.js
function mostrarCarregando() {
  Swal.fire({
    didOpen: () => Swal.showLoading()
  });
}

function esconderCarregando() {
  Swal.close();
}

export { mostrarCarregando, esconderCarregando };
```

- `mostrarCarregando()` abre o popup com uma animação de carregamento.
- `esconderCarregando()` fecha o popup.
- Exportamos as **duas** funções com **exportação nomeada** (Cap. 8), pois o arquivo oferece mais de uma coisa.

Para o `Swal` existir, incluímos a biblioteca no `index.html` (via CDN), no `<head>`:

```
<script src="https://cdn.jsdelivr.net/npm/sweetalert2@11"></script>
```

💡 **Nos bastidores — por que uma camada de serviços?** Repare no padrão de arquitetura que está nascendo. Vamos ter duas famílias de arquivos:

- **components/paginas/** — cuidam da **interface** (o que o usuário vê).
- **services/** — cuidam dos **dados** (conversa com a internet, formatação).

Essa separação — “quem desenha” versus “quem busca dados” — é uma das ideias mais valiosas do curso. Ela mantém cada arquivo focado, testável e fácil de mudar. É o mesmo princípio que sustenta aplicações profissionais gigantes.

14.6 Antevendo o uso

Nos próximos três capítulos, vamos escrever serviços específicos (para CEP, para livros, para personagens) que **usam** o **buscarDados** genérico, e páginas que **usam** esses serviços. A cadeia de dependências ficará assim:

```
página (livros.js)
  usa → serviço específico (services/livros.js)
        usa → função genérica (services/api.js)
              usa → fetch (navegador)
                    usa → a API na internet
```

Cada camada tem uma responsabilidade única e conversa apenas com a vizinha. É organização em estado puro. E tudo é sustentado pelo **async/await** que você acabou de dominar.

🧩 **Montando o quebra-cabeça** Um exemplo concreto do que vem por aí (Cap. 16). Repare como o **await** e o **buscarDados** se combinam naturalmente:

```
async function buscarListaLivros(pagina) {
  const dados = await buscarDados(`${BASE_URL}/?search=fiction&page=${pagina}`);
  // ...transforma 'dados' no formato que a tela precisa...
}
```

Você já entende cada pedaço dessa linha. Falta só juntá-los numa tela completa.

Recapitulando

- `async` marca uma função assíncrona (que sempre retorna uma Promise); `await` pausa até uma Promise resolver e devolve o valor.
 - `await` só funciona **dentro** de funções `async`.
 - `async/await` deixa o código assíncrono com cara de síncrono, lido de cima para baixo — é “açúcar sintático” sobre Promises.
 - Trate erros com `try/catch`; use `finally` para limpeza que deve rodar sempre (como esconder o “Carregando...”).
 - A função genérica `api.js` (`buscarDados`) centraliza toda requisição do projeto, com a checagem `if (!resposta.ok)`.
 - A **camada de serviços** separa “quem busca dados” de “quem desenha a tela” — uma arquitetura limpa e escalável.
-

Exercícios do Capítulo 14

1. Reescreva a função `buscarDados` do Capítulo 13 (versão com `.then()`) usando `async/await` e `try/catch`.
2. Crie o arquivo `services/api.js` com a função genérica `buscarDados` e teste no Console importando-a e chamando com a URL da ViaCEP.
3. Adicione um `finally` a uma função `async` que imprime “Requisição finalizada” — e confirme que ele roda tanto no sucesso quanto no erro.
4. Explique por que centralizar as requisições em `api.js` é uma aplicação do princípio **DRY** (Cap. 9).
5. **Reflexão:** descreva, com suas palavras, a diferença de papéis entre a pasta `components/paginas/` e a pasta `services/`.

Capítulo 15 — Formulário de cadastro com a API ViaCEP

Hora de colher os frutos. Com a camada de serviços pronta, vamos construir a primeira tela que **conversa com a internet**: um formulário de cadastro que, ao digitar o CEP, **busca e preenche o endereço automaticamente**. É um recurso que você vê em lojas online o tempo todo — e vai implementá-lo do zero.

Esta tela junta praticamente tudo: DOM, eventos, `async/await`, a camada de serviços e o indicador de carregamento.

15.1 O que vamos construir

A ideia é simples e encantadora: o usuário digita o CEP e **sai do campo**. Nesse instante, disparamos uma consulta à API **ViaCEP** e preenchemos sozinhos os campos de logradouro, bairro, cidade e estado.

O evento que detecta “o usuário saiu do campo” é o `blur` (Cap. 5). É o gatilho perfeito: não precisamos de um botão “buscar” — a busca acontece naturalmente quando o usuário termina de digitar o CEP e vai para o próximo campo.

 **Nos bastidores** — a API **ViaCEP** A **ViaCEP** é uma API pública e gratuita que devolve endereços brasileiros a partir do CEP. A URL segue o padrão:

```
https://viacep.com.br/ws/01001000/json/
```

E a resposta é um JSON assim:

```
{
  "cep": "01001-000",
  "logradouro": "Praça da Sé",
  "bairro": "Sé",
  "localidade": "São Paulo",
  "estado": "São Paulo"
}
```

Repare: é um **objeto** (Cap. 2), e vamos acessar `dados.logradouro`, `dados.bairro`, etc. Nada de novo — apenas dados vindos de longe.

15.2 Montando o HTML do formulário

Como toda página do App Livros, começamos com um módulo (Cap. 8) cuja função monta o HTML. Crie `src/js/components/paginas/formCad.js` e comece pela estrutura visual:

```
// src/js/components/paginas/formCad.js
async function telaCadastro(app) {
  const formulario = `
    <form id="cadastroCliente" class="bem-form bem-grid-auto">
      <div class="bem-form__group">
        <label for="cep" class="bem-form__label">CEP</label>
        <input type="text" id="cep" class="bem-form__input">
      </div>
      <div class="bem-form__group">
        <label for="logradouro" class="bem-form__label">Logradouro</label>
        <input type="text" id="logradouro" class="bem-form__input">
      </div>
      <div class="bem-form__group">
        <label for="bairro" class="bem-form__label">Bairro</label>
        <input type="text" id="bairro" class="bem-form__input">
      </div>
      <div class="bem-form__group">
        <label for="localidade" class="bem-form__label">Localidade</label>
        <input type="text" id="localidade" class="bem-form__input">
      </div>
      <div class="bem-form__group">
        <label for="estado" class="bem-form__label">Estado</label>
        <input type="text" id="estado" class="bem-form__input">
      </div>
    </form>
  `;

  app.innerHTML = formulario;
  await capturarCep();
}
```

Nada de novo na estrutura: uma template string com o formulário, injetada no `app`, seguida da chamada a `capturarCep()` — que vai registrar o evento. Note que cada `<input>` tem um `id` claro (`cep`, `logradouro`, `bairro` ...) — vamos usar esses ids para preencher os campos com os dados que chegarem.

Assim como no formulário de contato (Cap. 8), primeiro **desenhamos** o HTML e só **depois** registramos os eventos. O campo precisa existir na tela antes de podermos escutá-lo.

15.3 O coração: capturando o CEP no `blur`

Agora a função que faz a mágica. No topo do arquivo, importamos os serviços que construímos no capítulo anterior:

```
import buscarDados from "../../services/api.js";
import { mostrarCarregando, esconderCarregando } from "../../services/loading.js";
```

Repare nos caminhos: `../../services/` significa “suba duas pastas e entre em services” — de `components/paginas/` chegamos à raiz de `js/` e daí em `services/`. E note as duas formas de import (Cap. 8): `buscarDados` é **default** (sem chaves); as funções de loading são **nomeadas** (com chaves).

Agora a função `capturarCep`:

```
async function capturarCep() {
  const campoCep = document.getElementById("cep");

  campoCep.addEventListener("blur", async (event) => {
    mostrarCarregando();
    try {
      const dados = await buscarDados(
        `https://viacep.com.br/ws/${event.target.value}/json/`
      );
      document.getElementById("logradouro").value = dados.logradouro;
      document.getElementById("bairro").value = dados.bairro;
      document.getElementById("localidade").value = dados.localidade;
      document.getElementById("estado").value = dados.estado;
    } catch (erro) {
      console.error(erro);
    } finally {
      esconderCarregando();
    }
  });
}
```

Vamos ler com atenção, porque este trecho é a síntese de todo o curso:

1. `getElementById("cep")` (Cap. 5) — pega o campo de CEP.
2. `addEventListener("blur", ...)` (Cap. 5) — escuta o momento em que o usuário **sai** do campo. O callback é uma **arrow function** `async` (Caps. 3 e 14) — precisa ser `async` porque usa `await` lá dentro.
3. `mostrarCarregando()` (Cap. 14) — liga o indicador de carregamento antes da busca.

4. `event.target.value` (Cap. 5) — `event.target` é o campo de CEP, e `.value` é o que o usuário digitou. Interpolamos isso na URL com uma **template string** (Cap. 1).
5. `await buscarDados(...)` (Cap. 14) — chama nosso serviço genérico, que faz o `fetch`, checa erros e devolve os dados já como objeto.
6. **Preenchimento dos campos** — para cada campo do endereço, escrevemos no `.value` o valor correspondente do objeto `dados` (`dados.logradouro`, `dados.bairro` ...). Escrever em `.value` **preenche** o campo na tela.
7. `catch` (Cap. 14) — se algo falhar (CEP inválido, sem internet), registramos o erro no console.
8. `finally` (Cap. 14) — o carregamento é escondido **sempre**, dando certo ou errado.

Por fim, exportamos o módulo no formato padrão de página:

```
export default {  
  url: "#cep",  
  label: "Cadastro",  
  pagina: telaCadastro  
};
```

E, claro, adicionamos essa página ao array em `rotas.js` (Cap. 9):

```
import telaCadastro from "../paginas/formCad.js";  
  
const roteador = [  
  home,  
  telaCadastro, // ← nova página  
  sobre,  
  servicos,  
  contato,  
];
```

Pronto! Como aprendemos no Capítulo 9, isso é **tudo** o que precisamos: a página aparece no menu e funciona na navegação automaticamente. Teste: navegue até “Cadastro”, digite um CEP válido (ex.: `01001000`) e clique fora do campo. O endereço se preenche sozinho, com um “Carregando...” no meio do caminho.

15.4 O fluxo completo, passo a passo

Vamos rastrear o que acontece quando o usuário digita `01001000` e sai do campo:

```
1. Usuário sai do campo CEP → dispara o evento "blur"
   |
2. O callback async roda; mostrarCarregando() abre o popup
   |
3. await buscarDados("https://viacep.com.br/ws/01001000/json/")
   |      (buscarDados faz fetch, checa resposta.ok, converte JSON)
   |
4. O código PAUSA no await, sem travar a página, até o servidor responder
   |
5. Chega o objeto: { logradouro: "Praça da Sé", bairro: "Sé", ... }
   |
6. Preenchemos cada input com dados.logradouro, dados.bairro, etc.
   |
7. finally: esconderCarregando() fecha o popup
   |
8. A tela mostra o endereço preenchido.
```

Cada passo é um conceito que você já domina. Não há mágica — há engenharia bem organizada em camadas.

 **Cuidado — validação fica de lição de casa** Nossa versão é didática e não valida a entrada. Se o usuário digitar um CEP mal formatado ou inexistente, a ViaCEP devolve `{ "erro": true }`, e os campos seriam preenchidos com `undefined`. Numa aplicação real, você checaria `if (dados.erro) { ... }` e avisaria o usuário. Fica como um ótimo exercício de aprimoramento (veja os exercícios).

15.5 Por que este capítulo é um marco

Pare e reconheça o tamanho do que você acabou de fazer. Sua aplicação agora:

- **Conversa com um servidor real** na internet;
- Faz isso **sem travar** a interface (assincronismo);
- **Reage a uma ação natural** do usuário (sair do campo);
- Dá **feedback visual** durante a espera (carregando);
- **Trata erros** com elegância (try/catch/finally);
- Reutiliza a **camada de serviços** (api.js) e se encaixa no **roteador** sem esforço.

Isso é uma funcionalidade de nível profissional, construída inteiramente com os fundamentos que acumulamos capítulo a capítulo. Nos próximos dois capítulos, vamos escalar essa mesma técnica para telas com **listas paginadas** e **detalhes em modal**.

Recapitulando

- O evento `blur` (sair de um campo) é o gatilho ideal para a busca de CEP — dispensa botão.
 - A função de busca é uma **arrow function** `async`, porque usa `await` lá dentro.
 - Lemos o valor digitado com `event.target.value` e o interpolamos na URL.
 - Reutilizamos `buscarDados` (api.js) e o par `mostrarCarregando` / `esconderCarregando` (loading.js).
 - Preenchemos os campos escrevendo no `.value` de cada input.
 - O padrão `carregando` → `try` → `catch` → `finally` `esconde` aparece de novo — e vai se repetir.
-

Exercícios do Capítulo 15

1. Construa a tela de cadastro completa e teste com pelo menos três CEPs reais. Confirme o preenchimento automático e o “Carregando...”.
2. **Validação:** adicione uma checagem `if (dados.erro) { ... }` que, em caso de CEP inexistente, limpa os campos e mostra um alerta (`alert` ou `Swal.fire`) avisando “CEP não encontrado”.
3. Adicione um campo `número` e um `complemento` ao formulário (esses a ViaCEP não preenche — ficam para o usuário digitar).
4. Troque o evento `blur` por `change` e observe se há diferença de comportamento. Depois volte para `blur`.
5. **Reflexão:** por que o callback do `blur` precisa ser `async`? O que aconteceria se removêssemos o `async` mas mantivéssemos o `await`?

Capítulo 16 — Consumindo a API de Livros (lista, paginação e modal)

Este é o capítulo mais ambicioso do curso — e o que dá nome ao projeto. Vamos construir a tela de **Livros**: uma grade de cards vinda de uma API real, com **paginação** (botões Anterior/Próxima) e uma **janela de detalhes (modal)** que abre ao clicar num livro. É a tela mais completa do App Livros, e nela tudo o que aprendemos se encontra.

Vamos construí-la em duas camadas, como manda a boa arquitetura (Cap. 14): primeiro o **serviço** (que busca e organiza os dados), depois a **página** (que os exibe).

16.1 A API Gutendex

Usaremos a **Gutendex**, uma API gratuita que expõe o catálogo do Projeto Gutenberg (milhares de livros de domínio público). Dois endereços nos interessam:

- **Lista de livros** (com busca e paginação):

```
https://gutendex.com/books/?search=fiction&page=2
```

Repare na **query string** (Cap. 6): `search=fiction` filtra por “fiction” e `page=2` pede a segunda página. A resposta traz um objeto com `count` (total de livros), `results` (o array de livros desta página) e mais.

- **Detalhe de um livro** (pelo id):

```
https://gutendex.com/books/1342/
```

Cada livro no `results` é um objeto rico, com campos como `id`, `title`, `authors` (um array de objetos!), `formats` (um objeto com URLs de imagem) e `subjects` (um array de assuntos). Nosso serviço vai **peneirar** essa complexidade e entregar à tela apenas o que ela precisa.

16.2 A camada de serviço: `services/livros.js`

O princípio é o mesmo do capítulo anterior: a página **não** deve saber os detalhes da API. Ela pede “me dê a lista de livros da página 2” e recebe dados limpos. Quem lida com a bagunça da API é o serviço.

Crie `src/js/services/livros.js`. Começamos com constantes e funções auxiliares:

```
// src/js/services/livros.js
import buscarDados from "../api.js";

const BASE_URL = "https://gutendex.com/books";
const TERMO_PADRAO = "fiction";
const ITENS_POR_PAGINA = 32;

function extrairAutores(autores) {
  const nomes = [];
  for (const autor of autores) {
    nomes.push(autor.name);
  }
  return nomes.length ? nomes.join(", ") : "Autor desconhecido";
}

function extrairImagem(formatos) {
  return formatos["image/jpeg"] || "";
}
```

Duas funções auxiliares (Cap. 3) que “traduzem” os dados crus da API:

- **extrairAutores** recebe o array de autores (cada um um objeto com `.name`) e devolve um **texto** com os nomes separados por vírgula. Usa um `for...of` (Cap. 4) para coletar os nomes num array e `.join(", ")` (Cap. 2) para juntá-los. Se não houver autores (`nomes.length` é 0, que é “falso”), devolve “Autor desconhecido” — um ternário disfarçado com `?`.
- **extrairImagem** cava no objeto `formats` para pegar a URL da imagem JPEG. O `|| ""` garante um texto vazio caso não exista imagem (evita `undefined` na tela).

Agora a função que busca a **lista**:


```
async function buscarListaLivros(pagina) {
  const dados = await buscarDados(`${BASE_URL}?search=${TERMO_PADRAO}&page=${pagina}`);

  const livros = [];
  for (const item of dados.results) {
    livros.push({
      id: item.id,
      titulo: item.title,
      autores: extrairAutores(item.authors),
      imagem: extrairImagem(item.formats)
    });
  }

  const totalPaginas = Math.ceil(dados.count / ITENS_POR_PAGINA);
  return { totalPaginas, livros };
}
```

Aqui está o **coração da transformação de dados**:

1. `await buscarDados(...)` (Cap. 14) — pede a página à API usando nossa função genérica. Montamos a URL com a query string por template string.
2. Percorremos `dados.results` (o array cru da API) com `for...of` e, para cada `item`, montamos um **objeto limpo** só com o que a tela usa: `id`, `titulo`, `autores` (já como texto), `imagem` (já como URL). Isso é o padrão “array de objetos” (Cap. 2) sendo **construído** a partir de outro.
3. `Math.ceil(dados.count / ITENS_POR_PAGINA)` calcula o total de páginas: o total de livros dividido por 32, arredondado para cima (`Math.ceil`).
4. `return { totalPaginas, livros }` devolve um objeto com as duas informações que a tela precisa.

E a função que busca o **detalhe** de um livro:

```

async function buscarDetalheLivro(id) {
  const dados = await buscarDados(`${BASE_URL}/${id}/`);

  return {
    titulo: dados.title,
    autores: extrairAutores(dados.authors),
    imagem: extrairImagem(dados.formats),
    descricao: dados.summaries.length ? dados.summaries[0] : "Sem descrição disponível",
    assuntos: dados.subjects.length ? dados.subjects.slice(0, 5).join(", ") : "Não há assuntos",
  };
}

export { buscarListaLivros, buscarDetalheLivro };

```

Repare no `dados.subjects.slice(0, 5).join(", ")` — exatamente o `slice` que prometemos no Capítulo 2! Pegamos só os **cinco primeiros** assuntos e os juntamos num texto. E, de novo, ternários (`? :`) garantem valores padrão quando os campos vêm vazios. Por fim, **exportação nomeada** (Cap. 8) das duas funções.

💡 **Nos bastidores — por que “traduzir” os dados?** Poderíamos usar `item.title`, `item.authors[0].name` etc. direto na página. Mas aí a página ficaria **acoplada** ao formato da Gutendex. Se um dia trocássemos de API, ou a API mudasse seus campos, teríamos que caçar mudanças por toda a tela. Com a camada de serviço “traduzindo” para o **nosso** formato (`titulo`, `autores`, `imagem`), a página nunca precisa saber de onde os dados vêm. Essa independência é ouro em projetos reais.

16.3 A página: estrutura e o modal

Agora a interface. Crie `src/js/components/paginas/livros.js`. Ela é maior, então vamos por partes. Primeiro os imports e uma variável de estado:

```

// src/js/components/paginas/livros.js
import { buscarListaLivros, buscarDetalheLivro } from "../../services/livros.js";
import { mostrarCarregando, esconderCarregando } from "../../services/loading.js";

let paginaAtual = 1;

```

A variável `paginaAtual` guarda em que página da listagem estamos. Ela vive **fora** das funções (no escopo do módulo) para que todas possam lê-la e alterá-la — é o “estado” da nossa tela.

A função principal monta a estrutura (incluindo o modal, que fica escondido) e registra os eventos:

```
async function telaLivros(app) {
  paginaAtual = 1;
  app.innerHTML = `
    <div id="modalDetalhes" class="bem-modal bem-modal--hidden">
      <div class="bem-modal__dialog">
        <div class="bem-modal__header">
          <h2 class="bem-modal__title" id="detalheTitulo"></h2>
          <button id="fecharDetalhes" class="bem-modal__close">&times;</button>
        </div>
        <div class="bem-modal__body" id="detalheCorpo"></div>
      </div>
    </div>

    <div class="bem-container">
      <h1 class="bem-mb-md">Livros</h1>
      <div id="listaLivros" class="bem-grid-auto"></div>
      <div class="bem-flex bem-justify-center bem-items-center bem-gap-md bem-mt-lg">
        <button id="paginaAnterior" class="bem-btn bem-btn--outline">Anterior</button>
        <span id="indicadorPagina"></span>
        <button id="proximaPagina" class="bem-btn bem-btn--outline">Próxima</button>
      </div>
    </div>
  `;

  document.getElementById("fecharDetalhes").addEventListener("click", fecharDetalhes)
  document.getElementById("paginaAnterior").addEventListener("click", () => mudarPagina(-1))
  document.getElementById("proximaPagina").addEventListener("click", () => mudarPagina(1))

  await renderizarPagina();
}
```

Pontos-chave:

- O **modal** (`#modalDetalhes`) já está no HTML, mas nasce com a classe `bem-modal--hidden`, que o esconde (Cap. 11). Vamos mostrá-lo/escondê-lo com `classList` (Cap. 5).
- `#listaLivros` é o contêiner da grade (`bem-grid-auto`, Cap. 11) onde os cards entrarão.
- Os botões **Anterior/Próxima** e o `#indicadorPagina` formam a paginação.
- Registramos os eventos: fechar o modal, e os cliques de paginação chamando `mudarPagina(-1)` e `mudarPagina(1)` — arrow functions como callbacks (Cap. 3).
- Ao final, `await renderizarPagina()` busca e desenha a primeira página.

16.4 Renderizando a lista

A função que busca os livros da página atual e monta os cards:

```
async function renderizarPagina() {
  mostrarCarregando();
  try {
    const { totalPaginas, livros } = await buscarListaLivros(paginaAtual);

    let html = "";
    for (const livro of livros) {
      html += `
        <div class="bem-card" data-id="${livro.id}">
          
          <div class="bem-card__body">
            <h3 class="bem-card__title">${livro.titulo}</h3>
            <p class="bem-card__subtitle">${livro.autores}</p>
          </div>
        </div>
      `;
    }
    document.getElementById("listaLivros").innerHTML = html;

    document.getElementById("indicadorPagina").textContent = `Página ${paginaAtual} de ${totalPaginas}`;
    document.getElementById("paginaAnterior").disabled = paginaAtual <= 1;
    document.getElementById("proximaPagina").disabled = paginaAtual >= totalPaginas;

    for (const card of document.querySelectorAll("#listaLivros .bem-card")) {
      card.addEventListener("click", () => abrirDetalhes(card.dataset.id));
    }
  } catch (erro) {
    console.error(erro);
  } finally {
    esconderCarregando();
  }
}
```

Reconheça os padrões — todos já são seus:

- O padrão **carregando** → **try** → **catch** → **finally** (Cap. 14), idêntico ao da tela de CEP.
- **Desestruturação**: `const { totalPaginas, livros } = await ...` extrai as duas propriedades do objeto retornado pelo serviço, de uma vez. (É um atalho para `const resultado = ...; const livros = resultado.livros; .)`
- O padrão de renderização `let html = ""` → `for...of` com `html += ...` → `innerHTML` (Cap. 5), montando um card por livro.

- `data-id="${livro.id}"` — guardamos o id do livro num “atributo de dados” no próprio card. Vamos lê-lo depois, no clique, com `card.dataset.id`. É a forma elegante de vincular um dado a um elemento do DOM.
- `.disabled = paginaAtual <= 1` — o botão “Anterior” é desabilitado na primeira página; o “Próxima”, na última. Um booleano (Cap. 1) controlando o estado do botão.
- `indicadorPagina.textContent` mostra “Página X de Y”.
- No fim, percorremos todos os cards com `querySelectorAll` (Cap. 5) e, em cada um, registramos o clique que abre os detalhes daquele livro (`card.dataset.id`).

⚠ Cuidado — registre os eventos DEPOIS de criar os cards Os cards só existem no DOM **depois** de `innerHTML = html`. Por isso o `addEventListener` em cada card vem **depois** dessa linha. Tentar adicionar o evento antes de o card existir simplesmente não funcionaria — não há o que escutar. Sempre: desenhe primeiro, escute depois.

16.5 Mudando de página

A paginação é surpreendentemente simples graças à nossa variável de estado:

```
async function mudarPagina(direcao) {
  paginaAtual += direcao;
  if (paginaAtual < 1) paginaAtual = 1;
  await renderizarPagina();
}
```

`direcao` é `+1` (Próxima) ou `-1` (Anterior). Somamos à `paginaAtual`, garantimos que ela não fique abaixo de 1, e mandamos **re-renderizar**. Como `renderizarPagina` sempre usa `paginaAtual` para buscar, a tela se atualiza para a nova página. Elegante: toda a lógica de paginação cabe em quatro linhas porque o **estado** está centralizado numa variável.

16.6 Abrindo e fechando o modal de detalhes

Ao clicar num card, buscamos os detalhes daquele livro e os mostramos no modal:

```

async function abrirDetalhes(id) {
  mostrarCarregando();
  try {
    const livro = await buscarDetalheLivro(id);

    document.getElementById("detalheTitulo").textContent = livro.titulo;
    document.getElementById("detalheCorpo").innerHTML = `
      <strong>Assuntos:</strong> ${livro.assuntos}</p>
      <p>${livro.descricao}</p>
    `;

    document.getElementById("modalDetalhes").classList.remove("bem-modal--hidden");
  } catch (erro) {
    console.error(erro);
  } finally {
    esconderCarregando();
  }
}

function fecharDetalhes() {
  document.getElementById("modalDetalhes").classList.add("bem-modal--hidden");
}

```

- `abrirDetalhes(id)` recebe o id (que veio do `data-id` do card), busca o detalhe pelo serviço, preenche o título e o corpo do modal, e — o pulo do gato — `classList.remove("bem-modal--hidden")` (Cap. 5 e 11) faz o modal **aparecer**.
- `fecharDetalhes()` faz o oposto: `classList.add("bem-modal--hidden")` esconde o modal de novo.

Repare que **não criamos nem destruimos** o modal — ele existe o tempo todo, apenas alternamos sua visibilidade com uma classe CSS. Simples e eficiente, exatamente como prometido no Capítulo 5.

Por fim, o export e a inclusão nas rotas:

```

export default {
  url: "#livros",
  label: "Livros",
  pagina: telaLivros
};

```

```
// em rotas.js
import livros from "../paginas/livros.js";
// ...adicione 'livros' ao array roteador
```

16.7 A tela inteira, em uma frase por função

Vale um resumo de alto nível de como as peças conversam:

Função	Responsabilidade
<code>telaLivros</code>	monta a estrutura + modal e liga os eventos
<code>renderizarPagina</code>	busca a página atual e desenha os cards + paginação
<code>mudarPagina</code>	ajusta <code>paginaAtual</code> e manda re-renderizar
<code>abrirDetalhes</code>	busca o detalhe de um livro e mostra o modal
<code>fecharDetalhes</code>	esconde o modal
<code>buscarListaLivros</code>	(<i>serviço</i>) traz e limpa a lista da API
<code>buscarDetalheLivro</code>	(<i>serviço</i>) traz e limpa o detalhe da API

Cada função faz **uma coisa**. Essa divisão é o que torna uma tela complexa compreensível. Se algo der errado na paginação, você sabe onde olhar (`mudarPagina`); se o modal não abre, o problema está em `abrirDetalhes` . Código organizado é código depurável.

Recapitulando

- Separe **serviço** (busca e limpa os dados) de **página** (exibe) — a página nunca precisa conhecer o formato cru da API.
- O serviço **transforma** o array cru da API num array de objetos limpos, usando funções auxiliares, `for...of` , `.join()` e `.slice()` .
- A página usa uma **variável de estado** (`paginaAtual`) para controlar a paginação.
- Renderização segue o padrão `let html = "" + for...of + innerHTML` ; vinculamos dados aos elementos com `data-id` / `dataset` .
- Botões são desabilitados com um **booleano** (`disabled = paginaAtual <= 1`).
- O **modal** existe sempre e é alternado com `classList add/remove("bem-modal-hidden")` .
- Sempre **desenhe primeiro, registre eventos depois**.

Exercícios do Capítulo 16

1. Construa as duas camadas (serviço + página) e confirme: a grade de livros carrega, a paginação funciona e o modal abre/fecha.
2. Troque o `TERMO_PADRAO` de `"fiction"` para outro tema (ex.: `"children"`) e veja a listagem mudar.
3. Faça o modal **também** fechar ao clicar fora do diálogo (na área escura de fundo). Dica: registre um clique no `#modalDetalhes` e cheque se `event.target` é o próprio fundo.
4. Adicione um tratamento visual para quando um livro **não tem imagem** (`livro.imagem === ""`): mostre uma imagem placeholder no lugar.
5. **Reflexão:** explique por que registrar o `addEventListener` dos cards precisa acontecer **depois** de `innerHTML = html` .

Capítulo 17 — Reaproveitando o padrão: a API do Rick and Morty

Um dos maiores prazeres da programação bem-feita é quando você percebe que já tem o padrão pronto e só precisa aplicá-lo de novo. Neste capítulo vamos construir a tela do **Rick and Morty** — personagens de um desenho, com lista, paginação e modal de detalhes. E você vai notar algo revelador: é **praticamente idêntica** à tela de livros. Essa repetição não é preguiça — é a prova de que criamos uma **arquitetura sólida e reutilizável**.

17.1 A mesma receita, outros ingredientes

Compare mentalmente com o capítulo anterior. A estrutura é a mesma:

- Uma camada de **serviço** que busca e limpa os dados.
- Uma **página** com variável de estado, renderização de cards, paginação e modal.
- Os mesmos padrões: `async/await`, `try/catch/finally`, `data-id`, `classList`.

O que muda? Só **os dados e os campos**. Livros têm `titulo` e `autores`; personagens têm `nome`, `status` e `especie`. A “forma” da solução é idêntica. Isso é o que chamamos de **padrão de projeto**: uma solução testada que você reaplica em problemas semelhantes.



Nos bastidores — a API Rick and Morty A [Rick and Morty API](#) é gratuita e não exige cadastro. A lista de personagens vem de:

```
https://rickandmortyapi.com/api/character?page=2
```

A resposta traz `info` (com `pages`, o total de páginas — já calculado por eles!) e `results` (o array de personagens). E cada personagem tem campos **aninhados** como `origin.name` e `location.name` — lembra dos objetos aninhados do Capítulo 2? Chegou a hora de usá-los.

17.2 O serviço: `services/rickandmorty.js`

Crie `src/js/services/rickandmorty.js`. Note como ele é ainda mais simples que o de livros, porque a API já entrega os dados de forma amigável:

```
// src/js/services/rickandmorty.js
import buscarDados from "../api.js";

const BASE_URL = "https://rickandmortyapi.com/api/character";

async function buscarListaPersonagens(pagina) {
  const dados = await buscarDados(`${BASE_URL}?page=${pagina}`);

  const personagens = [];
  for (const item of dados.results) {
    personagens.push({
      id: item.id,
      nome: item.name,
      imagem: item.image,
      status: item.status,
      especie: item.species
    });
  }

  return { totalPaginas: dados.info.pages, personagens };
}

async function buscarDetalhePersonagem(id) {
  const dados = await buscarDados(`${BASE_URL}/${id}`);

  return {
    id: dados.id,
    nome: dados.name,
    imagem: dados.image,
    status: dados.status,
    especie: dados.species,
    genero: dados.gender,
    origem: dados.origin.name,
    localizacaoAtual: dados.location.name
  };
}

export { buscarListaPersonagens, buscarDetalhePersonagem };

```

Compare com `services/livros.js`. É a **mesma estrutura**:

- Importa o `buscarDados` genérico (Cap. 14).
- `buscarListaPersonagens` percorre `dados.results` com `for...of` e monta um array de objetos limpos.
- `buscarDetalhePersonagem` traz o detalhe de um personagem.
- Exportação nomeada das duas funções.

Duas diferenças dignas de nota:

1. **O total de páginas vem pronto:** `dados.info.pages`. Não precisamos calcular com `Math.ceil` como fizemos nos livros — cada API tem suas peculiaridades, e o serviço é justamente o lugar de lidar com elas.
2. **Objetos aninhados:** `dados.origin.name` e `dados.location.name`. Aqui está o Capítulo 2 em ação — “cavamos” dentro de um objeto que está dentro de outro objeto. `dados.origin` é um objeto; `.name` é a propriedade dentro dele. Nós achatamos isso para `origem` e `localizacaoAtual`, campos simples que a tela consumirá sem esforço.

🧩 **Montando o quebra-cabeça** Repare que a página **não faz ideia** de que os dados de origem estavam aninhados na API. Ela vai usar `personagem.origem`, um campo simples. Toda a complexidade do formato da API ficou contida no serviço. **É exatamente esse o propósito da camada de serviços** — e agora você viu o mesmo princípio funcionar para duas APIs diferentes, com formatos diferentes, sem a página perceber a diferença.

17.3 A página: `paginas/rickandmorty.js`

A página é um “gêmeo” da de livros. Crie `src/js/components/paginas/rickandmorty.js`:

```

// src/js/components/paginas/rickandmorty.js
import { buscarListaPersonagens, buscarDetalhePersonagem } from "../../services/ricka
import { mostrarCarregando, esconderCarregando } from "../../services/loading.js";

let paginaAtual = 1;

async function telaRickAndMorty(app) {
  paginaAtual = 1;
  app.innerHTML = `
    <div id="modalDetalhes" class="bem-modal bem-modal--hidden">
      <div class="bem-modal__dialog">
        <div class="bem-modal__header">
          <h2 class="bem-modal__title" id="detalheTitulo"></h2>
          <button id="fecharDetalhes" class="bem-modal__close">&times;</button>
        </div>
        <div class="bem-modal__body" id="detalheCorpo"></div>
      </div>
    </div>

    <div class="bem-container">
      <h1 class="bem-mb-md">Rick and Morty</h1>
      <div id="listaPersonagens" class="bem-grid-auto"></div>
      <div class="bem-flex bem-justify-center bem-items-center bem-gap-md bem-mt-lg">
        <button id="paginaAnterior" class="bem-btn bem-btn--outline">Anterior</button>
        <span id="indicadorPagina"></span>
        <button id="proximaPagina" class="bem-btn bem-btn--outline">Próxima</button>
      </div>
    </div>
  `;

  document.getElementById("fecharDetalhes").addEventListener("click", fecharDetalhes)
  document.getElementById("paginaAnterior").addEventListener("click", () => mudarPagina)
  document.getElementById("proximaPagina").addEventListener("click", () => mudarPagina)

  await renderizarPagina();
}

async function mudarPagina(direcao) {
  paginaAtual += direcao;
  if (paginaAtual < 1) paginaAtual = 1;
  await renderizarPagina();
}

async function renderizarPagina() {
  mostrarCarregando();
  try {
    const { totalPaginas, personagens } = await buscarListaPersonagens(paginaAtual);

    let html = "";
    for (const personagem of personagens) {

```



```
document.getElementById("modalDetalhes").classList.add("bem-modal--hidden");
}

export default {
  url: "#rickandmorty",
  label: "Rick and Morty",
  pagina: telaRickAndMorty
};
```

Se você comparar este arquivo com o `livros.js` do capítulo anterior, verá que a **estrutura é idêntica** — mudam apenas os nomes (`personagens` em vez de `livros` , `#listaPersonagens` em vez de `#listaLivros`) e os campos exibidos no card e no modal. Adicione a rota no `rotas.js` e a tela está no ar.

17.4 A lição mais importante: reconhecer padrões

Você pode estar pensando: “mas isso é código repetido, não é ruim?” É uma pergunta excelente, e a resposta tem duas camadas.

Primeiro: repetir um padrão que você entende é infinitamente melhor do que inventar uma solução diferente para cada tela. A consistência torna o projeto previsível — quem entende a tela de livros entende a de personagens em segundos.

Segundo: o fato de as duas telas serem tão parecidas é um **sinal**. Em projetos maiores, esse é o momento em que um desenvolvedor experiente pensa em **abstrair** — criar uma função ou componente genérico “tela de listagem com paginação e modal” que receba a fonte de dados e os campos como parâmetros. Não faremos isso agora (seria um salto de complexidade), mas plantamos a semente: a capacidade de **enxergar a repetição** é o primeiro passo para eliminá-la quando valer a pena.



Nos bastidores — a regra dos três Uma heurística famosa: na **primeira** vez, escreva a solução. Na **segunda**, tolere a duplicação (como fizemos aqui). Na **terceira**, abstraia. Duplicar cedo demais cria abstrações erradas; abstrair no momento certo, depois de ver o padrão se repetir, gera código realmente bom. Você acabou de viver a segunda ocorrência — e agora tem olhos para reconhecer a terceira.

17.5 O projeto completo

Com esta tela, o App Livros está **completo**. Vamos revisar a estrutura final de arquivos que você construiu, capítulo a capítulo:

```
app_livros/
├─ index.html                # Cap. 7, 8, 11
├─ src/
│   ├─ css/microframework.css # Cap. 11
│   └─ js/
│       ├─ main.js           # Cap. 7 → 10 (roteador)
│       ├─ components/
│           ├─ navbar/navbar.js # Cap. 9
│           ├─ rotas/rotas.js   # Cap. 9
│           └─ paginas/
│               ├─ home.js      # Cap. 8
│               ├─ sobre.js     # Cap. 8
│               ├─ servicos.js  # Cap. 9
│               ├─ contato.js   # Cap. 8
│               ├─ formCad.js   # Cap. 15
│               ├─ livros.js    # Cap. 16
│               └─ rickandmarty.js # Cap. 17
│       └─ services/
│           ├─ api.js           # Cap. 14
│           ├─ loading.js       # Cap. 14
│           ├─ livros.js        # Cap. 16
│           └─ rickandmarty.js  # Cap. 17
```

Cada arquivo dessa árvore tem uma responsabilidade clara, e você entende o porquê de cada um. Isso é uma aplicação de verdade — e você a construiu peça por peça, sem frameworks, entendendo cada linha. **Parabéns.**

Recapitulando

- A tela do Rick and Morty **reaproveita o padrão** da tela de livros — só mudam os dados e os campos.
- Cada API tem particularidades (o total de páginas vinha pronto em `info.pages` ; os dados vinham aninhados em `origin.name`), e o **serviço** é o lugar de lidar com elas.
- A página **nunca percebe** essas diferenças — a camada de serviços a protege do formato cru.
- Reconhecer a repetição é uma habilidade: aplique a **regra dos três** antes de abstrair.
- O projeto está **completo**: sete páginas, roteador, camada de serviços e CSS, tudo em JavaScript puro.

Exercícios do Capítulo 17

1. Construa o serviço e a página do Rick and Morty e confirme que lista, paginação e modal funcionam.
2. Adicione ao modal o campo `episódios` (dica: a API traz `episode`, um array — mostre quantos episódios usando `.length`).
3. **Desafio de abstração:** liste, no papel, tudo o que a tela de livros e a de personagens têm em comum. Desenhe a assinatura de uma função genérica `criarTelaListagem(config)` que poderia gerar as duas.
4. Adicione um campo de **busca por nome** à tela (a API aceita `?name=rick`). Um input, um evento, e passe o termo ao serviço.
5. **Reflexão:** em suas palavras, por que “código repetido que você entende” é preferível a “uma solução criativa e diferente para cada tela”?

Capítulo 18 — Boas práticas, organização e próximos passos

Você construiu uma SPA completa do zero. Neste capítulo final, vamos dar um passo atrás para enxergar o **quadro geral**: os princípios que guiaram cada decisão do projeto, as boas práticas que valem para qualquer código que você escrever daqui em diante, e o caminho para continuar evoluindo — inclusive rumo aos frameworks.

18.1 Os princípios que nos guiaram

Ao longo do curso, algumas ideias apareceram repetidamente. Elas não são específicas do App Livros — são princípios universais da boa engenharia de software. Guarde-os:

Responsabilidade única

Cada arquivo, cada função, faz **uma coisa**. `api.js` só faz requisições. `navbar.js` só desenha o menu. `buscarListaLivros` só busca e limpa a lista. Quando cada peça tem um propósito claro, o projeto inteiro fica fácil de entender e de consertar.

DRY — Don't Repeat Yourself

Cada informação tem uma **fonte única de verdade**. A lista de rotas em `rotas.js` alimenta tanto o menu quanto o roteamento. A função `buscarDados` centraliza toda requisição. Duplicação é dívida: um dia você muda um lugar e esquece o outro.

Separação de responsabilidades (camadas)

Separamos **estrutura** (HTML), **apresentação** (CSS) e **comportamento** (JavaScript). E, dentro do JS, separamos **interface** (`components/paginas/`) de **dados** (`services/`). Essa arquitetura em camadas é o que permite trocar uma API sem tocar na tela, ou redesenhar a tela sem tocar nos dados.

Nomes que comunicam

`renderizarRotaAtual` , `buscarDetalheLivro` , `mostrarCarregando` . Os nomes **dizem o que fazem**. Código é lido muitas mais vezes do que é escrito — escreva para quem vai ler (que muitas vezes é você mesmo, meses depois).

💡 **Nos bastidores** Esses quatro princípios são exatamente o que os frameworks impõem por baixo dos panos. React separa componentes; Angular tem serviços; todos pregam responsabilidade única e DRY. Você não aprendeu “JavaScript puro” como algo menor — aprendeu os **fundamentos** que todo framework apenas empacota.

18.2 Boas práticas de código

Um apanhado de hábitos que fazem diferença, muitos já vistos em contexto:

- Use **const** por padrão, **let** só quando o valor mudar, e evite **var**.
- Compare com **===** e **!==** — sempre estrito.
- Trate erros em toda operação assíncrona (**try/catch**), e limpe no **finally**.
- **Dê feedback ao usuário**: um “Carregando...” durante esperas, uma mensagem em caso de erro. Interface silenciosa parece quebrada.
- **Desenhe primeiro, escute depois**: registre eventos só após os elementos existirem no DOM.
- **Não confie em dados externos**: valide o que vem de formulários e de APIs (lembra do CEP inválido?).
- **Comente o “porquê”, não o “o quê”**: o código já diz o que faz; um bom comentário explica uma decisão não óbvia.
- **Constantes nomeadas** em vez de “números mágicos”: **ITENS_POR_PAGINA = 32** é melhor que um **32** solto no meio de uma conta.

18.3 Ferramentas do dia a dia

Você usou algumas o tempo todo; vale reconhecê-las como parte do ofício:

- **O Console do navegador (DevTools)** — seu laboratório e seu detetive. Use **console.log** para inspecionar valores, a aba *Network* para ver as requisições e a aba *Elements* para examinar o DOM ao vivo.
- **O Git** — o histórico do projeto conta a sua evolução, commit a commit (“criação da tela de cadastro”, “componente auto-montável”, “centralização do roteamento”). Versionar seu código é inegociável. Cada funcionalidade merece seu commit.
- **Um servidor local (Live Server)** — indispensável para os ES Modules.

🧩 **Montando o quebra-cabeça** O próprio App Livros nasceu assim: um repositório Git em que cada sábado de aula virou um ou mais commits. Se você reproduziu o projeto seguindo esta apostila, considere fazer o mesmo — um commit ao final de cada capítulo. Daqui a um ano, seu histórico contará a sua jornada de aprendizado.

18.4 O que ainda dá para melhorar no projeto

Nenhum projeto está “pronto”. Alguns aprimoramentos que seriam ótimos exercícios:

1. **Validação de formulários** — checar CEP, e-mail e campos obrigatórios, mostrando mensagens de erro amigáveis (o CSS já tem as classes `bem-form__error` e `bem-form__input--error`).
2. **Estado de “nenhum resultado”** — quando uma busca não retorna nada, mostrar “Nenhum item encontrado” em vez de uma grade vazia.
3. **Tratamento de imagens ausentes** — um placeholder quando o livro não tem capa.
4. **Abstração da tela de listagem** — a “regra dos três” (Cap. 17): unificar as telas de livros e personagens numa função genérica.
5. **Persistência** — salvar dados no `localStorage` (por exemplo, as mensagens de contato) para que sobrevivam ao recarregamento.
6. **Acessibilidade** — atributos ARIA, foco no modal, navegação por teclado.
7. **Botão de troca de tema** — a base CSS de temas já existe (Cap. 11); falta a interface para o usuário escolher.

18.5 A continuação imediata do curso: guardar dados no navegador

Nosso App Livros já busca dados da internet — mas ele tem uma “amnésia”: toda vez que você abre a tela de livros, ele vai **de novo** à API buscar tudo, mesmo que os dados sejam os mesmos de segundos atrás. Feche a aba, reabra, e todo o trabalho é refeito do zero. O próximo passo natural da sua jornada é ensinar a aplicação a **lembrar**. Para isso, o navegador oferece o **Web Storage**: `localStorage` e `sessionStorage`.

O que são

São dois “cofrinhos” que o navegador dá para cada site guardar dados no formato **chave** → **valor** (texto), do jeitinho que já conhecemos dos objetos (Cap. 2):

```
// guardar
localStorage.setItem("tema", "noite");

// ler
const tema = localStorage.getItem("tema"); // "noite"

// remover
localStorage.removeItem("tema");

// apagar tudo
localStorage.clear();
```

A diferença entre os dois está no **tempo de vida**:

Recurso	Vive enquanto...	Uso típico
<code>localStorage</code>	...você não apagar — persiste para sempre, mesmo fechando o navegador	tema escolhido, dados em cache, “lembrar de mim”
<code>sessionStorage</code>	...a aba estiver aberta — some ao fechá-la	dados de um formulário em preenchimento, estado temporário

⚠ **Cuidado — só guarda texto** O Web Storage só armazena **strings**. Para guardar um objeto ou array, converta-o para texto com `JSON.stringify` ao salvar e de volta para objeto com `JSON.parse` ao ler:

```
const livros = [{ id: 1, titulo: "Dom Casmurro" }];
localStorage.setItem("livros", JSON.stringify(livros)); // objeto → texto
const salvos = JSON.parse(localStorage.getItem("livros")); // texto → objeto
```

Lembra do `resposta.json()` do Capítulo 13, que virava texto JSON em objeto? Aqui é o mesmo formato JSON, agora no sentido inverso ao salvar.

O grande motivo: economizar rede e criar cache

Aqui está a ideia mais valiosa deste tópico, e o motivo pelo qual ele vem **logo depois** do consumo de API. Cada requisição à internet **custa**: custa tempo (o usuário espera), custa dados (importante em redes móveis) e custa recursos do servidor (que atende a chamada). Buscar **o mesmo recurso várias vezes** é desperdício puro.

O `localStorage` permite construir um **cache** — uma cópia local dos dados já buscados. A lógica é simples e poderosa:

Antes de ir à internet, **pergunte ao cache**. Se o dado já estiver lá, use-o e **evite a requisição**. Se não estiver, busque na API e **guarde no cache** para a próxima vez.

Veja como aplicaríamos isso ao nosso próprio `services/api.js`, criando um cache personalizado por cima da função `buscarDados` que já construímos no Capítulo 14:

```
// uma versão da busca que consulta o cache antes de ir à rede
async function buscarComCache(url) {
  const emCache = localStorage.getItem(url); // a própria URL é a chave!

  if (emCache) {
    console.log("Servido do cache (sem rede):", url);
    return JSON.parse(emCache); // devolve a cópia local, instantâneo
  }

  console.log("Buscando na internet:", url);
  const dados = await buscarDados(url); // nossa função do Cap. 14
  localStorage.setItem(url, JSON.stringify(dados)); // guarda para a próxima
  return dados;
}
```

Repare na elegância: usamos a **própria URL como chave** do cache. Assim, a página 2 dos livros (`.../books/?page=2`) e a página 3 (`.../books/?page=3`) têm caches separados e independentes. Ao visitar a página 2, os dados aparecem **instantaneamente**, sem uma única requisição — nem o “Carregando...” pisca.

 **Nos bastidores — cache com validade** Um cache “eterno” tem um risco: os dados podem **ficar velhos** (a API mudou, mas você continua mostrando a cópia antiga). Caches profissionais guardam, junto com o dado, um **carimbo de tempo** e uma **validade** (o famoso “TTL”, *time to live*). Ao ler, você checa: “esse cache tem menos de 10 minutos? Se sim, uso; se não, busco de novo e atualizo”. É um ótimo exercício para evoluir o `buscarComCache` acima — guarde `{ dados, salvoEm: Date.now() }` e compare com `Date.now()` na leitura.

 **Montando o quebra-cabeça** Este é o gancho perfeito para a continuação do curso. O App Livros está **pronto** para receber cache: como toda requisição já passa por um único lugar (`services/api.js`, o princípio DRY do Cap. 9), basta trocar as chamadas de `buscarDados` por `buscarComCache` e **toda** a aplicação — livros, personagens, CEP — ganha cache de uma vez. É a recompensa de ter uma arquitetura bem separada em camadas.

18.6 Indo além: Service Workers e o app que funciona offline

O `localStorage` guarda **dados**. Mas e se pudéssemos guardar o **próprio aplicativo** — HTML, CSS, JS, imagens — para que ele abrisse **sem internet**? É isso que os **Service Workers** permitem, e é o passo seguinte natural depois do Web Storage.

Um **Service Worker** é um script especial que roda **em segundo plano**, separado da sua página, funcionando como um **intermediário entre a aplicação e a rede**. Toda requisição que a aplicação faz pode passar por ele primeiro. Isso lhe dá superpoderes:

- **Interceptar requisições** e responder com uma cópia em cache — levando a economia de rede da seção anterior a outro nível, agora para os arquivos do próprio site.
- **Funcionar offline**: com os arquivos essenciais guardados, o app abre mesmo sem conexão.
- **Estratégias de cache** inteligentes (“cache primeiro”, “rede primeiro”, “rede com fallback para cache”).

```
// exemplo mínimo: guardar os arquivos do app na instalação
self.addEventListener("install", (event) => {
  event.waitUntil(
    caches.open("app-livros-v1").then((cache) =>
      cache.addAll(["/", "/index.html", "/src/css/microframework.css"])
    )
  );
});

// interceptar requisições: tentar o cache antes da rede
self.addEventListener("fetch", (event) => {
  event.respondWith(
    caches.match(event.request).then((resposta) => resposta || fetch(event.request))
  );
});
```

Não se assuste com a sintaxe nova (`self` , `caches` , `event.respondWith`) — o que importa agora é reconhecer o **conceito**, que é o mesmo do `buscarComCache` : *pergunte ao cache antes de ir à rede*. Repare no `caches.match(...) || fetch(...)` : é o mesmíssimo padrão do `||` que usamos no roteador (`mapaDeRotas[hash] || paginaNaoEncontrada` , Cap. 10). Você já pensa assim.

💡 **Nos bastidores — isso vira um PWA** Um site com Service Worker, um cache bem planejado e um pequeno arquivo de configuração (o *manifest*) se transforma num **PWA** (*Progressive Web App*): um site que **pode ser instalado** como um aplicativo, com ícone na tela inicial, tela cheia e funcionamento offline. É a fronteira entre “site” e “aplicativo” — e o App Livros está a poucos passos de cruzá-la.

🧩 **Montando o quebra-cabeça — o roteiro da sequência** Juntando tudo, a continuação imediata do que você construiu tem uma ordem clara e cada etapa se apoia na anterior:

1. **Consumo de API** (*feito — Caps. 15 a 17*): trazer dados da internet.
2. **localStorage** / **sessionStorage**: lembrar dados e **cachear** respostas para economizar rede.
3. **Service Workers**: cachear o próprio app, interceptar requisições e funcionar **offline**.
4. **PWA**: empacotar tudo num aplicativo instalável. Cada peça encaixa na anterior — exatamente o espírito de quebra-cabeça deste curso.

18.7 Aprofunde o JavaScript

Paralelamente, vale robustecer a linguagem em si:

- Métodos de array além do `.map()`: `.filter()` (filtrar), `.reduce()` (acumular), `.find()` (achar um), `.forEach()` (percorrer). Eles substituem muitos loops com elegância.
- **Desestruturação** e **spread/rest** (`...`), que você já viu de relance.
- **JSON.stringify** / **JSON.parse** a fundo (essenciais para o cache acima).
- **Datas** (`Date`) e o objeto **Math**, que já usamos no `Math.ceil`.
- **Módulos mais avançados** e organização de projetos maiores.

18.8 Conheça um framework — agora com fundamento

Quando você abrir o **React**, vai reconhecer tudo:

No App Livros (você construiu)	No React (equivalente)
Página = função que retorna HTML string	Componente = função que retorna JSX
Roteador com <code>hash</code> e mapa de rotas	React Router
<code>paginaAtual</code> como variável de estado	<code>useState</code>
Camada <code>services/</code> com <code>fetch</code>	...exatamente igual!
<code>navbar(roteador)</code> gerando o menu	Renderização de listas com <code>.map()</code>

A diferença é que o framework **automatiza** a atualização da tela e oferece ferramentas prontas. Mas o **modelo mental** — componentes, estado, dados assíncronos, roteamento — é **o mesmo que você já domina**. Você não vai aprender React do zero; vai apenas aprender a sintaxe de algo que já entende.

Qual framework escolher?

Todos partem das mesmas ideias que você construiu na mão. As opções mais comuns:

- **React** — o mais usado no mercado; ecossistema gigante. Ótimo para empregabilidade. É o que mais se parece com o que fizemos: componentes que são funções retornando “HTML”.
- **Vue** — considerado o mais amigável para quem vem do JavaScript puro; a curva de aprendizado é suave e a documentação é excelente.
- **Angular** — mais “robusto” e opinativo, usado em grandes empresas; já traz tudo embutido (roteamento, requisições, formulários), à maneira do nosso `services/`, só que oficial.
- **Svelte** — o mais moderno e enxuto; compila para JavaScript puro parecido com o que você já escreve.

 **Nos bastidores — como estudar um framework partindo daqui** Sugestão de método: pegue o **próprio App Livros** e reconstrua-o no framework escolhido, tela por tela. Comece pela navbar (uma lista com `.map()`), depois uma página estática (Sobre), depois a tela de livros (estado + `fetch` + lista). Como você **já conhece o resultado esperado**, poderá focar 100% na sintaxe nova, sem se preocupar com a lógica. É a forma mais rápida de aprender: traduzir algo que você domina para uma nova linguagem.

O que também sustenta o front-end profissional

- **HTTP e APIs REST** a fundo (métodos GET/POST/PUT/DELETE, status codes, cabeçalhos).
- **Ferramentas de build** (Vite) e **npm** para gerenciar dependências.
- **TypeScript** — JavaScript com tipos, que previne muitos erros.
- **Testes automatizados** — para ter confiança ao mudar o código.

18.9 O outro lado da força: o backend

Até aqui, sempre **consumimos** APIs feitas por outras pessoas (ViaCEP, Gutendex, Rick and Morty). Mas quem **constrói** essas APIs? O **backend** — o lado do servidor. Aprender backend é o que fecha o ciclo e te torna capaz de construir uma **aplicação completa**, do banco de dados até a tela.

Pense no que sempre esteve “do outro lado” do nosso `fetch`:

Front-end (você domina)

`fetch("../gutendex.com/books")`

mostra na tela (você fez isso)

Back-end (o próximo passo)

→ um servidor recebe o pedido

→ consulta um banco de dados

← devolve os livros em JSON

O que estudar para dominar esse lado:

- **Node.js** — rodar JavaScript **fora** do navegador, no servidor. A grande vantagem para você: é **a mesma linguagem** que já aprendeu. Funções, arrays, objetos, `async/await` — tudo se aplica, agora no servidor.
- **Express** (ou Fastify) — um framework para **criar as suas próprias APIs**, definindo rotas como `GET /livros` e `POST /livros`. É o espelho do nosso roteador de front-end, do lado do servidor.
- **Bancos de dados** — onde os dados **realmente** ficam guardados de forma permanente. Duas famílias: **SQL** (PostgreSQL, MySQL — dados em tabelas) e **NoSQL** (MongoDB — dados em documentos parecidos com objetos JS).
- **Autenticação e autorização** — logins, senhas, tokens (JWT), sessões: garantir que cada usuário acesse só o que pode.
- **Deploy** — colocar a aplicação no ar (Vercel, Netlify para o front; Render, Railway, ou um servidor na nuvem para o back).

🧩 **Montando o quebra-cabeça — o projeto que fecha o ciclo** O exercício definitivo: **construa a sua própria API de livros** com Node + Express + banco de dados, e faça o App Livros consumir **a sua API** em vez da Gutendex. Você teria escrito as **duas pontas** — e entenderia, de verdade, o que acontece dos dois lados de cada `fetch`. Esse é o momento em que você deixa de ser “quem faz telas” e passa a ser **desenvolvedor full-stack**: capaz de construir uma aplicação inteira sozinho.

18.10 Um roteiro de estudos sugerido

Reunindo tudo, uma ordem possível para os próximos meses — cada etapa se apoia na anterior:

1. **Web Storage** (`localStorage` / `sessionStorage`) e **cache** de requisições — *a continuação direta deste curso*.
2. **Service Workers** e **PWA** — offline e app instalável.
3. **Métodos de array** avançados e **desestruturação** — polir o JavaScript.
4. **Um framework** (React ou Vue) — reconstruindo o App Livros nele.
5. **TypeScript** — tipos para código mais seguro.
6. **Node + Express + banco de dados** — o backend, criando a sua própria API.
7. **Autenticação e deploy** — colocar uma aplicação completa no ar.

Não precisa correr. Cada item, bem estudado, é uma competência valiosa por si só.

18.11 Palavras finais

Comece de novo, do topo, e repare no tamanho da jornada. No Capítulo 1, você declarava variáveis. No Capítulo 17, você orquestrava duas APIs, com paginação, modais e tratamento de erros, tudo numa arquitetura em camadas. Entre um ponto e outro, cada conceito se encaixou no anterior como uma peça de quebra-cabeça:

- Variáveis e condicionais viraram **decisões de rota**.
- Arrays de objetos viraram **a lista de rotas e os dados das telas**.
- Funções viraram **páginas e serviços**.
- Loops viraram **renderização de listas**.
- O DOM virou **a tela viva**.
- O assincronismo virou **dados do mundo real**.

Você não decorou receitas — você **entendeu o mecanismo**. E entender o mecanismo é o que separa quem apenas usa ferramentas de quem as domina, adapta e, um dia, constrói as suas.

O melhor código que você vai escrever ainda está à sua frente. Continue montando quebra-cabeças. 🧩🚀

Recapitulando

- Os pilares do projeto — **responsabilidade única, DRY, separação em camadas e nomes claros** — são princípios universais.

- Boas práticas: `const` por padrão, `===`, tratar erros, dar feedback, desenhar antes de escutar, validar entradas.
- Domine as ferramentas: **DevTools**, **Git**, servidor local.
- A continuação imediata é o **Web Storage**: `localStorage` / `sessionStorage` para lembrar dados e **cache**ar requisições, economizando rede; depois **Service Workers** e **PWA** para funcionar offline.
- Guardar objetos exige `JSON.stringify` ao salvar e `JSON.parse` ao ler.
- Um **framework** (React/Vue) vai parecer familiar, porque você já construiu na mão o que ele automatiza; e o **backend** (Node/Express/banco) fecha o ciclo rumo ao **full-stack**.

Exercícios do Capítulo 18 (projeto final)

1. Escolha **três** melhorias da seção 18.4 e implemente-as no seu App Livros.
2. Implemente o **botão de troca de tema** usando a base CSS do Capítulo 11 e **guarde a escolha no `localStorage`**, de modo que o tema persista ao recarregar a página.
3. **Cache de rede**: implemente a função `buscarComCache` da seção 18.5 e faça o serviço de livros usá-la. Abra o Console e a aba *Network* das DevTools e confirme que, ao revisitar uma página já vista, **nenhuma requisição nova** é feita.
4. **Cache com validade**: evolua o `buscarComCache` para guardar `{ dados, salvoEm: Date.now() }` e só usar o cache se ele tiver menos de 10 minutos; caso contrário, busque de novo e atualize.
5. Use o `sessionStorage` para salvar o que o usuário digitou no formulário de contato, de forma que, ao trocar de página e voltar, os campos continuem preenchidos (mas sejam esquecidos ao fechar a aba).
6. Versione seu projeto no Git com commits organizados, um por funcionalidade.
7. **Projeto full-stack**: usando a mesma arquitetura, crie uma tela que consome **outra** API pública (ex.: PokéAPI, TheMealDB). Como desafio maior, construa a **sua própria** API com Node + Express e faça o App Livros consumi-la.

Capítulo 19 — Guardando dados no navegador: armazenamentos, cache e estratégias

No Capítulo 18 fizemos um panorama dos próximos passos. Agora vamos **colocar a mão na massa** no primeiro deles — e talvez o de melhor custo-benefício de todo o projeto. Nosso App Livros sofre de uma “amnésia”: toda vez que você entra na tela de livros ou de personagens, ele vai **de novo** à internet buscar exatamente os mesmos dados. Feche a aba, reabra, e todo o trabalho é refeito do zero.

Neste capítulo vamos curar essa amnésia conhecendo **todos os armazenamentos do navegador**, construindo um **cache de verdade** sobre a camada de serviços — com dois padrões de projeto clássicos, o **Decorator/Proxy** e o **Strategy** — e aprendendo o vocabulário de **estratégias de cache** que o mercado usa todos os dias.



Nos bastidores — onde este capítulo se encaixa Esta é a sequência natural depois do consumo de API (Caps. 15 a 17). A ordem completa da nossa jornada de dados é: **buscar da API** → **guardar/cachear localmente (este capítulo)** → **funcionar offline com Service Workers (próximo capítulo)**. Cada peça se apoia na anterior.

19.1 O problema, medido com os próprios olhos

Antes de resolver, vamos **enxergar** o desperdício. Abra o App Livros, entre na tela de personagens e abra as DevTools (**F12**) na aba **Network** (Rede). Navegue para a página 2, volte para a 1, vá para a 3, volte para a 1 de novo.

Cada ida e volta dispara uma **nova requisição** — inclusive para páginas que você **já tinha visto**. A página 1 foi baixada da internet duas, três vezes, sempre com o mesmo conteúdo. Isso é desperdício em três frentes:

- **Tempo** — o usuário espera o “Carregando...” de novo a cada visita.
- **Dados** — em uma rede móvel, cada requisição consome o pacote de internet do usuário.
- **Servidor** — cada chamada ocupa recursos de quem hospeda a API (que muitas vezes tem limites de uso).

A pergunta que guia este capítulo é: *por que buscar de novo algo que já temos?*

Cache está em todo lugar (você usa o dia inteiro)

A resposta a essa pergunta tem nome — **cache**: uma **cópia local** de um dado que custa caro buscar na origem. Antes do código, repare como a indústria inteira funciona sobre camadas de cache:

- O **navegador** guarda imagens, CSS e scripts dos sites que você visita — por isso a segunda visita é sempre mais rápida que a primeira.
- O **Instagram** e o **X** mostram o feed antigo na hora quando você abre o app sem internet, e atualizam quando a conexão volta. Aquele feed é um cache.
- O **Spotify** guarda as músicas baixadas para tocar sem rede.
- A **Netflix** instala servidores de cache dentro dos provedores de internet (as famosas **CDNs**) para o filme sair de perto de você, e não do outro lado do planeta.
- O seu **processador** tem memórias de cache (L1, L2, L3), porque ir até a RAM é lento demais para ele.
- O **DNS**, que traduz `google.com` em endereço IP, é cacheado pelo sistema operacional para não perguntar de novo a cada clique.

Estudos famosos da Amazon e do Google associam **cada 100 ms a mais de espera** a quedas mensuráveis de vendas e engajamento. Velocidade é funcionalidade — e cache é a forma mais barata de comprá-la.

“Existem apenas duas coisas difíceis na computação: invalidação de cache e dar nomes às coisas.” — Phil Karlton

Vamos encontrar as duas neste capítulo.

19.2 O tour completo: os cinco armazenamentos do navegador

Para ter cache no front-end, precisamos de um lugar para guardar as cópias. O navegador oferece **cinco opções**, cada uma com sua vocação. Conhecer as cinco é o que permite escolher bem.

Cookies — o veterano

Guardam textos minúsculos (~4 KB) e têm uma característica única: são **enviados automaticamente ao servidor** em toda requisição àquele domínio. Por isso servem para **identificação de sessão** (o servidor reconhece que você é você) — e por isso mesmo **não** servem para cache: inflariam cada requisição com dados que o servidor não pediu.

```
document.cookie = "tema=noite; max-age=31536000";
```

localStorage — o armário permanente

Chave → valor, ~5 a 10 MB por site, **persiste para sempre** (mesmo fechando o navegador), até alguém apagar. Quatro métodos e você domina o essencial:

```
localStorage.setItem("tema", "noite"); // guardar
localStorage.getItem("tema");          // ler (null se não existir)
localStorage.removeItem("tema");       // remover uma chave
localStorage.clear();                  // apagar tudo do site
```

sessionStorage — o bolso do passeio

Mesma API do `localStorage`, mas os dados **morrem quando a aba fecha**. Pense no `localStorage` como um armário em casa e no `sessionStorage` como o bolso durante um passeio: esvazia quando o passeio acaba. Ideal para rascunhos de formulário e estado temporário (voltaremos a ele na seção 19.9).

IndexedDB — o banco de dados local

Um banco de dados **de verdade** dentro do navegador: guarda objetos sem conversão para texto, aceita **centenas de MB**, tem índices de busca e transações, e toda a API é assíncrona. É o que aplicações grandes (Google Docs, por exemplo) usam para trabalhar offline com muitos dados. O custo é uma API verbosa — para o nosso volume, o `localStorage` resolve. Fica registrado como o próximo degrau quando o cache crescer.

Cache API — feita para a rede

Em vez de chave → texto, ela guarda **pares de requisição e resposta HTTP completas** — como um mini servidor dentro do navegador:

```
const cache = await caches.open("app-livros-v1");
await cache.add("https://rickandmortyapi.com/api/character/?page=1");
const resposta = await caches.match("https://rickandmortyapi.com/api/character/?page=
```

É a parceira natural dos **Service Workers**, estrelas do próximo capítulo.

Tabela de decisão

Armazenamento	Capacidade	Vive até...	Vocação
Cookies	~4 KB	a expiração definida	sessão e identificação junto ao servidor
<code>localStorage</code>	~5–10 MB	ser apagado	preferências e cache simples de dados
<code>sessionStorage</code>	~5 MB	a aba fechar	rascunhos e estado temporário
IndexedDB	centenas de MB	ser apagado	grandes volumes estruturados, offline pesado
Cache API	grande (cota)	ser apagado	respostas HTTP completas, Service Workers

 **Cuidado — o que NÃO guardar** Qualquer script da página consegue ler esses armazenamentos. Nunca guarde dados **sensíveis** (senhas, tokens de acesso, dados de cartão) neles. Cache é para dados públicos e preferências — não para segredos.

 **Nos bastidores — quanto cabe e onde vejo?** Você pode inspecionar tudo o que está guardado nas DevTools: aba **Application** (ou *Armazenamento*) → *Local Storage* / *Session Storage* / *IndexedDB* / *Cache Storage*. Deixe essa aba aberta durante o capítulo inteiro.

19.3 O detalhe que pega todo mundo: só texto

Há uma regra de ouro no Web Storage:

O Web Storage guarda **apenas strings (texto)**. Nada de objetos ou arrays diretamente.

Se você tentar guardar um objeto, ele vira o texto inútil `"[object Object]"`:

```
const livro = { id: 1, titulo: "Dom Casmurro" };
localStorage.setItem("livro", livro);
localStorage.getItem("livro"); // "[object Object]" ❌ perdeu tudo!
```

A solução é converter para o formato **JSON** — o mesmo formato de troca de dados que já vimos no `fetch` (Cap. 13). Duas funções fazem a ponte:

- `JSON.stringify(objeto)` — transforma um objeto/array em **texto JSON** (para **salvar**).

- `JSON.parse(texto)` — transforma o texto JSON de volta em **objeto** (para ler).

```
const livro = { id: 1, titulo: "Dom Casmurro" };

localStorage.setItem("livro", JSON.stringify(livro)); // objeto → texto
const salvo = JSON.parse(localStorage.getItem("livro")); // texto → objeto
console.log(salvo.titulo); // "Dom Casmurro" ✓
```

✂ **Montando o quebra-cabeça** Repare na simetria com o Capítulo 13: lá, `resposta.json()` converte o **texto JSON vindo da internet** em objeto. Aqui, `JSON.parse` / `JSON.stringify` fazem a mesma conversão nas **duas direções**, agora para guardar localmente. É o mesmo formato JSON, o mesmo conceito — você já conhecia metade disso.

19.4 Primeira vitória: lembrar o tema escolhido

Vamos a um uso simples e direto antes do cache de rede. No Capítulo 11 criamos temas (dia, tarde, noite) trocáveis via `data-theme`. O problema: ao recarregar a página, o tema volta ao padrão. Com `localStorage`, fazemos a escolha **persistir**.

```
// aplicar e guardar o tema escolhido
function escolherTema(tema) {
  document.documentElement.setAttribute("data-theme", tema); // Cap. 5 e 11
  localStorage.setItem("tema", tema); // lembra a escolha
}

// ao iniciar a aplicação, restaura o tema salvo (se houver)
function restaurarTema() {
  const temaSalvo = localStorage.getItem("tema");
  if (temaSalvo) {
    document.documentElement.setAttribute("data-theme", temaSalvo);
  }
}
```

Bastaria chamar `restaurarTema()` dentro da função `iniciar()` do `main.js` (Cap. 10), e o App Livros passaria a respeitar a preferência do usuário entre visitas. Simples, e já uma melhora real de experiência.

19.5 O padrão Strategy: estratégias de armazenamento intercambiáveis

Agora rumo ao prêmio principal — o cache de requisições. Antes de escrevê-lo, uma decisão de projeto: **onde** guardar as cópias? Em memória (um `Map` — rápido, mas morre no F5)? No `localStorage` (sobrevive ao F5)? Amanhã, no IndexedDB?

Em vez de amarrar o cache a uma resposta, o projeto define um **contrato**: qualquer armazenamento serve, desde que saiba responder `has`, `get` e `set`. Cada implementação desse contrato é uma **estratégia**. É o arquivo real `src/js/components/services/storageStrategy.js` do repositório:

```
// services/storageStrategy.js
const Memoria = {
  _cache: new Map(),

  has(key) {
    return this._cache.has(key);
  },
  get(key) {
    return this._cache.get(key);
  },
  set(key, value) {
    this._cache.set(key, value);
  }
};

const LocalStorage = {
  has(key) {
    return localStorage.getItem(key) !== null;
  },
  get(key) {
    const data = localStorage.getItem(key);
    return data ? JSON.parse(data) : null;
  },
  set(key, value) {
    localStorage.setItem(key, JSON.stringify(value));
  }
};

export { Memoria, LocalStorage };
```

Duas observações de mestre de obras:

- A estratégia `LocalStorage` **esconde dentro de si** o `JSON.stringify` / `JSON.parse` da seção 19.3. Quem a usa entrega e recebe **objetos prontos**, sem saber que por baixo só

existe texto.

- As duas estratégias têm **exatamente os mesmos métodos**. Trocar de uma para a outra é trocar **uma palavra** — como você verá na próxima seção.

Isso é um padrão de projeto clássico chamado **Strategy**: uma família de algoritmos intercambiáveis atrás de uma mesma interface.

🧩 **Montando o quebra-cabeça** É o mesmo espírito do contrato das nossas páginas (`url` , `label` , `pagina` , Cap. 9): quem cumpre o contrato entra no jogo sem que o resto do código mude. Contratos uniformes são o segredo recorrente de toda a arquitetura do curso.

19.6 O Decorator/Proxy: um cache que envolve a busca

Agora a peça central. O arquivo real `services/apiCache.js` cria uma função que **envolve** a função genérica da camada de serviços, adicionando o comportamento de cache **por fora**, sem alterar uma linha dela:

```
// services/apiCache.js
import buscarServicos from "../api.js";
import { Memoria, LocalStorage } from "../storageStrategy.js";

const storage = LocalStorage;

async function buscarComCache(url, dados = '', forma = '') {
  const formataURL = `${url}${dados}${forma}`;
  if (storage.has(formataURL)) {
    console.time(`[CACHE] Tempo para: ${dados || 'página inicial'}`);
    const resultadoEmCache = storage.get(formataURL);
    console.timeEnd(`[CACHE] Tempo para: ${dados || 'página inicial'}`);
    return resultadoEmCache;
  }
  console.time(`[SERVIDOR] Tempo para: ${dados || 'página inicial'}`);
  try {
    const resultadoServidor = await buscarServicos(url, dados, forma);
    storage.set(formataURL, resultadoServidor);
    console.timeEnd(`[SERVIDOR] Tempo para: ${dados || 'página inicial'}`);
    return resultadoServidor;
  } catch (error) {
    console.timeEnd(`[SERVIDOR] Tempo para: ${dados || 'página inicial'}`);
    console.error("Erro na busca:", error);
    throw error;
  }
}

export default buscarComCache;
```

 **Nos bastidores — `buscarServicos` ?** No repositório, a função genérica da camada de serviços chama-se `buscarServicos(url, dados, forma)` — ela monta a URL em três partes (base, dado variável, sufixo) e serve qualquer API. É a mesma personagem que chamamos de `buscarDados` nos Capítulos 14 a 17; mudou o nome, não o papel.

Vamos apreciar as decisões, uma a uma:

- **A URL completa é a chave do cache.** Genial na simplicidade: `...character/?page=1` e `...character/?page=2` têm entradas separadas e independentes, automaticamente.
- **Cache primeiro (`if (storage.has(...))`).** Se a estratégia tem a chave, devolvemos a cópia local e a rede **nem é acordada**.
- **A estratégia é plugável.** A linha `const storage = LocalStorage` é o interruptor: troque por `Memoria` e toda a persistência muda — sem tocar em mais nada. O Strategy da seção anterior em ação.
- **A prova está no cronômetro.** `console.time` / `console.timeEnd` medem e imprimem o tempo de cada caminho. Navegue com o console aberto: a busca no servidor leva **dezenas**

ou centenas de milissegundos; a resposta do cache, **frações de milissegundo**. Não acredite em mim — leia os seus números.

Esse desenho — uma função que envolve outra **preservando a mesma assinatura** e adicionando comportamento — é o padrão **Decorator** (e, quando a intenção é controlar o acesso ao original, **Proxy**). O detalhe que sustenta tudo: `buscarComCache(url, dados, forma)` recebe **os mesmos parâmetros** de `buscarServicos(url, dados, forma)`. Assinaturas iguais tornam as duas **intercambiáveis**.

A troca de uma linha

E aqui a camada de serviços paga todos os seus dividendos. Para a tela de cadastro (Cap. 15) e a página de personagens (Cap. 17) ganharem cache, a mudança foi **somente o import**:

```
// antes:
// import buscarServicos from "../services/api.js"

// depois:
import buscarServicos from "../services/apiCache.js"
```

Nada mais mudou nas páginas — o decorator responde pelo original sem que ninguém perceba a troca. **Refaça o teste da seção 19.1**: página 1, página 2, volta para a 1. A aba Network fica **quieta** na revisita e o console imprime o tempo do caminho `[CACHE]`.

🧩 **Montando o quebra-cabeça** Porque **toda** requisição do projeto passa por um ponto único (Cap. 14), adotar cache custou **uma linha por página**. Se o `fetch` estivesse espalhado pelos componentes, teríamos que caçar e alterar dezenas de lugares. **Boas decisões de estrutura tornam melhorias futuras baratas** — essa lição vale mais que o próprio cache.

19.7 O problema do cache eterno: dados que envelhecem

Nosso cache tem uma falha: ele é **eterno**. Uma vez guardado, o dado nunca é atualizado. Se a API mudar, continuaríamos mostrando a cópia antiga para sempre — eis a **invalidação de cache** da frase do Karlton.

A técnica mais comum é dar ao cache uma **validade** (o **TTL**, *time to live*): guardar, junto com os dados, o **momento** em que foram salvos, e conferir a idade na leitura. E repare **onde** a mudança mora na nossa arquitetura: dentro da **estratégia**, não no decorator — o `apiCache.js` continua intocado:

```
// evolução da estratégia LocalStorage com validade de 10 minutos
const VALIDADE_MS = 10 * 60 * 1000;

const LocalStorage = {
  has(key) {
    return this.get(key) !== null; // reusa o get, que já checa validade
  },
  get(key) {
    const bruto = localStorage.getItem(key);
    if (!bruto) return null;
    const { dados, salvoEm } = JSON.parse(bruto);
    if (Date.now() - salvoEm > VALIDADE_MS) return null; // expirou!
    return dados;
  },
  set(key, value) {
    localStorage.setItem(key, JSON.stringify({ dados: value, salvoEm: Date.now() }));
  }
};
```

- Guardamos `{ dados, salvoEm }` em vez dos dados crus; `Date.now()` é o carimbo de tempo (milissegundos desde 1970).
- Na leitura, calculamos a idade e devolvemos `null` para cache vencido — o decorator então busca de novo e regrava com carimbo novo, sem saber de nada.

Agora temos **velocidade** (dados recentes vêm do cache) e **frescor** (dados velhos são renovados). Ajustar `VALIDADE_MS` é ajustar o compromisso: catálogo de livros clássicos pode ter validade longa; cotações e estoque pedem validade curta.

19.8 As estratégias de cache clássicas (o vocabulário do mercado)

O que implementamos tem nome na indústria: **cache first** — olhe o cache antes, rede só na falta. Existe um vocabulário completo, e cada estratégia resolve um compromisso diferente entre **velocidade** e **frescor**:

Estratégia	Como age	Ideal para
Cache first	responde do cache; rede apenas se não houver cópia	dados que mudam pouco (a nossa!)
Network first	tenta a rede; se falhar, cai para o cache	dados que precisam estar frescos, com plano B offline
Stale-while-revalidate	responde o cache na hora e atualiza em segundo plano	feeds e notícias — ninguém espera
Network only / Cache only	os extremos, sem fallback	casos especiais

Você acabou de construir a primeira **à mão** — e por isso as outras três são variações que você já sabe ler: mude a ordem do `if`, adicione um `catch` que consulta a estratégia, dispare a revalidação sem `await`.



Nos bastidores — onde esses nomes brilham Esses mesmos nomes são o **coração dos Service Workers** (próximo capítulo): um script que intercepta **todas** as requisições do site — HTML, CSS, imagens, scripts, não só as chamadas de API — e aplica uma dessas estratégias a cada uma, usando a **Cache API** da seção 19.2. É o que permite o feito máximo: o App Livros **abrir sem internet nenhuma**. A base conceitual você acabou de construir; o Capítulo 20 é a promoção dela para o app inteiro.

19.9 `sessionStorage` na prática: rascunho de formulário

Nem todo dado deve viver para sempre. Um formulário em preenchimento é o cenário perfeito para o `sessionStorage`: se a pessoa troca de página por engano e volta, o texto continua lá; fechou a aba, o rascunho morre junto.

```
// salvar o rascunho enquanto digita
const campoMensagem = document.getElementById("mensagem");
campoMensagem.addEventListener("input", (event) => {
  sessionStorage.setItem("rascunho-mensagem", event.target.value);
});

// ao montar a página, restaurar o rascunho (se houver)
const rascunho = sessionStorage.getItem("rascunho-mensagem");
if (rascunho) {
  campoMensagem.value = rascunho;
}

// ao enviar com sucesso, limpar o rascunho
sessionStorage.removeItem("rascunho-mensagem");
```

A escolha entre `localStorage` e `sessionStorage` é, no fundo, uma pergunta de design: “esse dado deve sobreviver ao fechamento da aba?”.

Recapitulando

- O navegador tem **cinco armazenamentos**: cookies (sessão com o servidor), `localStorage` (persistente), `sessionStorage` (por aba), IndexedDB (banco local) e Cache API (respostas HTTP completas).
 - O Web Storage só guarda **texto** — `JSON.stringify` / `JSON.parse` fazem a ponte (e a estratégia `LocalStorage` esconde isso de quem a usa).
 - O `storageStrategy.js` aplica o padrão **Strategy**: `Memoria` e `LocalStorage` cumprem o mesmo contrato `has` / `get` / `set` e são intercambiáveis com a troca de uma palavra.
 - O `apiCache.js` é um **Decorator/Proxy**: envolve `buscarServicos` com a lógica de cache **mantendo a mesma assinatura**, e prova o ganho com `console.time` — servidor em centenas de ms, cache em frações de ms.
 - A **URL como chave** dá a cada recurso seu próprio cache, de graça.
 - Um **TTL** (validade via `Date.now()`) cura o cache eterno — e mora dentro da estratégia, não do decorator.
 - O vocabulário do mercado: **cache first** (a nossa), **network first**, **stale-while-revalidate** — os mesmos nomes que os Service Workers usam para levar o cache ao app inteiro (Cap. 20).
 - Cache é onipresente no mundo real: navegador, CDNs, feeds de apps, DNS, processador. Velocidade é funcionalidade.
-

Exercícios do Capítulo 19

1. Implemente o **tema persistente** (seção 19.4): um botão que troca o tema e o salva no `localStorage`, e a restauração ao carregar a página.
2. Com o console aberto, navegue pelas páginas de personagens e **anote os tempos** de `[SERVIDOR]` e `[CACHE]`. Calcule quantas vezes o cache é mais rápido.
3. Troque a estratégia do `apiCache.js` para `Memoria`, recarregue com F5 e explique a diferença de comportamento em relação à `LocalStorage`.
4. Crie uma terceira estratégia, `SessionStorage`, cumprindo o contrato `has / get / set`, e plugue-a no decorator.
5. Evolua a estratégia `LocalStorage` para a versão **com validade** (seção 19.7). Teste com `VALIDADE_MS` de 10 segundos e observe o cache expirar.
6. Implemente o **rascunho de formulário** com `sessionStorage` (seção 19.9) na página de contato.
7. **Desafio:** crie uma função `limparCache()` que remove do `localStorage` apenas as chaves que começam com `http` (as do cache de API), preservando outras como o `tema`. Dica: percorra `Object.keys(localStorage)`.
8. **Desafio:** transforme o `apiCache.js` em **network first**: tente a rede e, no `catch`, devolva o que houver na estratégia. Em qual cenário essa versão é superior à nossa?
9. **Reflexão:** para um site de notícias, qual estratégia você escolheria — cache first, network first ou stale-while-revalidate? Justifique pelo compromisso entre velocidade e frescor.

Capítulo 20 — Service Workers e PWA: o app que funciona offline

No capítulo anterior aprendemos a guardar **dados** no navegador e a cachear respostas de API para economizar rede. Mas ficou uma pergunta no ar: e se pudéssemos guardar o **próprio aplicativo** — o HTML, o CSS, o JavaScript, as imagens — de modo que ele **abrisse mesmo sem internet**?

É isso que os **Service Workers** tornam possível, e é o passo que transforma o nosso App Livros de um “site” em um **aplicativo instalável** (um PWA). Este é o capítulo mais “avançado” do curso — mas você vai perceber que ele se apoia inteiramente em conceitos que já domina: eventos, callbacks, Promises, `async/await` e a mesma lógica de cache do Capítulo 19.

20.1 O que é um Service Worker

Um **Service Worker** é um arquivo JavaScript especial que roda **em segundo plano, separado** da sua página. Ele funciona como um **intermediário entre a aplicação e a rede** — um “porteiro” por onde passam as requisições.

Compare com o que fizemos até agora:

```
SEM Service Worker:
  página —fetch→ internet

COM Service Worker:
  página —fetch→ [ Service Worker ] → internet
                    |
                    └→ cache (pode responder sem ir à rede!)
```

Como o Service Worker intercepta **toda** requisição, ele pode decidir: “isto eu já tenho no cache, respondo na hora” ou “isto preciso buscar na rede”. É a mesma ideia do `buscarComCache` (Cap. 19), mas num nível muito mais poderoso — agora para **qualquer** recurso do app, não só as chamadas de API.

Características importantes que o tornam especial:

- **Roda separado da página** — continua “vivo” mesmo depois de a aba fechar, e é compartilhado por todas as abas do site.
- **Não tem acesso ao DOM** — ele não desenha nada na tela; sua função é gerenciar rede e cache. Para falar com a página, usa mensagens.

- **É orientado a eventos** — assim como aprendemos no Capítulo 5, ele “escuta” eventos (`install` , `activate` , `fetch`) e reage a eles.

 **Cuidado** — só funciona em HTTPS (ou localhost) Por serem tão poderosos (interceptam requisições!), os Service Workers só são permitidos em conexões seguras: HTTPS em produção, ou `localhost` no desenvolvimento. Aquele nosso Live Server (`localhost`) funciona perfeitamente. Abrir com `file://` não funciona — mais um motivo para sempre usar um servidor, como falamos lá no Capítulo 0.

20.2 A Cache API

Antes do Service Worker em si, precisamos conhecer a ferramenta que ele usa para guardar arquivos: a **Cache API**. Diferente do `localStorage` (que guarda texto chave→valor), a Cache API é feita para guardar **pares de requisição e resposta** — ou seja, arquivos inteiros (páginas, folhas de estilo, scripts, imagens).

```
// abrir (ou criar) um cache com um nome de versão
const cache = await caches.open("app-livros-v1");

// guardar vários arquivos de uma vez
await cache.addAll(["/", "/index.html", "/src/css/microframework.css"]);

// procurar uma resposta no cache
const resposta = await caches.match("/index.html");
```

Repare que já é tudo `async/await` (Cap. 14) — a Cache API é baseada em Promises, então você já sabe conversar com ela. `caches` é um objeto global, como `document` ou `window`.

 **Nos bastidores** — por que o nome tem “v1”? Chamamos o cache de `"app-livros-v1"`. Esse número de versão é uma prática essencial: quando você atualizar o app (novo CSS, novo JS), vai mudar para `"v2"`, o que permite **descartar o cache antigo** e forçar o navegador a guardar os arquivos novos. Sem versionar, o usuário poderia ficar preso a uma versão velha para sempre — o problema de “invalidação de cache” que citamos no Capítulo 19, agora do lado dos arquivos.

20.3 O ciclo de vida: `install`, `activate`, `fetch`

Um Service Worker “vive” reagindo a três eventos principais. Vamos criar o arquivo `sw.js` na raiz do projeto (a localização importa — veremos por quê) e tratar cada evento.

1. `install` — guardar os arquivos essenciais

Dispara **uma vez**, quando o Service Worker é instalado. É o momento de baixar e guardar o “esqueleto” do app (o chamado *app shell*):

```
// sw.js
const CACHE = "app-livros-v1";
const ARQUIVOS = [
  "/",
  "/index.html",
  "/src/css/microframework.css",
  "/src/js/main.js"
  // ...demais módulos e imagens essenciais
];

self.addEventListener("install", (event) => {
  event.waitUntil(
    caches.open(CACHE).then((cache) => cache.addAll(ARQUIVOS))
  );
});
```

Duas novidades de vocabulário:

- `self` — dentro do Service Worker, `self` é o “eu mesmo” (o equivalente ao `window` da página). `self.addEventListener` registra um ouvinte de evento — exatamente como fizemos com botões no Capítulo 5.
- `event.waitUntil(promessa)` — diz ao navegador “não considere a instalação concluída até esta Promise terminar”. Garante que todos os arquivos sejam guardados antes de o Service Worker seguir em frente.

2. `activate` — limpar caches antigos

Dispara quando o Service Worker assume o controle. É o lugar ideal para **apagar versões antigas** do cache:

```
self.addEventListener("activate", (event) => {
  event.waitUntil(
    caches.keys().then((nomes) =>
      Promise.all(
        nomes.filter((nome) => nome !== CACHE)
          .map((nome) => caches.delete(nome))
      )
    )
  );
});
```

Traduzindo a lógica: pegue os nomes de todos os caches, **filtre** (Cap. 18) os que **não** são a versão atual, e **apague** cada um. Quando você subir a **v2**, essa função limpa a **v1** automaticamente.

3. **fetch** — interceptar as requisições

O evento estrela. Dispara a **cada requisição** que a página faz. Aqui decidimos: responder do cache ou ir à rede?

```
self.addEventListener("fetch", (event) => {
  event.respondWith(
    caches.match(event.request).then((resposta) => {
      return resposta || fetch(event.request);
    })
  );
});
```

Leia o miolo com carinho: **resposta || fetch(event.request)**. Se **caches.match** achou uma resposta no cache, usamos ela (sem rede!). Se não achou (**undefined**, que é “falso”), o **||** vai buscar na rede. É **o mesmíssimo padrão** do nosso roteador — lembra de **mapaDeRotas[hash] || paginaNaoEncontrada** (Cap. 10)? Você já pensava assim desde a Parte IV; agora aplica a estratégia à rede inteira.



Montando o quebra-cabeça Note como cada peça deste Service Worker é um reencontro: **addEventListener** (Cap. 5), **callbacks** e **arrow functions** (Cap. 3), **Promises** e **.then()** (Cap. 13), **async / Cache API** (Cap. 14), o operador **||** para fallback (Cap. 10) e o **.filter()** / **.map()** (Caps. 9 e 18). Não há nada realmente novo aqui — há a **combinação** de tudo o que você aprendeu, aplicada a um problema novo. É assim que conhecimento sólido rende juros.

20.4 Registrando o Service Worker na aplicação

O arquivo `sw.js` sozinho não faz nada — a página precisa **registrá-lo**. Fazemos isso no `main.js` (Cap. 10), com uma verificação de suporte:

```
// no final do main.js
if ("serviceWorker" in navigator) {
  window.addEventListener("load", () => {
    navigator.serviceWorker
      .register("/sw.js")
      .then((reg) => console.log("Service Worker registrado:", reg.scope))
      .catch((erro) => console.error("Falha ao registrar:", erro));
  });
}
```

Analisando:

- `if ("serviceWorker" in navigator)` — checa se o navegador suporta a tecnologia antes de usá-la. Isso se chama *melhoria progressiva*: em navegadores antigos, o app continua funcionando normalmente (só sem o modo offline).
- `navigator.serviceWorker.register("/sw.js")` — registra nosso arquivo. Retorna uma Promise, então encadeamos `.then()` / `.catch()` (Cap. 13).
- Registramos após o evento `load` para não competir com o carregamento inicial da página.

 **Cuidado** — a localização do `sw.js` define o “escopo” O Service Worker só controla requisições **dentro da pasta onde ele está** (e subpastas). Isso se chama **escopo**. Se você colocar o `sw.js` na raiz do projeto, ele controla o site inteiro (`/`). Se colocá-lo em `/src/js/`, ele só controlaria `/src/js/...`. **Por isso o `sw.js` fica na raiz** — para “enxergar” tudo. É a regra que mais confunde iniciantes; anote-a.

20.5 Testando no navegador

Como saber se funcionou? As DevTools têm uma seção dedicada:

1. Abra `F12` → aba **Application** (ou *Aplicativo*).
2. No menu lateral, clique em **Service Workers**. Você verá o seu registrado, com status “activated and running”.
3. Em **Cache Storage**, veja os arquivos que foram guardados no `app-livros-v1`.
4. O teste definitivo: marque a caixa **Offline** (ainda em Application → Service Workers, ou na aba Network) e **recarregue a página**. Se tudo estiver certo, **o App Livros abre mesmo**

sem internet — servido inteiramente do cache. 🎉

💡 **Nos bastidores — o “pulo” da atualização** Um comportamento que assusta no começo: depois de mudar o `sw.js`, o navegador **não** aplica a nova versão de imediato — o Service Worker antigo continua no comando até todas as abas do site fecharem. Durante o desenvolvimento, marque **“Update on reload”** na aba Application → Service Workers para forçar a atualização a cada recarga. Isso evita horas de confusão do tipo “mudei o código e nada muda”.

20.6 Estratégias de cache

No `fetch` acima usamos a estratégia mais simples, **“cache primeiro”**. Mas, como vimos no Capítulo 19, existem outras, cada uma com seu compromisso entre **velocidade** e **frescor**:

Estratégia	Como funciona	Boa para
Cache primeiro	tenta o cache; só vai à rede se faltar	arquivos que mudam pouco (CSS, JS, logos)
Rede primeiro	tenta a rede; cai no cache se estiver offline	dados que mudam sempre (API de livros)
Stale-while-revalidate	responde do cache na hora e atualiza em segundo plano	o melhor de percepção de velocidade

Um exemplo de **“rede primeiro”**, ideal para as chamadas de API do App Livros (onde queremos dados novos, mas com o cache como rede de segurança offline):

```
// dentro do fetch, para requisições à API:
event.respondWith(
  fetch(event.request)
    .then((resposta) => {
      // deu certo na rede: atualiza o cache e devolve
      const copia = resposta.clone();
      caches.open(CACHE).then((cache) => cache.put(event.request, copia));
      return resposta;
    })
    .catch(() => caches.match(event.request)) // offline: usa o cache
);
```

Numa aplicação real, você combina estratégias: “cache primeiro” para os arquivos do app (o *shell*) e “rede primeiro” para os dados da API. O Service Worker pode inspecionar `event.request.url`

e escolher a estratégia conforme o recurso.

✖ **Montando o quebra-cabeça** Percebe a simetria? No Capítulo 19, o cache de **dados** (via `localStorage`) economizava as chamadas de API. Aqui, o cache de **arquivos** (via Service Worker) permite o app abrir offline. Juntas, as duas camadas cobrem tudo: dados e aplicação, ambos disponíveis sem rede. É a diferença entre um site que “quebra sem internet” e um aplicativo de verdade.

20.7 O toque final: o manifesto e o PWA

Falta uma peça para o App Livros virar um **PWA** (*Progressive Web App*) — um app **instalável**, com ícone na tela inicial e abertura em tela cheia. Essa peça é um pequeno arquivo de configuração: o **manifesto**.

Crie `manifest.json` na raiz do projeto:

```
{
  "name": "App Livros",
  "short_name": "Livros",
  "start_url": "/index.html",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#3b82f6",
  "icons": [
    { "src": "/src/img/icone-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "/src/img/icone-512.png", "sizes": "512x512", "type": "image/png" }
  ]
}
```

Cada campo tem um papel:

- **name** / **short_name** — o nome completo e o nome curto (para o ícone).
- **start_url** — a página que abre quando o app é iniciado.
- **display: "standalone"** — abre **sem a barra do navegador**, com cara de app nativo.
- **theme_color** — a cor da barra de status (usamos o azul primário do nosso microframework — Cap. 11).
- **icons** — os ícones do app em vários tamanhos.

E ligamos o manifesto no `<head>` do `index.html`:

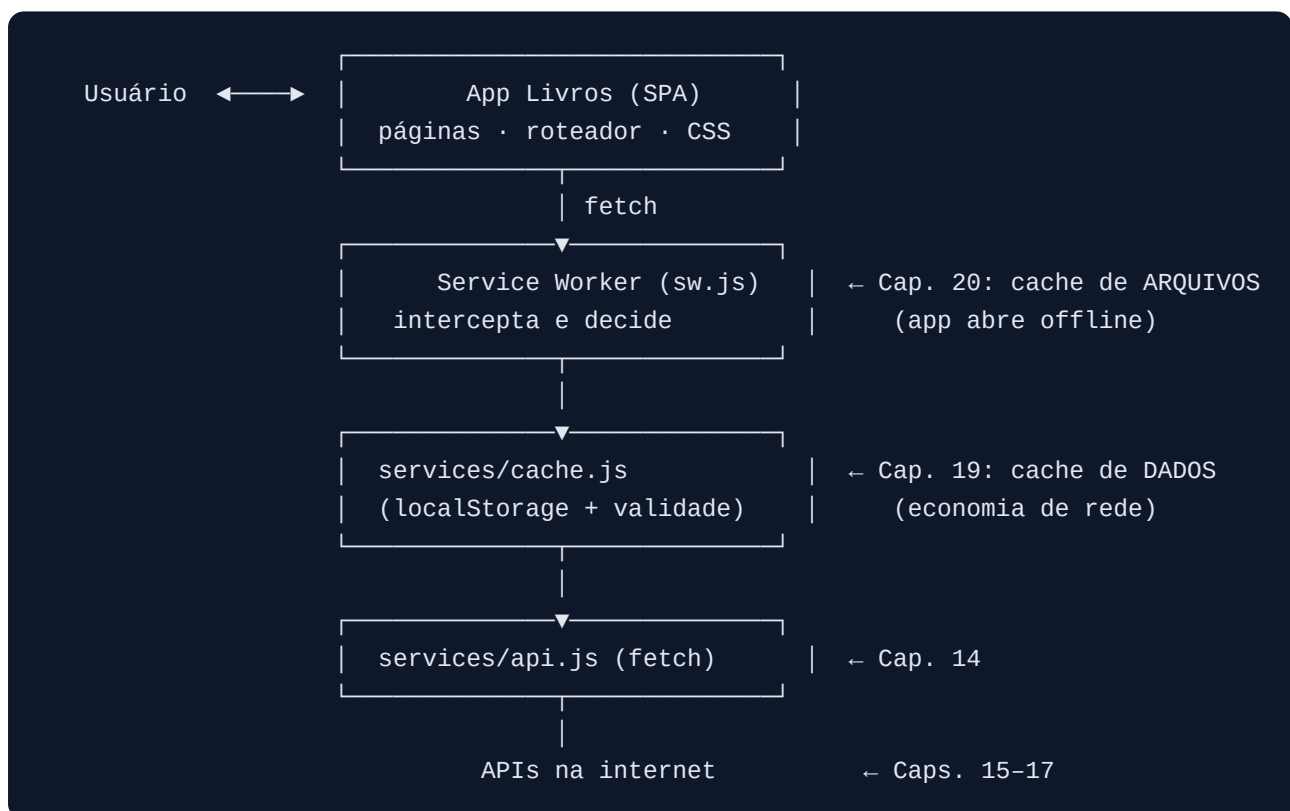
```
<link rel="manifest" href="/manifest.json">
<meta name="theme-color" content="#3b82f6">
```

Pronto. Com um **Service Worker ativo** + um **manifesto válido** + **HTTPS**, o navegador passa a oferecer um botão **“Instalar”** na barra de endereço. O usuário clica, e o App Livros ganha um ícone na área de trabalho ou na tela do celular, abrindo em tela cheia como qualquer aplicativo. Você cruzou a fronteira entre “site” e “aplicativo”.

💡 **Nos bastidores — os critérios de “instalável”** Para o navegador oferecer a instalação, ele checa alguns requisitos (“critérios de instalabilidade”): servir em HTTPS, ter um manifesto com **name**, **icons** (pelo menos 192px e 512px) e **start_url**, e ter um Service Worker registrado. As DevTools (aba Application → Manifest) apontam exatamente o que falta — use-as como checklist.

20.8 O quadro completo

Vale ver, em uma imagem mental, tudo o que a Parte VI construiu sobre o App Livros já pronto:



Da tela ao servidor, cada camada tem uma responsabilidade única, e você entende **todas**. Isso é uma aplicação web moderna, completa, construída peça por peça — sem frameworks, sem

mágica.

Recapitulando

- Um **Service Worker** é um script que roda em segundo plano e **intercepta as requisições** da aplicação, funcionando como um porteiro entre app e rede.
- Só funciona em **HTTPS** ou **localhost**, e seu **escopo** depende de **onde o arquivo está** (por isso, na raiz).
- A **Cache API** (`caches.open`, `addAll`, `match`) guarda **arquivos inteiros**; versione o nome do cache (`v1`, `v2`) para poder atualizá-lo.
- O ciclo de vida tem três eventos: **install** (guarda o *app shell*), **activate** (limpa caches antigos) e **fetch** (decide cache × rede).
- Registramos o SW com `navigator.serviceWorker.register("/sw.js")`, com verificação de suporte (melhoria progressiva).
- Existem **estratégias** de cache (cache primeiro, rede primeiro, stale-while-revalidate) — combine-as por tipo de recurso.
- Um **manifesto** + Service Worker + HTTPS transformam o site num **PWA** instalável, que abre offline e em tela cheia.

Exercícios do Capítulo 20

1. Crie o `sw.js` na raiz com os eventos **install** e **fetch** (estratégia “cache primeiro”) e registre-o no `main.js`. Confirme na aba **Application** que ele está “activated”.
2. Adicione ao **ARQUIVOS** todos os módulos JS e o CSS do App Livros. Marque **Offline** nas DevTools e recarregue: o app deve abrir sem internet.
3. Implemente o evento **activate** que limpa caches antigos. Suba a versão do cache para `"app-livros-v2"` e confirme que a `v1` é apagada.
4. Crie o `manifest.json` e ligue-o no `index.html`. Verifique na aba Application → **Manifest** se o navegador considera o app instalável (e o que falta, se não).
5. **Desafio de estratégia:** faça o **fetch** do Service Worker usar “cache primeiro” para arquivos `.css` / `.js` e “rede primeiro” para as chamadas de API (dica: cheque `event.request.url`).
6. **Reflexão:** explique, com suas palavras, a diferença entre o cache de **dados** do Capítulo 19 e o cache de **arquivos** deste capítulo. Por que precisamos dos dois?

Apêndice A — Referência rápida

Um cartão de consulta com o que mais usamos no App Livros. Volte aqui sempre que precisar relembrar uma sintaxe.

A.1 Variáveis e tipos

```
const nome = "Maria";    // não reatribuível (padrão)
let contador = 0;        // reatribuível
// evite: var

typeof valor;            // descobre o tipo
`Olá, ${nome}!`          // template string (interpolação)
```

A.2 Operadores

```
===  !==                // comparação estrita (use sempre estes)
<  >  <=  >=           // relacionais
&&  ||  !               // E, OU, NÃO
cond ? a : b             // ternário
valor || padrao          // valor, ou 'padrao' se valor for "falso"
```

A.3 Arrays

Método	O que faz
<code>arr.push(x)</code>	adiciona no fim
<code>arr.pop()</code>	remove do fim (retorna o item)
<code>arr.unshift(x)</code>	adiciona no início
<code>arr.shift()</code>	remove do início (retorna o item)
<code>arr.slice(i, f)</code>	cópia de um pedaço (fim não incluído)
<code>arr.map(fn)</code>	transforma cada item → novo array
<code>arr.join(sep)</code>	junta os itens num texto
<code>arr.length</code>	quantidade de itens
<code>arr[i]</code>	acessa por índice (começa em 0)

```
// padrão de renderização do projeto
let html = "";
for (const item of lista) {
  html += `<li>${item.nome}</li>`;
}
elemento.innerHTML = html;
```

A.4 Objetos

```
const pessoa = { nome: "Ana", idade: 28 };
pessoa.nome;           // notação de ponto
pessoa["idade"];        // notação de colchete (útil com variável)
pessoa.email = "...";   // adiciona/modifica propriedade

// array de objetos (a estrutura rainha do front-end)
const rotas = [
  { url: "#home", label: "Home", pagina: home }
];
rotas[0].label;         // "Home"

// desestruturação
const { totalPaginas, livros } = resultado;
```

A.5 Funções

```
function somar(a, b) { return a + b; }           // declaração
const somar = function (a, b) { return a+b; };   // expressão
const somar = (a, b) => a + b;                   // arrow (retorno implícito)
const dobro = x => x * 2;                         // 1 parâmetro, sem ( )
const oi = () => console.log("oi");               // sem parâmetros

// callback
botao.addEventListener("click", () => fazerAlgo());
```

A.6 Loops

```
for (let i = 0; i < lista.length; i++) { ... } // clássico (com índice)
for (const item of lista) { ... }              // valores (arrays)
for (const chave in objeto) { ... }           // chaves (objetos)
while (condicao) { ... }                       // enquanto for verdade
do { ... } while (condicao);                   // ao menos uma vez

break;    // encerra o loop
continue; // pula para a próxima volta
```

A.7 DOM

```
document.getElementById("id");
document.querySelector(".classe");           // # id, . classe, tag
document.querySelectorAll(".card");          // vários

el.textContent = "texto";                     // texto puro
el.innerHTML = "<b>html</b>";                  // interpreta HTML
el.value;                                     // valor de inputs
el.style.color = "blue";
el.classList.add("x");                        // .remove("x"), .toggle("x")

document.createElement("li");
pai.appendChild(filho);
el.remove();

el.addEventListener("click", callback);
el.dataset.id;                               // lê data-id="..."
```

Eventos comuns: `click`, `submit`, `input`, `blur`, `mouseover`, `keyup`.

```
form.addEventListener("submit", (event) => {
  event.preventDefault();          // impede recarregar (essencial na SPA)
  // ...
});
```

A.8 SPA e roteamento

```
window.location.hash;          // "#sobre"
window.location.search;        // "?a=1&b=2"
new URLSearchParams(location.search).get("a");

window.addEventListener("hashchange", renderizar);

// mapa de rotas
const mapa = {};
for (const rota of rotas) { mapa[rota.url] = rota; }
const rota = mapa[hash] || paginaNaoEncontrada;
```

A.9 Módulos

```
// export default (uma coisa principal)
export default funcao;
import qualquerNome from "./arquivo.js";

// export nomeado (várias coisas)
export { fnA, fnB };
import { fnA, fnB } from "./arquivo.js";
```

No HTML: `<script type="module" src="main.js"></script>` (exige servidor).

A.10 Assíncrono e fetch

```
// async / await (forma preferida)
async function buscar(url) {
  try {
    const resposta = await fetch(url);
    if (!resposta.ok) throw new Error(`Erro ${resposta.status}`);
    return resposta.json();
  } catch (erro) {
    console.error(erro);
  } finally {
    // limpeza que roda sempre (ex.: esconder "Carregando...")
  }
}

// Promise com .then / .catch (forma clássica)
fetch(url)
  .then(r => r.json())
  .then(dados => console.log(dados))
  .catch(err => console.error(err));
```

A.11 Convenção BEM (CSS)

```
.bem-bloco           /* componente      */
.bem-bloco__elemento /* parte do bloco      */
.bem-bloco--modificador /* variação/estado    */
```

```
:root { --bem-primary: #3b82f6; } /* variável CSS */
.x { color: var(--bem-primary); } /* uso          */
[data-theme="noite"] { --bem-bg: #0f172a; } /* tema         */
```

Apêndice B — Gabarito comentado dos exercícios

Aqui estão soluções de referência para os exercícios de programação mais importantes de cada capítulo. Tente resolver sozinho **antes** de consultar — o aprendizado está na tentativa, não na resposta pronta. Há muitas formas corretas; estas são apenas uma delas.

Capítulo 1 — Variáveis e condicionais

Ex. 2 — Classificar nota:

```
let nota = 8;
if (nota >= 7) {
  console.log("Aprovado");
} else if (nota >= 5) {
  console.log("Recuperação");
} else {
  console.log("Reprovado");
}
```

Ex. 3 — Aluno aprovado (tabela verdade):

```
let nota = 7.5;
let faltas = 15;
if (nota >= 7.0 && faltas <= 20) {
  console.log("Aprovado");
} else {
  console.log("Reprovado");
}
// Teste as 4 combinações: (nota alta/baixa) x (faltas ok/altas)
```

Ex. 6 — Segurança do banco com ternário:

```
const nomeValido = true;
const senhaCorreta = true;
console.log(nomeValido && senhaCorreta ? "Acesso permitido" : "Acesso negado");
```

Capítulo 2 — Arrays e objetos

Ex. 1 — Pannel de fila com prioridade:

```
const fila = [];  
  
function chegou(idade) {  
  if (idade > 65) {  
    fila.unshift(idade); // prioridade: vai para o início  
  } else {  
    fila.push(idade);    // fim da fila  
  }  
}  
  
function atender() {  
  return fila.shift(); // remove e retorna o primeiro  
}  
  
chegou(30); chegou(70); chegou(40);  
console.log(fila);      // [70, 30, 40]  
console.log(atender()); // 70
```

Ex. 4 — Ano do livro “Python”:

```
const livros = [{ titulo: "PHP", ano: 1995 }, { titulo: "Python", ano: 1991 }];  
console.log(livros[1].ano); // 1991
```

Capítulo 3 — Funções

Ex. 1 e 2 — Calculadora (tradicional e arrow):

```
// tradicionais  
function somar(a, b)      { return a + b; }  
function subtrair(a, b)   { return a - b; }  
function multiplicar(a, b) { return a * b; }  
function dividir(a, b)    { return a / b; }  
  
// como arrow functions  
const somar      = (a, b) => a + b;  
const subtrair   = (a, b) => a - b;  
const multiplicar = (a, b) => a * b;  
const dividir    = (a, b) => a / b;
```


Ex. 4 — Callback:

```
function processar(valor, callback) {
  callback(valor);
}
processar("olá", (v) => console.log(v.toUpperCase())); // "OLÁ"
```

Ex. 5 — Recursão:

```
function contagemRegressiva(n) {
  console.log(n);
  if (n <= 0) return;
  contagemRegressiva(n - 1);
}
contagemRegressiva(5); // 5, 4, 3, 2, 1, 0
```

Capítulo 4 — Loops

Ex. 1 — Inverter array:

```
const original = [1, 2, 3, 4, 5];
const inverso = [];
for (let i = original.length - 1; i >= 0; i--) {
  inverso.push(original[i]);
}
console.log(inverso); // [5, 4, 3, 2, 1]
```

Ex. 2 — Maior número:

```
const numeros = [5, 3, 9, 1, 6];
let max = numeros[0];
for (let i = 1; i < numeros.length; i++) {
  if (numeros[i] > max) max = numeros[i];
}
console.log(max); // 9
```

Ex. 3 — Contar caracteres:

```
const palavra = "banana";
const alvo = "a";
let conta = 0;
for (const letra of palavra) {
  if (letra === alvo) conta++;
}
console.log(`'${alvo}' aparece ${conta} vezes.`); // 3
```

Ex. 4 — Números pares:

```
const N = 5;
const pares = [];
for (let i = 1; i <= N; i++) {
  pares.push(i * 2);
}
console.log(pares); // [2, 4, 6, 8, 10]
```

Capítulo 5 — DOM e eventos

Ex. 3 — Formulário que adiciona `<li>`:

```
const form = document.getElementById("form");
form.addEventListener("submit", (event) => {
  event.preventDefault();
  const valor = document.getElementById("campo").value;
  const li = document.createElement("li");
  li.textContent = valor;
  document.getElementById("lista").appendChild(li);
});
```

Ex. 4 — Renderizar produtos:

```
const produtos = [
  { nome: "Produto 1", preco: "R$ 10,00" },
  { nome: "Produto 2", preco: "R$ 20,00" }
];
let html = "";
for (const p of produtos) {
  html += `<div class="card">${p.nome} - ${p.preco}</div>`;
}
document.getElementById("app").innerHTML = html;
```

Capítulo 7 — Primeiro roteador

Ex. 1 — SPA mínima:

```
const app = document.getElementById("app");
const paginas = {
  "#home": "<h1>Home</h1>",
  "#sobre": "<h1>Sobre</h1>",
  "#contato": "<h1>Contato</h1>"
};
function renderizar() {
  const hash = window.location.hash || "#home";
  app.innerHTML = paginas[hash] || "<h1>404</h1>";
}
renderizar();
window.addEventListener("hashchange", renderizar);
```

(Repare: já usamos um “mapa” aqui, antecipando o Capítulo 10!)

Capítulo 9 — Rotas e menu dinâmico

Ex. 4 — Cards de serviço com `.map()`:

```
function servicos(app) {
  const cards = detalhes.map(d => `
    <div class="bem-card">
      
      <div class="bem-card__body">
        <h3 class="bem-card__title">${d.titulo}</h3>
        <p>${d.descricao}</p>
      </div>
    </div>`);
  app.innerHTML = `<div class="bem-grid-auto">${cards}</div>`;
}
```

Capítulo 13 — Promises e fetch

Ex. 1 — Fetch do CEP:

```
fetch("https://viacep.com.br/ws/01001000/json/")
  .then(r => r.json())
  .then(dados => {
    console.log(dados.localidade); // São Paulo
    console.log(dados.logradouro); // Praça da Sé
  })
  .catch(err => console.error(err));
```

Ex. 3 — Promise manual:

```
const espera = new Promise((resolve) => {
  setTimeout(() => resolve("Pronto!"), 2000);
});
espera.then(msg => console.log(msg)); // "Pronto!" após 2s
```

Capítulo 14 — async/await

Ex. 1 — Reescrever com async/await:

```
async function buscarDados() {
  try {
    const resposta = await fetch("https://viacep.com.br/ws/01001000/json/");
    const dados = await resposta.json();
    console.log(dados);
  } catch (erro) {
    console.error(erro);
  }
}
```

Capítulo 15 — Formulário CEP

Ex. 2 — Validação de CEP inexistente:

```
const dados = await buscarDados(`https://viacep.com.br/ws/${event.target.value}/json/`);
if (dados.erro) {
  Swal.fire("CEP não encontrado");
  document.getElementById("logradouro").value = "";
  document.getElementById("bairro").value = "";
  document.getElementById("localidade").value = "";
  document.getElementById("estado").value = "";
  return;
}
// ...preenche normalmente
```

Capítulo 16 — API de Livros

Ex. 3 — Fechar o modal ao clicar no fundo:

```
document.getElementById("modalDetalhes").addEventListener("click", (event) => {
  // fecha só se o clique foi no fundo (não no diálogo)
  if (event.target.id === "modalDetalhes") {
    fecharDetalhes();
  }
});
```

Ex. 4 — Placeholder para imagem ausente:

```
const imagem = livro.imagem || "src/img/sem-capa.png";
// use ${imagem} no template do card
```

Capítulo 17 — Rick and Morty

Ex. 4 — Busca por nome:

```
// no serviço:
async function buscarListaPersonagens(pagina, termo = "") {
  const url = `${BASE_URL}?page=${pagina}&name=${termo}`;
  const dados = await buscarDados(url);
  // ...resto igual
}

// na página: um <input id="busca"> e
document.getElementById("busca").addEventListener("input", (e) => {
  termoBusca = e.target.value;
  paginaAtual = 1;
  renderizarPagina();
});
```

Nota final: os exercícios de “reflexão” não têm resposta única — a intenção é fazer você **verbalizar** o entendimento. Escreva suas respostas com suas próprias palavras; ensinar (mesmo a um caderno) é a melhor forma de aprender.